Flaris Language Reference

The Flaris programming language · flaris-lang.org · github.com/flaris-lang/flaris · flaris-lang.org/discord

Version 1.0.1.1 **Generated** 2026-08-21 **License** see repository

Table of Contents

- R1 - CLI Reference
- R2 - Type System
- R3 - Built-in Functions
- R4 - The Exception class
- R5 - Operator Precedence
- R6 - VM Limits
- R7 - Performance Model
- R8 - Standard Library
- R9 - Debug Reference
- R10 - Writing Fast Flaris Code
- R11 - JIT Compilation
- R12 - Static Analyzer

R1 - CLI Reference

The Flaris VM binary is `flarisvm`.

Invocation Modes

Command	Description
`flarisvm <file.fls> [options]`	Compile and run source file
`flarisvm - [options]`	Compile and run source from **stdin**
`flarisvm --compile <input.fls> <output.flx> [options]`	Compile source to bytecode
`flarisvm --compile - <output.flx> [options]`	Compile source from **stdin** to bytecode
`flarisvm --exec <file.flx> [options]`	Execute compiled bytecode
`flarisvm --exec - [options]`	Execute bytecode from **stdin**
`flarisvm --disasm <file.flx>`	Disassemble bytecode (debug view)
`flarisvm --eval '<code>' [options]`	Run code directly from string
`flarisvm --sha256 <file.flx>`	Print the whole-file SHA-256 (identical to `sha256sum`) - the value to pin via `library(file, version, hash)`
`flarisvm --sig-info <file.flx>`	Print the embedded signature status, machine-readable: `unsigned`, or `signed|<trusted|valid|invalid>|<pubkey-hex>|<signer>` (see *Package signing*)
`flarisvm --import-key <name> <pubkey>`	Add a 64-hex Ed25519 public key (with a label) to `~/.flaris/trusted_keys`

Command	Description
`flarisvm --check <file.fls>`	Compile without running: full analyzer diagnostics plus a JIT eligibility report - see R11 and R12
`flarisvm --format <file.fls|->`	Print the file in canonical layout on stdout - see *Formatting* below
`flarisvm --format-write <file.fls>`	Rewrite the file in canonical layout, in place
`flarisvm --format-check <file.fls>`	Exit 1 if the file is not already formatted (prints nothing on success)
`flarisvm -Wno-<name> …`	Silence one analyzer warning, e.g. `-Wno-unused-symbol` - see R12
`flarisvm -Werror …`	Treat any analyzer warning as an error
`flarisvm -w …`	Silence all analyzer warnings
`flarisvm --diagnostics json …`	Emit diagnostics as a JSON array on stdout
`flarisvm --version`	Print version string
`flarisvm --compile <input.fls> <output.flx> --bundle [options]`	Compile and bundle every module the program imports (transitively) into a self-contained `.flx`
`flarisvm --compile <input.fls> <output.flx> --bundle='<file1.flx>;...' [options]`	Compile and bundle the named modules into a self-contained `.flx`
`flarisvm --embed <input.fls|.flx> <output> [options]`	Build a self-contained native executable (program + runtime in one file). See *Self-contained executables*

Formatting

`--format` re-lays out a source file: indentation, spacing, brace placement and blank lines become canonical, and nothing else changes. It works on stdin (`--format -`), which is what the VS Code extension uses so an unsaved buffer formats as typed.

Flag	Effect
`--indent=<n>`	Spaces per indent level (default 4, 1-16)
`--width=<n>`	Right margin at which argument lists and literals wrap (default 100, 40-400)
`--tabs`	Indent with one tab per level instead of spaces

The formatter works on the token stream, never the AST, and every token is copied verbatim out of the source - so string literals, escapes, `"""` blocks and comments come through byte-for-byte. Before emitting anything it re-tokenizes its own output and compares it against the input token stream; if they differ it writes nothing and exits non-zero. A format can therefore only move whitespace, never change what the program means.

Layout is canonical with one exception: a construct the author put on a single line keeps that line if it still fits the margin. That covers one-line blocks (`fn sq(x) { return x * x; }`), one-line `case` clauses and brace-less bodies (`if (!ok) return;`). Everything wider than the margin, and everything the author already spread across lines, is laid out by the formatter.

CRLF line endings and a leading UTF-8 BOM are preserved. Output always ends in exactly one newline and never contains trailing whitespace.

```
flarisvm --format-check src/app.fls   # CI gate: non-zero if unformatted
flarisvm --format-write src/app.fls   # rewrite in place
```

Flags - Disablers (default ON)

Flag	Effect
`--no-opt`	Disable compiler optimizations
`--strip`	Disable debug symbols (smaller, faster bytecode)

Flag	Effect
`--small`	Extra size optimizations for compiled `.flx`; also removes string symbol names
`--no-verify`	Disable FFI library (Ffi.Load) SHA-256 checks

Flags - Enablers (default OFF)

Flag	Effect
`--unsafe`	Enable unsafe code and FFI
`--mem`	Memory report at shutdown: peak RSS, slab footprint, peak/total object counts, heap bytes, block regions, and leak count. Exits non-zero if leaks are detected.
`--stats`	Print VM statistics after execution (implies `--time`)
`--time`	Print timing report
`--verbose`	Verbose output (also enables the timing report)
`--list-keys`	Print the built-in and trusted signing keys, then exit
`--trace`	Very verbose / print disassembled bytecode before execution
`--jit-disable`	Turn the JIT off. It is ON by default: eligible functions get JIT IR at compile time and native code at run time (ARM64 and x86-64)
`--debug`	Stop at the start of `Main` and open the debugger prompt; disables the JIT (see R9)
`--dap=<port>`	Serve the Debug Adapter Protocol on loopback for an editor; implies `--debug`

Diagnostics are colored only when stderr is a VT-capable TTY; setting the `NO_COLOR` environment variable (non-empty, no-color.org convention) forces plain output.

Exit Codes

Code	Meaning
0	Success
`fn Main` return	A numeric return value from `Main` becomes the process exit code
exception code	An uncaught exception exits with the exception's numeric code
1	Tokenizer error
2	Compile error (bad source on the CLI or in a compiled unit)
3	Assert failure (`Assert` / `AssertEq` or an internal invariant)
4	File error (missing or unreadable source/bytecode)
5	Debugger quit (`q`)
6	Reserved (analyzer failures currently surface as code 2)
7	Fingerprint/pin mismatch on a signed or pinned import
8	Reserved

VM Tuning Flags

These override the compiled-in defaults. Values must be within min/max ranges listed in R6.

Flag	Effect
`--stack=<int>`	Override evaluation stack size
`--fifo=<int>`	Override object cache (FIFO) size
`--slabs=<int>`	Override slab allocator count
`--fibers=<int>`	Override maximum fiber count
`--frames=<int>`	Override maximum call-frames
`--io-threads=<int>`	Worker threads for the I/O pool (1-16). Default: half the logical CPUs, minimum 4. Raise it if `--stats` reports jobs run inline.

Imports

Flag	Effect
`--min-version=<M.m.p.b>`	Set the minimum runtime version stamped into compiled `.flx` output. Compile mode only; silently ignored at runtime. Examples: `--min-version=1.0`, `--min-version=2.1.3`. Default: `1.0.0.0`
`--libs=<path>`	Set default library loading-path (otherwise `.`) for VM to load libraries for later resolving imports
`--sign=<hex|keyfile>`	Compile mode only. Embed an Ed25519 signature in the `.flx`. Accepts the 128-hex-char secret key inline or a path to a file containing it. See *Package signing*.
`--require-signed`	Runtime. Refuse to load any `.flx` that is unsigned, has an invalid signature, or is signed by a key not in `~/.flaris/trusted_keys`.

Module name resolution (case-sensitive)

`library(name, …)` (and `VM.Import`) resolve `name` to a file by **exact, case-sensitive match**: `library("Signals")` → `Signals.flx` (on the `--libs` path), `library("./x/Y")` → `./x/Y.flx`, a URL → that exact path. The name is never lowercased, so `library("signals")` finds `Signals.flx` only on case-insensitive filesystems (macOS/Windows) and **fails on Linux**. Keep the import string, the on-disk filename, and any pinned name byte-identical (convention: capitalized first letter). Imported symbol names must also match the library's `export` exactly - `{ Jwt }` ≠ `{ JWT }`.

Search order for a non-path name: (1) the name as a path relative to the working directory, (2) each `;`-separated entry of `--libs`, (3) the per-user install dir - `~/.flaris/libs` (POSIX) / `%USERPROFILE%\.flaris\libs` (Windows), or `$FLARIS_LIBS` if set. The first version-satisfying candidate wins (`--libs` beats the global dir). Step 3 lets a package manager install once into the home directory and have `flarisvm script.fls` resolve imports with no `--libs` flag.

Library integrity (pinning)

`library(name, version, hash)` (and `VM.Import(name, version, hash)`) verify the loaded module against `hash` - the **whole-file SHA-256** of the `.flx`, identical to `sha256sum file.flx` and to `flarisvm --sha256 file.flx`. The hash is recomputed over the actual bytes loaded, so it detects any tampering of a downloaded library. Supply it from a trusted source (a registry, a README, or `flaris.json`).

- An optional `sha256:` prefix is accepted, so a value recorded by flarispm in `flaris.json` (`"sha256:<hex>"`) can be pasted verbatim.
- A mismatch halts the VM (fail-closed) and is **not** affected by `--no-verify` (`--no-verify` only disables the FFI shared-library check in `Ffi.Load`).
- Omit the hash to load without pinning.
- No hash is embedded in the `.flx` itself: a self-recomputed embedded hash would be self-certifying and offer no protection against tampering. The trust anchor is the hash you pass in, delivered out-of-band.

Package signing (Ed25519)

Where the pinned hash above answers *"are these the same bytes I vetted?"* (trust on first use), an **embedded Ed25519 signature** answers *"who produced this?"* - authenticity that survives an untrusted transport, registry, or mirror.

A signature is carried inside the `.flx` header (192-byte header; algorithm + 32-byte public key + 64-byte signature). It covers the SHA-256 of the **entire file with the signature field masked to zero**, so it

authenticates the header metadata (name, version, entry), the constants, and the code together.

Producing a signed `.flx`:

```
# One-time: generate a key pair (keep the secret key offline).
flarisvm --eval 'let kp = Crypto.Ed25519KeyPair();
             File.WriteText("publisher.sk", kp.SecretKey);   // 128 hex chars
             File.WriteText("publisher.pk", kp.PublicKey)'    // 64 hex chars

# Compile and sign (signing is deterministic — the .flx stays reproducible).
flarisvm --compile mylib.fls mylib.flx --sign=publisher.sk
```

The public key is printed at compile time and embedded in the file; verify it later with `Crypto.Ed25519Verify`.

Verifying at load:

Situation	Default (soft)	With `--require-signed`
Unsigned `.flx`	runs (no warning)	**rejected**
Valid signature, signer **in** `trusted_keys`	runs	runs
Valid signature, signer **not** in `trusted_keys`	runs	**rejected** (untrusted signer)
Invalid / tampered signature	**rejected (fail-closed)**	**rejected**

- A **tampered** signed file always fails to load, in both modes - the signature no longer matches the recomputed digest.
- **The official Flaris signing key is compiled into the runtime** as a built-in trust anchor, so the bundled standard library verifies under `--require-signed` with no file to download or configure. The published key is also on the website as a cross-check.
- Additional publishers are trusted via `~/.flaris/trusted_keys` (or `$FLARIS_TRUSTED_KEYS`): one 64-hex public key per line, optionally followed by whitespace and a human-readable label, e.g. `b6e2…79be acme corp`. Blank lines and `#` comments are ignored. This file is **additive** on top of the built-in anchor - add a third-party publisher's key here to trust their packages too. Use `flarisvm --import-key <name> <pubkey>` to append one safely (it dedupes). The label (and the built-in `flaris-lang.org` name) is shown as the *signer* by `flarisvm --disasm` and `flarisvm --sig-info`.
- Signature verification is independent of `--no-verify` and of the `library(..., hash)` pin; use signing for *authenticity* and the hash pin for *exact-artifact* checks.

Security model (trust boundary)

A `.flx` is **executable code, not a sandboxed format**. Running one is equivalent to running a native program: it can read and write files, spawn processes (`Os.Run`), open sockets, and - with `--unsafe` - call arbitrary C via FFI and access raw memory. On load the VM validates bytecode structure (validate chunks, JIT-IR validation, size/count caps) to contain *malformed* input safely, but it is **not** a security sandbox for *malicious* input.

- **Only run `.flx` files you trust.** For third-party libraries, pin the whole-file hash via `library(name, version, hash)` (above) so you execute exactly the artifact you vetted, and/or require a trusted Ed25519 signature (`--require-signed`, see *Package signing*) to authenticate the publisher. These are the primary defenses.

- **The JIT widens the trust surface.** JIT-compiled functions omit the runtime type checks the interpreter performs, so a *tampered* `.flx` run with the JIT on can crash or corrupt memory where the interpreter would raise a clean error. Do not run untrusted bytecode with `--jit-disable`.
- `--unsafe` grants FFI and raw-memory access; never combine it with untrusted code.

Bytecode verification

Before any chunk runs, the loader structurally validates it. This *contains malformed input* - a corrupt or truncated `.flx` is rejected with a clean error instead of crashing - but it is **not** type verification and **not** a sandbox for hostile input (see the trust-boundary note above). Validation runs in three passes over each function chunk and recurses into nested functions (nesting depth is bounded):

1. **Decode & operand checks.** Every instruction is decoded and its operands are range-checked: constant-pool indices must be in range (and property/method keys must actually be strings), local slots must be `< 128` (`MAX_LOCALS`), call and invoke argument counts must be `<= 16` (`MAX_CALL_ARGUMENTS`), and built-in module/function references must exist. Each instruction's decoded length must also match its canonical length, so an opcode cannot be mis-sized. Instruction boundaries are recorded for pass 2.
2. **Branch-target check.** Every jump, loop, branch, `try`, `foreach`/iterator and jump-table target must land exactly on a recorded instruction boundary - control can never jump into the middle of an instruction.
3. **Stack-depth check.** An abstract simulation computes the maximum value-stack depth the chunk can reach, which the VM reserves up front per call frame so the value stack cannot overflow at run time. Any opcode the simulation does not model is rejected (fail-closed) rather than assumed harmless.

Size and count caps also apply: `10 MB` of bytecode and `16,385` constants per function (see *Limits*). A chunk that fails any check is refused at load time; an invalid instruction encountered at run time raises `Exception.IllegalInstruction` (code `16`), a hard failure. These guarantees hold for the interpreter; the JIT omits the interpreter's runtime type checks and so widens the trust surface as noted above.

Bundle files

A bundle is a self-contained `.flx` file produced with `--bundle`. It embeds dependency chunks in a single file with a TOC at the start. At runtime the VM pre-loads all bundled deps before execution begins - no filesystem access occurs for those modules.

Choosing the dependency set:

Form	Packs
`--bundle` (or `--bundle=auto`)	The program's whole import chain: every module it imports, every module those import, and so on
`--bundle='<f1.flx>;<f2.flx>;...'`	Exactly the named files

Both may be given together; a file named explicitly is not packed a second time when the walk also finds it. The walk resolves each import through the same search order the loader uses (the path as written, then each `--libs` segment, then `~/.flaris/libs`) and honours the version requirement, so the bundle contains the same file the import would have loaded from disk. A content pin (`library(name, version, "sha256:...")`) is verified while building, and a URL import is downloaded and packed like any other dependency.

Limitations:

- Named modules have to be readable from a direct path, via a `--libs` folder, or from `~/.flaris/libs`
- A module imported at runtime through a direct `VM.Import(name, version)` call is not bundled: its name is a computed string, invisible to a build-time walk. Ship that `.flx` alongside the program. Only `import ... from library(...)` declarations are followed
- Native libraries opened with `Ffi` are not bundled - they are OS shared objects loaded at runtime. A build that packs a program using `Ffi` warns about it
- An embedded binary's runtime always enforces `--require-signed`, so its bundled deps must be signed by a trusted key
- `--bundle` applies to source input only. A `.flx` given to `--embed` is embedded byte for byte (that is what preserves its signature), so the flag is refused there - compile the bundle first, then embed it

Inspecting a bundle:

```
flarisvm --disasm my_bundle.flx
```

The disassembler shows the bundle TOC, each dep chunk, and the main chunk.

Self-contained executables

`--embed <input> <output>` produces a single native executable that carries both your program and the runtime - one file to distribute, nothing to install on the target machine (comparable to Go's static binaries).

```
# From source, in one step
flarisvm --embed app.fls myapp
./myapp arg1 arg2            # runs standalone — no flaris on PATH, no --libs

# Statically link every library the program imports into the same binary
flarisvm --embed app.fls myapp --bundle

# ...or name the exact files to link
flarisvm --embed app.fls myapp --bundle='./mylib.flx'

# Embed precompiled bytecode directly (embedded as-is; --bundle is refused here)
flarisvm --embed app.flx myapp
```

How it works. The full `flarisvm` compiles the input (or reads the `.flx` directly), then appends the bytecode to the compiler-free `flaris` runtime, followed by a 32-byte settings block and a fixed 24-byte trailer recording the payload's offset and length. On startup any Flaris binary reads that trailer out of its own image; if a payload is present it applies the recorded settings, runs the embedded program, and passes all CLI arguments to the script, otherwise it behaves as the normal VM. The payload rides *after* the executable image, so the OS loader ignores it and a plain file copy still works.

Details:

- **Input** is detected by content (the `FLS2` magic), not by extension: a `.fls` is compiled first, a `.flx` is embedded as-is.

- The same compile-time options as `--compile` apply to a `.fls` input: `--bundle`, `--sign`, `--no-opt`, `--strip`.
- **Runtime settings are chosen at embed time.** A self-contained binary hands every CLI argument to the script, so flags meant for the VM are recorded in the settings block instead: give `--jit-disable`, `--unsafe`, or any VM limit (`--stack=`, `--slabs=`, `--fibers=`, `--frames=`, `--fifo=`, `--io-threads=`) alongside `--embed` and the binary applies them on every start. Nothing is on by default - an embedded program gets JIT or unsafe/FFI mode only if you opted in when building it. Limits are validated at startup against the same ranges as the CLI flags; a tampered or out-of-range value refuses to run rather than clamping. Settings live in the binary, not the bytecode: `--compile` never records them, and the `.flx` payload stays byte-identical whatever settings you choose.
- **Requires the `flaris` runtime stub.** It is resolved next to `flarisvm` first, then on `PATH` (the official installer places both). Only the full `flarisvm` can build embeds; `flaris` itself has no compiler and no `--embed`.
- The embedded runtime has no compiler and no `eval` - a bundled binary ships exactly the bytecode you built.
- Combine with `--bundle` so the shipped binary needs no `~/.flaris/libs` and no `--libs`.

Platform notes. The mechanism is platform-agnostic: ELF, Mach-O, and PE all tolerate the payload as trailing "overlay" bytes past the mapped image, so self-contained binaries run on Linux, macOS (including Apple Silicon, where the runtime's ad-hoc signature covers the stub and the trailing payload is tolerated), and Windows. The only per-platform gap is code-signing for distribution: appending bytes invalidates strict macOS `codesign`/notarization and Windows Authenticode (and an unsigned `.exe` may draw SmartScreen friction, though it still runs).

Common Recipes

```
# Development - run with timing and verbose
flarisvm app.fls --verbose --time

# Production - execute pre-compiled bytecode
flarisvm --exec app.flx

# Compile for production (no debug symbols, optimized)
flarisvm --compile app.fls app.flx --strip

# Enable FFI, skip the FFI shared-library (.so) SHA-256 check
flarisvm app.fls --unsafe --no-verify

# Compile-only: analyzer diagnostics + JIT eligibility report, no execution
flarisvm --check app.fls

# Compile library with explicit version
flarisvm --compile mylib.fls mylib.flx --min-version=1.2.0

# Print the whole-file SHA-256 (== sha256sum) to pin as the third arg to library()
flarisvm --sha256 mylib.flx
```

```
# Compile and embed an Ed25519 signature, then run requiring a trusted signer
flarisvm --compile mylib.fls mylib.flx --sign=publisher.sk
flarisvm --exec mylib.flx --require-signed

# Inspect a package's signature (machine-readable) and trust a publisher
flarisvm --sig-info mylib.flx
flarisvm --import-key "acme corp" <ed25519-pubkey-hex>

# Compile with all imports bundled into a single .flx
flarisvm --compile app.fls app.flx --bundle

# Bundle a hand-picked set, with an explicit version
flarisvm --compile app.fls app.flx --bundle='./data.flx' --min-version=2.0.0

# Inspect bundle contents
flarisvm --disasm app.flx

# Build a single self-contained executable (program + runtime), libs bundled in
flarisvm --embed app.fls app --bundle
./app

# Compile with JIT code embedded (for every eligible function)
flarisvm --compile app.fls app.flx

# Run with JIT active (runs eligible functions as native code)
flarisvm --exec app.flx

# See which functions are JIT-eligible during compilation
flarisvm --verbose app.fls

# Read source from stdin, compile and run
echo 'fn Main() { Console.WriteLine("hello"); }' | flarisvm -

# Compile from stdin to a .flx file
cat app.fls | flarisvm --compile - app.flx

# Execute pre-compiled bytecode piped via stdin
cat app.flx | flarisvm --exec -

# End-to-end pipe: compile and execute without touching the filesystem
cat app.fls | flarisvm --compile - /tmp/app.flx && flarisvm --exec - < /tmp/app.flx
```

R2 - Type System

Lexical Structure

How source text is broken into tokens (the numeric/float literal forms are detailed under Type Limits).

Encoding. Source is UTF-8. A leading UTF-8 BOM (`EF BB BF`) is skipped. Identifiers are ASCII: a leading `_`/letter followed by `_`/letters/digits.

Comments.

```
// line comment — to end of line
/* block comment */
/* block /* comments */ nest */   // fully balanced
```

Numeric literals. Decimal, `0x`/`0X` hex, `0b`/`0B` binary, `0o`/`0O` octal, and floats with a fractional part and/or `e`/`E` exponent (optional `+`/`-`). A digit is required on both sides of the decimal point (`0.5`/`5.0`, never `.5` or `5.`). `_` digit separators are allowed in every base and in the fraction and exponent (`0xFF_FF`, `0b1010_1010`, `1_000.000_5`); they are removed before the value is parsed.

String literals. `"..."` (single line) and `"""..."""` (triple-quoted, spans newlines, verbatim - no escape processing). Inside `"..."` a backslash at end of line is a line continuation (the newline is dropped). A raw newline inside `"..."` is an error - use `\n`, a line continuation, or `"""..."""`. Recognised escapes:

- `\n` `\r` `\t` `\b` `\f` - control characters (newline, return, tab, backspace, form-feed)
- `\\` `\"` `\'` - literal backslash, double quote, single quote
- `\xHH` - one byte from two hex digits
- `\uXXXX` - Unicode code point from exactly four hex digits (max `U+FFFF`)
- `\u{H..H}` - Unicode code point from 1-6 hex digits, up to `U+10FFFF` (e.g. `\u{1F600}` for 😀)

Both `\u` forms map a surrogate value to `U+FFFD`. Use `\u{...}` for code points above `U+FFFF` (emoji, etc.); the four-digit `\uXXXX` cannot reach them. An unknown escape keeps the character literally (`"\q"` is `q`). Because Flaris strings are NUL-terminated at runtime, a `\0` or `\x00` **in a string literal is a compile error** - put binary data in a `Buffer`. String literals are capped at 64 KB of content.

Char literals. `'...'` holds exactly one Unicode code point: a raw UTF-8 character (`'a'`, `'å'`, `'😄'`) or an escape (`'\n'`, `'\x41'`, `'å'`, `'\u{1F600}'`). Unlike strings, `'\0'` is valid - it is the code point 0, not an embedded NUL.

Keywords. `fn` `async` `static` `inline` `let` `var` `const` `global` `if` `else` `for` `foreach` `iter` `from` `to` `while` `switch` `case` `default` `break` `continue` `return` `yield` `await` `guard` `nil` `true` `false` `class` `this` `super` `new` `enum` `import` `export` `library` `try` `catch` `finally` `throw` `breakpoint`. The words `and` `or` `in` `is` `as` are operators. (`var` is an alias for `let`.)

Runtime Types

These are the actual types values carry at runtime.

Primitive Types

Type	Description
`nil`	Absence of value. Only one instance exists.
`bool`	Boolean `true` or `false`.
`char`	Single Unicode codepoint (unsigned 32-bit).
`int`	Signed 64-bit integer. Values within $\pm 2^{60}$ are tagged immediates (no allocation); larger values are heap-allocated. Full int64 range supported.
`float`	64-bit IEEE-754 double-precision.
`string`	Immutable UTF-8 byte sequence.

Composite Types

Type	Description
`array`	Dynamic, zero-indexed array. Max 10,000,000 elements.
`object`	Hash-based dictionary with **string keys** (property names). Indexing with a non-string key (`o[42] = x`) raises. For int/char/float or other value keys use `Collections.HashMap` / `Collections.OrderedMap`. Max 10,000,000 properties. Property iteration order is not stable across processes (see `Json.Canonicalize` for a deterministic ordering).
`exception`	Exception with code and message.

Structured Types

Type	Description
`class`	Class definition object.
`instance`	Instance of a class.

Execution Types

Type	Description
`fiber`	Lightweight cooperative coroutine.
`fn`	Any callable value: a named function, an anonymous function, a closure, a bound method (`obj.method`), a builtin (`String.Length`), or an FFI function. Anything you can call satisfies `fn`.
`module`	Built-in module namespace.

Binary Types

Type	Description
`block`	Raw memory buffer. Element count × element size bytes.
`pointer`	Runtime allocated pointer (FFI use).
`stream`	File/IO stream handle.

Type Limits

Integer

Flaris integers are full signed 64-bit. Values within $\pm 2^{60}$ are stored as **tagged pointer immediates** (zero heap allocation, fast path). Values outside that range fall back to heap-allocated objects - same type, same semantics, just slower to create.

Property	Value
Representation	tagged immediate for $\|x\| < 2^{60}$, heap-allocated otherwise
Minimum	−9,223,372,036,854,775,808 (-2^{63})
Maximum	9,223,372,036,854,775,807 ($2^{63}-1$)
Fast (no-alloc) range	−1,152,921,504,606,846,976 to 1,152,921,504,606,846,975 ($\sim\pm 2^{60}$)
Overflow behavior	Wraps (no trap)
Division	`int / int` → `int` (truncates toward zero); `float` if either operand is `float`
Power	`int ^^ int` → `int` (negative exponent truncates toward zero: `2 ^^ -1` = 0); `float` if either operand is `float`

Integer literals:

```
42          // decimal
-10         // negative
0x1234      // hexadecimal
0b1001001   // binary
0o755       // octal
1_000_000   // underscore separator (any base)
0xFF_FF     // separators work in hex / binary / octal too
```

`_` digit separators are accepted in every base (and in a float's fraction and exponent); they are stripped before the value is parsed.

Float

Property	Value
Standard	IEEE-754 double precision (64-bit)
Precision	~15–17 significant decimal digits
Minimum positive	2.2250738585072014e−308
Maximum	1.7976931348623157e+308
Special values	`Infinity`, `-Infinity`, `NaN`

Display vs. round-trip: `str()` and `Console.WriteLine`/`Print` use a **compact** form - an integral value prints with no decimal point or exponent (`5000.0` shows as `5000`), and a fractional value uses `%g` (up to ~6 significant digits, so `0.1 + 0.2` shows as `0.3`). For a **precise, round-trippable** string (full `%.17g`) use `Convert.ToString` / `String.ToString` instead.

Float literals:

```
3.14
-0.5
1.0e6     // scientific notation
6.02e23   // exponent, optional +/- sign
1_000.5   // underscore separators (integer, fraction and exponent)
```

A digit is required on both sides of the decimal point: write `0.5`, not `.5` (a leading `.` is the member-access operator), and `5.0`, not `5.` (`5.foo` is a method/property access on `5`). Mixed int + float operations always yield `float`.

String

Property	Value
Encoding	UTF-8
Immutable	Yes - mutation creates a new string
Indexing	Byte-level (returns 1-byte string)
Max size	256 MB

UTF-8 Indexing Model

Strings store raw UTF-8 bytes. Characters may occupy 1–4 bytes. The language distinguishes:

- **Byte index** - position in the raw byte buffer. Used by `foreach`, `String.Utf8CpAt`, `String.Utf8CpNext`. Range `0..string.byteLength`.
- **Code-point index** - the Nth Unicode scalar value. Used by `String.Utf8GetCharAt`, `String.Utf8Substr`, `String.Utf8ByteIndexOf`. Range `0..String.Utf8Length(s)`.

Use `String.*` UTF-8 helpers for correct Unicode handling. Standard string indexing (`s[i]`) returns bytes, not characters.

Char

`char` is an unsigned 32-bit Unicode codepoint value. The `char()` built-in creates one from an integer codepoint.

Character escapes in a `'...'` literal denote **one code point**: `'\n'`, `'\t'`, `'\0'`, `'\xHH'` (U+00HH), `'\uXXXX'`, and `'\u{H..H}'` (up to U+10FFFF). `'\xHH'` is a code point, not a raw byte - `'\xE0'` is U+00E0 (`'à'`), not a lone byte.

Arithmetic with `char`:

- `char + char` and `char + string` **concatenate** into a `string` (each char contributes its UTF-8 encoding): `'a' + 'b'` is `"ab"`, and `'a' + 1` is **not** used for concatenation.
- `char + int` (and other numeric operators) treat the char as its integer code point: `'a' + 1` is `98`, `'z' - 'a'` is `25`.

Bool

Falsy values: `nil`, `false`, `0`, `0.0`, `""`, `'\0'`, empty arrays and empty objects. Everything else is truthy.

Type Annotations

Type annotations are optional. They produce **warnings**, not errors. All existing unannotated code continues to work.

Annotation Syntax

```
// Parameter types only
fn greet(name: string) { ... }
```

```
// Return type only
fn add(a, b): int { ... }

// Full signature
fn calculate(x: int, y: int): int { return x * y; }

// Mixed - some typed, some not
fn format(template: string, value) { return template + str(value); }
```

Type Keywords

Primitive:

Keyword	Runtime Type	Notes
`int`	Integer	Full int64; tagged immediate for $\|x\| < 2^{60}$
`float`	Float	64-bit IEEE-754
`number`	int or float	Shorthand for `int|float`
`string`	String	UTF-8 text
`bool`	Boolean	true / false
`char`	Char	32-bit codepoint
`nil`	Nil	Null / absent

Collection:

Keyword	Runtime Type
`array`	Dynamic array
`object`	Key-value hashmap

Execution:

Keyword	Runtime Type
`fn`	Any callable: function, closure, bound method, builtin or FFI function
`fiber`	Cooperative coroutine
`class`	Class definition
`instance`	Class instance

Special:

Keyword	Meaning
`any`	Accepts any type - disables checking for that parameter
`stream`	File / stream handle
`block`	Binary data buffer
`pointer`	Runtime pointer (FFI)

Union Types

Multiple acceptable types separated by `|`:

```
fn findUser(id: int): object|nil { ... }
fn abs(n: int\|float): int\|float { ... }
```

```
// 'number' is shorthand for int\|float
fn abs(n: number): number { ... }
```

The checker validates that provided values match **at least one** type in the union.

Array Type Annotations

Wrap an element type in `[...]` to annotate an array variable or parameter:

```
let scores:[int] = [10, 20, 30];
let names:[string] = ["alice", "bob"];

fn sum(values:[int]): int { ... }
fn process(rows:[[object]]): nil { ... }  // nested arrays
```

The element type is checked by the analyzer but is not tracked at runtime - the value's runtime type remains `array`. The `[T]` form can appear anywhere a plain type name is valid, including unions:

```
let data:[int]|nil = nil;
```

Optional Parameters

Append `?` directly after the parameter name to mark it as optional. The caller may omit trailing optional arguments; the VM automatically passes `nil` for each one not provided.

```
fn greet(name?: string)            { ... }  // name is nil if omitted
fn log(msg: string, level?: int)   { ... }  // level is nil if omitted
fn tag(a: string, b?, c?: string)  { ... }  // b is any?, c is string?
```

Rules:

- `?` goes on the **name**, before the `:` - `x?`, `x?: int`, both valid. `x: int?` is not valid.
- Optional parameters must trail all required ones.
- Omitted arguments always arrive as `nil` - use `??` to substitute a default inside the function body.
- Type checking still applies for arguments that *are* provided.

```
fn greet(name?: string) {
    Console.WriteLine("Hello " + (name ?? "stranger"));
}

greet("World");  // Hello World
greet();         // Hello stranger
```

Variable Type Annotations

Form	Scope	Notes
`let name = expr`	Local (block / function)	Cannot be used at module top level
`var name = expr`	Local (block / function)	Identical to `let`
`const name = expr`	Local or global	Immutable binding
`global name = expr`	Module (global)	Accessible from all scopes in the file. Redeclaring in the same file is a compile error; redefining from another compile unit (`VM.Eval` , another module) warns and rebinds
`global name:type = expr`	Module (global)	Type-annotated global

Variables can be optionally annotated with a type at declaration time using `let name:type = value`. Once annotated, only values of that type may be assigned.

```
// Inside a function — use let/var:
let y:int = 2;
y = "test";         // Error — y declared as int

let i:instance = nil;
i = new MyClass();  // OK — instance, string, object, array may be nil

let scores:[int] = [1, 2, 3];  // array of int

// At module level — use global:
global counter:int = 0;
global config:object = { debug: false };
```

Type Cast Expression

A **type cast** is a compile-time and (for scalar types) runtime type coercion. Syntax: `(type)expr`

```
let arr = (array)Object.Clone(original);
let n   = (int)3.9;       // → 3 (truncates)
let s   = (string)42;     // → "42"
let b   = (bool)0;        // → false
```

Supported Cast Types

Type	Kind	Runtime effect
`int`	Value	`OP_TO_INT` - truncates float, converts string/bool/char
`float`	Value	`OP_TO_FLOAT` - widens int to double
`char`	Value	`OP_TO_CHAR` - converts int/string to a single character value
`string`	Value	`OP_TO_STRING` - any value - string representation
`bool`	Value	Double logical-NOT (`!!x`) - truthy - `true` , falsy - `false`
`array`	Hint	No bytecode emitted - analyzer type update only
`object`	Hint	No bytecode emitted - analyzer type update only

Type	Kind	Runtime effect
`class`	Hint	No bytecode emitted - analyzer type update only
`instance`	Hint	No bytecode emitted - analyzer type update only
`block`	Hint	No bytecode emitted - analyzer type update only
`stream`	Hint	No bytecode emitted - analyzer type update only

Analyzer Behaviour

- The inferred type of `(T)expr` is always `T`, regardless of what `expr` would otherwise infer to.
- If the cast is the RHS of a `let`/`var` declaration or assignment, the variable's tracked type is updated to `T` immediately. All subsequent uses of that variable are checked against the new type.
- No warning is emitted for type changes caused by a cast - the programmer's intent is explicit.

Compiler Behaviour

- Value casts (`int`, `float`, `char`, `string`, `bool`) compile the inner expression then emit one conversion opcode.
- Hint casts compile only the inner expression - zero extra bytecode.
- Cast nodes are fully transparent to the optimizer and inliner.

Disambiguation

The parser uses two-token lookahead to distinguish a cast from a grouped expression:

```
(int)x        // cast — type keyword followed immediately by ')'
(int + 1)     // grouped expression — not a cast
(myVar)expr   // grouped expression — myVar is not a cast-type keyword
```

Only the eleven listed type names - plus the sized-int aliases `u8` `u16` `u32` `u64` `i8` `i16` `i32` `i64`, which cast exactly like `(int)` - are recognised as cast targets. All other identifiers inside `(...)` are treated as grouped expressions.

> **`(u8)x` cast vs `u8(x)` built-in - not the same.** A sized-int *cast* `(u8)x` converts to an integer without narrowing (it behaves like `(int)`): `(u8)300` is `300`. The sized-int *conversion built-in* `u8(x)` wraps modulo the type width: `u8(300)` is `44`, and `i8(200)` is `-56`. Use the built-in when you want two's-complement wrap-around; use the cast only as a type hint.

Closures

A **closure** is an anonymous function that captures variables from its enclosing scope. Captured variables are snapshotted **by value** at the moment the inner function is created (i.e., when the enclosing function executes the expression `fn(...) { ... }`).

What can be captured

Both parameters and body-local variables of the immediately enclosing function are eligible for capture. Module-level globals are always accessible directly and are not captured - they are read live from the global

environment on each access.

Capture semantics

- **Scalars** (`int`, `float`, `bool`, `string`, `char`, `nil`): snapshot copy. Later changes to the outer variable do not affect the captured value.
- **References** (`array`, `object`, `instance`): reference copy. The closure and the outer scope share the same heap object; mutations are visible on both sides.

Example

```
fn makeAdder(x) {
    return fn(n) { return x + n; };
}

let add5 = makeAdder(5);
Console.WriteLine(add5(3));   // 8
```

```
fn makeCounter() {
    let state = [0];
    return fn() {
        state[0] = state[0] + 1;
        return state[0];
    };
}

let c = makeCounter();
Console.WriteLine(c());  // 1
Console.WriteLine(c());  // 2
```

Loop capture

Each loop iteration creates a new closure with its own snapshot of the loop variable:

```
fn makeFns() {
    let funcs = [];
    iter(i from 0 to 3) {
        Array.Append(funcs, fn() { return i; });
    }
    return funcs;
}

let fs = makeFns();
Console.WriteLine(fs[0]());  // 0
Console.WriteLine(fs[2]());  // 2
```

Runtime representation

A closure is a cloned function object with a private environment table that holds the captured names and values. At call time, the VM sets this table as the active environment, so captured names resolve before the module globals. The `function` type applies to both plain functions and closures; `VM.GetFunctionInfo` returns the same fields for both.

Capturing `this`

An anonymous function created inside an instance method that references `this` captures the receiver too, so it can read/write fields (`this.field`) after the method has returned. A closure may capture at most **64** distinct outer variables.

Sealed instance fields

Instances are **sealed**: every instance field must be declared with `let` or `var` in the class body. Writing to a field that was never declared is a **compile error** (`Exception.FieldUndeclared`, code 24):

```
class Point {
    let x = 0;
    let y = 0;
    fn Constructor(a, b) {
        this.x = a;      // ok - declared
        this.y = b;      // ok - declared
        this.z = 0;      // COMPILE ERROR: 'this.z' is not declared in class 'Point'
    }
}
```

This catches typo'd field names at compile time and lets the VM lay out instances as fixed-size slot arrays. The full field set is the class's own declared fields plus every field inherited from a resolvable base class.

Leniency across module boundaries. If a class extends a base that is not visible in the current compilation unit (an imported base class), its inherited field set cannot be seen, so the seal is relaxed for that subclass: undeclared `this.<field>` writes are allowed and validated by the VM's runtime guard instead of at compile time.

Inheritance and `super`

A subclass reaches its base class through `super`:

- `super(args)` inside `Constructor` chains to the base constructor. It resolves statically on the defining class's base, so it works even when the subclass overrides the constructor.
- `super.method(args)` calls the base class's version of a method statically ("invokespecial"), letting an overriding method reach the one it overrides.

```
class Animal {
    let name = "";
    fn Constructor(n) { this.name = n; }
```

```
    fn describe() { return "animal " + this.name; }
}
class Dog : Animal {
    fn Constructor(n) { super(n); }              // chain to Animal.Constructor
    fn describe() { return super.describe() + " (dog)"; }
}
```

`switch` matching

`switch` compares the scrutinee against each `case` label:

- A **value** label matches by equality (`==`). Multiple labels may share a body.
- A **class** label - `case SomeClass:` - matches by type (`is` / instanceof, subclass-aware). Imported classes are supported: the most specific matching class wins. An identifier whose type cannot be resolved falls back to value equality (with a compiler warning).
- `default` runs when no case matches. Cases do **not** implicitly fall through to the next case's body; use `break` to leave a case early.

When every case label is a dense range of integer literals, the compiler emits a **jump table** (one O(1) indexed branch) instead of a comparison chain.

```
switch (shape) {
    case Circle:            return "round";      // type match (instanceof)
    case Square: case Rect: return "cornered";   // value/type, shared body
    default:                return "unknown";
}
```

R3 - Built-in Functions

These functions are VM instructions - available everywhere, no namespace required.

Type Information

Function	Description
`type(x)`	Returns the runtime type as an `int` bitmask. Compare with `Type` module constants: `type(x) == Type.Int`.
`is_nil(x)`	Returns `true` if `x` is `nil`.
`is_array(x)`	Returns `true` if `x` is an array.
`is_object(x)`	Returns `true` if `x` is an object.

Type Conversion

Function	Converts to	Notes
`int(x)`	Integer	Truncates floats toward zero
`float(x)`	Float	Widens int to double
`str(x)`	String	Converts any value to string
`char(x)`	Char	32-bit Unicode codepoint

Bitcast / Narrowing

Function	Result type	Wraps on overflow
`u8(x)`	unsigned 8-bit (0–255)	Yes
`i8(x)`	signed 8-bit (–128–127)	Yes
`u16(x)`	unsigned 16-bit	Yes
`i16(x)`	signed 16-bit	Yes
`u32(x)`	unsigned 32-bit	Yes
`i32(x)`	signed 32-bit	Yes

```
let x = u8(300);   // - 44 (wraps)
let y = i8(-200);  // - 56 (wraps)
```

Length

```
len([1, 2, 3]);        // 3 (array element count)
len("hello");          // 5 (byte count, not char count)
len({ a: 1, b: 2 });   // 2 (property count)
```

Guard

```
guard(expr)   // Raises Exception.GuardCheck if expr evaluates to nil
```

R4 - The Exception class

`Exception` is a built-in base **class**. Every thrown value - whether from a user `throw` or an internal VM error (index-out-of-bounds, divide-by-zero, …) - is an `Exception` (or subclass) instance.

Construct / throw:

```
throw new Exception(code, msg);   // explicit
throw(code, msg);                 // shorthand - desugars to new Exception(code, msg)
class NetworkError : Exception {}  // subclass; new NetworkError(503, "down") inherits the
constructor
```

Constructor: `Exception(code:int, msg:string)` - sets `Code`, `Error`, and captures `StackTrace`.

Instance fields:

Field	Type	Description
`Code`	int	Numeric error code (see the constants below)
`Error`	string	Human-readable message
`StackTrace`	array	Call frames at throw time, as strings

Instance methods:

Method	Signature	Description
`ToString`	`ToString() - string`	`"<TypeName> (code N): message"` (uses the subclass name)
`Name`	`Name() - string`	The exception's type name (its class name)
`StackTraceString`	`StackTraceString() - string`	`StackTrace` joined into one block of text

Generic class introspection works too (`Class.InstanceOf(e, Exception)`, `Class.Name(e)`, …). For type dispatch use `e is SomeError` or `switch (e) { case SomeError: … }`. See the language guide's *Exceptions* section for full semantics.

Error-code constants

Exposed as **static members of the `Exception` class** (`Exception.RuntimeError`, …). Each has a numeric code (stable ABI - must not be renumbered) used as the `Code` of the raised exception.

Constant	Code	Description
`Exception.NullPtr`	`0`	Null-pointer access
`Exception.DivByZero`	`1`	Division by zero
`Exception.ModByZero`	`2`	Modulo by zero
`Exception.InvalidArguments`	`3`	Invalid function arguments
`Exception.OutOfBounds`	`4`	Index out of bounds
`Exception.IOError`	`5`	I/O error (file, stream, socket)
`Exception.RuntimeError`	`6`	General VM runtime error
`Exception.InvalidState`	`7`	VM or object in invalid state
`Exception.OutOfMemory`	`8`	Memory allocation failed
`Exception.InvalidMemoryAccess`	`9`	Invalid memory access
`Exception.SizeLimit`	`10`	Container or allocation size limit exceeded
`Exception.GuardCheck`	`11`	`guard()` failed - value was nil
`Exception.StackError`	`12`	Stack overflow / underflow / corruption
(reserved)	`13`	Intentionally unused
`Exception.UnsafeOperation`	`14`	Unsafe operation blocked (requires `--unsafe`)
`Exception.NestingError`	`15`	Excessive recursion or nesting depth
`Exception.IllegalInstruction`	`16`	Invalid bytecode instruction
`Exception.ExecOutOfMemory`	`17`	Out of memory during execution
`Exception.OutOfFibers`	`18`	Fiber limit reached
`Exception.ConstAssign`	`19`	Attempt to assign to a constant
`Exception.ChecksumError`	`20`	FFI shared-library SHA-256 verification failed (`Ffi.Load`)
`Exception.ClassNonStaticCall`	`21`	Non-static method called without instance
`Exception.TypeMismatch`	`22`	Value incompatible with a typed-array element type
`Exception.AssertionFailed`	`23`	`Debug.Assert` / assertion failed
`Exception.FieldUndeclared`	`24`	Write to an instance field not declared with `let` / `var` (see *Sealed instance fields*)

Design notes:

- Exceptions 16, 20 indicate corrupted or hostile bytecode - treated as hard failures.
- `OutOfMemory` (8) and `ExecOutOfMemory` (17) are separate to enable more precise diagnostics.
- When an exception fires, the current fiber unwinds to the nearest enclosing error boundary (`try` / `catch`). If a boundary catches it, the fiber resumes there, other fibers continue, and the VM stays alive. If the exception escapes the top of the fiber with no handler, the VM prints the trace and exits the whole process with the exception's code (see R9).

R5 - Operator Precedence

Operators listed from **highest** (evaluated first) to **lowest** (evaluated last).

Level	Category	Operators	Associativity
1	Primary	literals, identifiers, `(...)`	N/A
2	Postfix	`obj.prop` `arr[i]` `fn(...)` `x++` `x--`	Left
3	Prefix Unary	`+x` `-x` `!x` `~x` `await` `new` `guard()`	Right
4	Multiplicative & Bitwise	`*` `/` `%` `^^` `<<` `>>` `&` `|` `^`	Left
5	Additive	`+` `-`	Left
6	Comparison	`<` `<=` `>` `>=`	Left
7	Equality	`==` `!=` `≈` `in` `is`	Left
8	Logical AND	`&&` `and`	Left
9	Logical OR	`||` `or`	Left
10	Null Coalescing	`??`	Left
11	Conditional	`? :`	Right
12	Assignment	`=` `+=` `-=` `*=` `/=` `%=` `&=` `|=` `^=` `<<=` `>>=` `??=` `^^=`	Right

Key rules:

- `&&` binds tighter than `||` - `a || b && c` means `a || (b && c)`. `and` / `or` are tokenizer aliases with identical precedence.
- `&&` / `||` short-circuit and yield a **value, not always a bool**: `a && b` evaluates to `b`'s value when `a` is truthy, otherwise `false`; `a || b` evaluates to `true` when `a` is truthy, otherwise `b`'s value. (`nil` and `0` are falsy.)
- Shifts operate on `int64` and mask the shift count to `& 63`, so a count `>= 64` wraps into range. `<<` is a logical left shift; `>>` is an **arithmetic** right shift (the sign bit is replicated), so `-8 >> 1` = -4.
- `??` (null coalescing) is lower than all arithmetic - `x + y ?? z` means `(x + y) ?? z`.
- Division `/` and power `^^` produce `int` when both operands are `int` (division truncates toward zero; a negative power truncates too, so `2 ^^ -1` = 0), and `float` when either operand is `float`. Force a float result with a float operand: `a / (b * 1.0)` or `float(a) / b`.
- `^^` is exponentiation (`x ^^ y` = x raised to y).
- `+` is **overloaded by operand type** and never a type error (see also Operator operand checking):
  - `number + number` - arithmetic (int lane unless an operand is `float`).
  - `char + char` / `char + string` - concatenate into a `string`; `char + int` is arithmetic on the code point (`'a' + 1` is `98`).
  - `array + array` - a **new** array with the elements of both (neither operand is mutated); `array + value` appends `value` to a copy.
  - `object + object` - a **new** merged object (right operand's keys win).
  - anything else - the operands are coerced to their string forms and concatenated (`nil` renders as `"null"`, `true` / `false` as themselves), so `nil + 5` is `"null5"`.
- Comparison (`<` `<=` `>` `>=`) and equality (`==` `!=`) on **containers** are structural and recursive (bounded by the comparison-depth limit below):
  - `array` / `array` and `block` / `block` compare **lexicographically** for ordering (first differing element decides; a prefix is "less"), and element-by-element for equality. `!=` is the negation of `==`.
  - `object` / `object` and `exception` / `exception` compare equal when they hold the same keys mapping to equal values (order-independent).
  - `instance` / `instance` compare **structurally** (same class, field-by-field); `class` / `class` compare by **identity**.

  - Mixed numeric kinds compare by value across `int` / `float` / `char` / `bool` (`'A' == 65` is `true`; `1 == 1.0` is `true`). Two values of otherwise unrelated types are never equal.
- Unary `+x` is an identity operator (yields `x` unchanged); `await expr` is a prefix operator usable in expression position, whereas `yield` is a **statement** only and cannot appear inside a larger expression.
- `≈` (approximate equality) uses `APPROX_REL_EPS = 1e-2` and `APPROX_ABS_EPS = 1e-9`.

R6 - VM Limits and portability

Fibers and Scheduling

Limit	Default	Maximum	Description
Fibers	256	1024	Concurrent fibers tracked by the scheduler
Events	64	64	Event slots (I/O, async, etc.)
Timers	64	64	Active timers
Per-fiber scheduling quantum	10,000	100,000	Scheduling checkpoints (loop back-edges + call/return boundaries) per slice before auto-yield; tune with `Fiber.SetQuantum`

Call Stack and Execution

Limit	Default	Maximum	Description
Call frames	64	1,024	Call stack depth; override with `--frames=`
Nested try/catch handlers	24	24	Maximum nested try/catch handlers
Evaluation stack	4,096	65,535	VM evaluation stack size; override with `--stack=`

Locals and Arguments

Limit	Value	Description
Local variables per function	128	Maximum local variables per function
Function call arguments	16	Maximum arguments in user function calls

Compiler and Parser

Limit	Value	Description
AST nodes per compilation unit	100,000	
Parser nesting depth	256	
Literal nesting depth	64	Max container nesting when building array/object literals (raises `Exception.NestingError`)
Comparison depth	1,024	Max nesting for structural `==` /ordering of containers (raises `Exception.NestingError`; above the JSON depth cap, so parsed documents always compare)
Clone depth	512	Max nesting for `Object.Clone` / `Array.Clone`; a deeper or **cyclic** value raises `Exception.NestingError` instead of overflowing the stack (above the JSON depth cap, so parsed documents always clone)
Print depth	64	Console value printing descends this deep, then elides deeper values with `...`
`VM.Eval()` / `VM.Compile()` source	10 KB	Max source length

Bytecode and Chunks

Limit	Value	Description
Source file size	10 MB	Maximum source file size
Bytecode per function	10 MB	Maximum compiled bytecode per function
Constants per function	16,385	Maximum constant pool entries per function

Strings and Text

Limit	Value	Description
String size	256 MB	Maximum size of a single string
String split parts	100,000	Maximum parts returned by string split
OS command output	16 MB	Maximum captured stdout/stderr from OS commands

Memory Blocks and Buffers

Limit	Value	Description
Single block allocation	2 GiB	Maximum size of a single block allocation
Total block allocations	64 GiB	Total cap across all active block allocations
`Memory.Process()` input	16 MB	Max bytes per call

Collections

Limit	Value	Description
Array elements / object properties	10,000,000	Max elements in an array or properties in an object
Hash function input	1 GiB	Max input to hashing functions

Classes

Limit	Value	Description
Inheritance chain depth	8	Maximum class inheritance depth
CLI arguments	128	Maximum CLI arguments passed to the VM

Special Methods

Method	Called when	Notes
`Constructor(...)`	`new ClassName(...)`	Optional. Return value is ignored; `new` always returns the instance.
`Destructor()`	Last reference to the instance is released	Optional. `this` is the dying instance. Exceptions are swallowed. Not inherited. **Not run for instances still alive at process exit** - teardown tears down the fiber that would execute it, so release anything that must be cleaned up (or do it explicitly) before returning from `Main`. **Do not let `this` escape** (store it in a global, array, or another object's field): the instance is already being freed, so a captured reference dangles - copy out the field values you need instead.

Float Approximation Thresholds

Threshold	Value	Description
Relative epsilon (`≈`)	`1e-2`	Relative tolerance for the `≈` operator
Absolute epsilon (`≈`)	`1e-9`	Absolute tolerance for the `≈` operator

Platform Portability

Flaris targets 64-bit platforms only (x86-64 and ARM64 on Linux, macOS, and Windows).

Compiled Bytecode (.flx)

`.flx` files are fully portable between all supported platforms running the same VM version. The bytecode format uses only fixed-width integer types stored little-endian throughout (header, constant pool, bytecode operands and JIT IR); no pointer-sized fields are stored on disk. Strings used more than once within a file are held once in a shared string pool and referenced by index, so a name mentioned by many functions costs a single copy on disk.

Integer Range

Small integers are stored as tagged pointers (range ±2^60). Integers outside this range are heap-allocated transparently - behavior is identical, only allocation strategy differs. Scripts do not need to account for this.

Pointer-width APIs

Functions that expose raw memory addresses (`Buffer.GetAddress`, `Memory.Alloc`) return 64-bit process-local addresses. Addresses stored in `.flx` constants or serialized to disk are meaningless outside the process that produced them and must never be shared or persisted.

Detection

Use `Os.Arch()` and `Os.Name()` to branch on platform at runtime (`Os.Bits()` is always `64`):

```
if (Os.Arch() == "arm64") {
    // Apple-silicon / AArch64 path
}
```

R7 - Performance Model

Understanding how Flaris executes helps write fast, predictable programs.

Execution Model

Flaris compiles source to bytecode, then runs it on a stack-based VM. On ARM64 (`aarch64`) and x86-64, eligible functions are additionally compiled to native machine code at first call - see R11 for details. On other platforms the bytecode interpreter is the only execution tier.

Dynamic Typing and Runtime Checks

Types are checked at runtime. Invalid operations fail fast with no undefined behavior. In hot loops, avoid changing the type of a variable frequently - type stability helps the VM dispatch efficiently.

```
// Good - type-stable loop
let sum = 0;
```

```
for (let i = 0; i < n; i += 1) {
    sum += arr[i];
}
```

Memory Management

Flaris uses **deterministic reference counting**, not a garbage collector.

- Objects are freed immediately when the last reference drops.
- No stop-the-world pauses, and `Destructor` runs at a predictable point.
- Avoid creating short-lived objects inside tight loops.
- Reuse arrays and objects where possible.

Objects come from pre-allocated SLAB pools. A release that reaches zero hands the object to a deferred-free FIFO, which is drained at safe points; this keeps the release path short and bounds cascading frees. Nothing in the VM traces from roots - there is no mark-and-sweep phase and no cycle collector.

Reference cycles are never collected

Pure reference counting cannot reclaim a cycle: each object in the loop is still referenced by another, so no count ever reaches zero. The usual source is a **back-reference** - a child that points at its parent.

```
class Node {
    let _parent = nil;      // holds the parent...
    let _children:array = [];
}
// ...while the parent holds the child. Neither is ever freed.
```

Flaris has no `weak` reference, so the fix is structural. In order of preference:

1. **Do not store the parent.** Most back-references exist to read one or two values. Copy those values into the child at construction instead of the object.
2. **Let the owner pass itself in** as a method argument when the child needs it, so the reference lives only for the duration of the call.
3. **Break the cycle explicitly** before dropping the structure - assign `nil` to the back-reference in a teardown method or `Destructor`.

The same applies to any loop, not just parent/child: two objects referencing each other, a closure stored on the object it captures, or a fiber handle kept in a field of the object whose method the fiber runs.

Finding leaks

Run with `--mem`. It prints live/peak object counts and a leak dump at shutdown, and **exits non-zero when leaks are found**, so it works in CI:

```
flarisvm --mem myprogram.fls
```

A leak count that is stable across runs but non-zero almost always means a cycle rather than a missing release. `Debug.Refs(value)` reports an individual value's reference count, and `VM.MemoryStats()` returns the same counters programmatically for an in-test check. Note that the project's own suite runs `flarisvm --mem`, so a program that looks clean without the flag can still fail there.

Locals vs Property Access

Local variables are the fastest storage. Property lookups require a hashmap traversal.

```
// Cache property before a hot loop
let v = obj.value;
for (...) {
    sum += v;   // fast — local
}
```

Arrays vs Objects

Structure	Access	Memory	Best for
`array`	O(1) index	Compact	Numeric loops, dense data
`object`	O(1) hash	Flexible	Structured data, named fields

Prefer arrays for anything numeric or iteration-heavy.

Built-ins vs Script Loops

Built-in functions execute closer to the VM and reduce dispatch overhead. Prefer them over equivalent hand-written loops.

```
// Better — built-in
let squares = Array.Select(xs, fn(x){ return x * x; });

// Slower — script loop
let r = [];
for (...) { Array.Append(r, x * x); }
```

Fibers and Scheduling

Fibers are cooperatively scheduled - they never run unless resumed or awaited. No data races by default.

- Batch work inside a fiber before yielding.
- Avoid spawning fibers for tiny tasks.
- Avoid `yield` inside tight numeric loops.
- Default scheduling quantum per fiber: 10,000 checkpoints (tunable via `Fiber.SetQuantum()`, up to 100,000). A checkpoint occurs at each loop back-edge and at call/return boundaries - not at every instruction.

Strings

Strings are immutable. Each concatenation allocates.

- Avoid repeated concatenation in loops.
- Accumulate data first, convert to string once.
- For large text building, collect into an array and use `String.Join()`.

Binary and Numeric Heavy Work

For heavy data processing, use `block` values and `Memory.*` functions. This minimizes object allocation and works similarly to `Span<byte>` in systems languages.

```
let buf = Buffer.Create(1024, 1);
Buffer.Fill(buf, 0);
Memory.Process(Buffer.GetAddress(buf), 1024, 1, fn(x){ return x + 1; });
```

Measure, Don't Guess

Most common bottlenecks are:

- Excessive allocations in hot paths
- Too many small fibers
- Repeated property lookups in loops
- Overuse of dynamic object structures where arrays suffice

Use `--time` (timing) and `--stats` (statistics) flags. Profile real workloads before optimizing.

Mental Model Summary

What	Cost
Local variable read/write	Very cheap
Array index access	Cheap
Property lookup	Moderate (hashmap)
Built-in function call	Cheap
Script function call	Moderate
Object/array allocation	Moderate
String concatenation	Creates new string
Fiber creation	Cheap
Fiber context switch	Cheap

R8 - Standard Library

Flaris ships 32 built-in modules covering everything from math and string processing to networking, concurrency, graphics, and low-level memory access.

Usage

All standard library functions are accessed through their namespace:

```
let sorted = Array.Sort(myArr);
let digest = Hash.Sha256("hello");
let root = Math.Sqrt(2.0);
```

No import statement is needed - all modules are always available. Two modules (`Ffi`, `Memory`) additionally require the `--unsafe` flag at runtime.

Each function table has a `JIT` column stating whether the function can be called directly from JIT-compiled code (see R11 for the legend and the surrounding rules).

Module Index

Category	Module	Description
Data	`Array`	Map, filter, sort, and aggregate arrays
	`Buffer`	Low-level fixed-size binary buffers
	`Collections`	Stack, Queue, HashMap, PriorityQueue, MaxPriorityQueue, LinkedList, Set, and OrderedMap factory functions
	`Object`	Introspect and manipulate objects and instances
	`Class`	Type-level reflection and inheritance inspection
	`Type`	Runtime type constants for use with `type()`
Strings & Text	`String`	UTF-8 string operations
	`Char`	Unicode character classification and conversion
	`Regex`	POSIX extended regular expression matching
	`Json`	JSON parse, serialize, and path query
	`Convert`	Type conversion across all integer widths
Math & Encoding	`Math`	Full mathematical library
	`Random`	Seeded, deterministic PCG32 random streams
	`Hash`	Non-cryptographic and cryptographic hashes
	`Crypto`	XChaCha20-Poly1305 encryption and CSPRNG
I/O & Files	`File`	File read, write, and metadata
	`Directory`	Filesystem directory operations
	`Path`	Cross-platform path string manipulation
	`Stream`	Unified I/O for files, sockets, and serial ports
	`Console`	Terminal I/O, color, and cursor control
	`FileWatch`	Watch file-states on files and callbacks
Networking	`Net`	DNS, IP conversion, and interface inspection
Concurrency	`Fiber`	Green-thread creation, messaging, and lifecycle
	`Event`	One-shot fiber synchronization (max 64 events)
	`Scheduler`	Low-level fiber scheduling primitives
	`Timers`	Fiber-based timer scheduling (max 64 timers)
System	`OS`	Process, environment, and system interface
	`VM`	VM introspection and control
	`Ffi`	Dynamic native library loading ⚠
	`Memory`	Unsafe raw memory access ⚠
Time	`Time`	Unix timestamp manipulation
Debug	`Debug`	Assertions, introspection, and runtime checks

⚠ = requires `--unsafe` flag (`flarisvm --unsafe <file>`)

Type Annotation Shorthands

Shorthand	Meaning
`any`	every type available
`number`	`int` or `float`
`numeric`	`int`, `float`, `bool`, or `char`
`ptr`	`int` (raw address), `string`, `block`, or `pointer` - anything pointer-like

Shorthand	Meaning
`string|block`	string or binary block
`object|instance|class`	any object-like value

Return type is shown after `-`. Optional arguments are shown as `arg?`.

Array

Namespace: `Array`

Higher-level operations on arrays: mapping, filtering, sorting, aggregation, and capacity management. Most functions create new arrays; some mutate in-place (noted in Description).

Equality: `Contains`, `IndexOf`, and `Distinct` use strict value+type equality — `2` (int) does not match `2.0` (float), unlike the `==` operator and `BinarySearch` / `Append`, which coerce ints to float when the array is a `[float]` array.

Sort stability: `Sort` / `SortBy` with the *default* order are not guaranteed stable (they use `qsort`). `Sort` with a user comparator is stable when the comparator is a strict-less predicate (`a < b`); a non-strict (`a <= b`) predicate is also accepted but does not preserve the original order of equal elements.

Function	Signature	Description	JIT
Append	`Append(arr:array, value:any) - arr`	Appends `value` to end of `arr`. Mutates.	—
Average	`Average(arr:array) - float`	Returns the arithmetic mean of all elements as `float`. Returns `nil` if empty.	✓
BinarySearch	`BinarySearch(arr:array, value:any, less?:fn) - int`	Binary search over a SORTED array. Returns the index of a match, or `-(insertionPoint) - 1` when absent (so `-(result) - 1` is where it would go). Optional `less(a,b)-bool` must be the same predicate the array was sorted with. O(log n).	—
Capacity	`Capacity(arr:array) - int`	Returns current internal capacity (may exceed length).	✓
Clear	`Clear(arr:array) - bool`	Removes all elements; capacity kept. Mutates.	—
Concat	`Concat(a:array, b:array) - array`	Returns new array: all of `a` followed by all of `b`.	—
Contains	`Contains(arr:array, value:any) - bool`	`true` if any element equals `value` (deep equality).	✓
Create	`Create(size:int, type?:int) - array`	Creates new array with `size` pre-allocated slots. Optional `type` (a `Type.*` constant) pre-fills all slots with the zero value for that type - `Type.Float` gives a `[float]` array of 0.0 values; `Type.Int` gives a `[int]` array of 0 values; `Type.String` gives a `[string]` array of "" values. Without a type argument all slots are `nil`.	✓
Distinct	`Distinct(arr:array) - array`	Returns new array with only first occurrence of each value.	—
Clone	`Clone(arr:array) - array`	Returns a shallow copy: a new array with the same elements (element refs shared). Preserves a typed array's element type.	—
Count	`Count(arr:array, pred:fn) - int`	Number of elements for which `pred(x)` is truthy.	—
Fill	`Fill(arr:array, value:any, start?:int, end?:int) - array`	Sets elements in `[start, end)` to `value` in-place (whole array by default). Negative `start` / `end` count from the end. Returns same array. Mutates.	—

Function	Signature	Description	JIT
Find	`Find(arr:array, pred:fn) - any`	First element for which `pred(x)` is truthy, or `nil`.	—
ForEach	`ForEach(arr:array, func:fn) - array`	Calls `fn(x)` for every element; callback return value is discarded. Returns the original array.	—
First	`First(arr:array) - any`	Returns first element or `nil` if empty.	—
Flatten	`Flatten(arr:array) - array`	Returns new array with one level of nested arrays collapsed. Non-array elements are kept as-is.	—
Chunk	`Chunk(arr:array, size:int) - array`	Splits `arr` into sub-arrays of at most `size` elements. The last chunk may be smaller. Returns a new array of arrays.	—
IndexOf	`IndexOf(arr:array, value:any) - int`	Returns first index of `value` or `-1` if not found.	✓
InsertAt	`InsertAt(arr:array, index:int, value:any) - arr`	Inserts `value` at `index`, shifts remaining right; returns the array (`nil` on out-of-range, or raises on a typed-array element-type mismatch). Mutates.	—
Last	`Last(arr:array) - any`	Returns last element or `nil` if empty.	—
Process	`Process(arr:array, func:fn) - array`	Applies `fn(value:any)` to every element in-place, replacing each element with the return value. Returns same array. Mutates.	—
ProcessIdx	`ProcessIdx(arr:array, func:fn) - array`	Applies `fn(value:any, index:int)` to every element in-place, replacing each element with the return value. Returns same array. Mutates.	—
IsAllInt	`IsAllInt(arr:array) - bool`	`true` if every element is an integer (an empty array is `true`). Use before `ProcessCallback` / `ProcessEvent` with an `int` kernel.	—
IsAllFloat	`IsAllFloat(arr:array) - bool`	`true` if every element is a float (an empty array is `true`). Use before `ProcessCallback` / `ProcessEvent` with a `float` kernel.	—
ProcessCallback	`ProcessCallback(fn:function, arr:array, len:int, cb:function) - nil`	Processes a scalar array with your worker function `fn(arr, len)`, then calls `cb(arr, result)` when finished. Runs on another CPU core when `fn` is simple enough (only touches `arr`, plain number math, no calls or allocations) — otherwise runs normally, same result. The array must be all-`int`, `float`, `char`, or `bool`, and match the kernel's parameter type (`fn(arr:[int],…)` needs an int array, `fn(arr:[float],…)` a float array, etc.); anything else (mixed, or holding strings/arrays/objects) returns `nil`. See `Buffer.ProcessCallback`.	✓
ProcessEvent	`ProcessEvent(fn:function, arr:array, len:int, eventId:int) - nil`	Like `ProcessCallback`, but signals event `eventId` with the result instead of calling a callback. Wait for several with `Event.WaitFor([ids])`.	✓
Reduce	`Reduce(arr:array, func:fn, initial?:any) - any`	Aggregates left-to-right: `acc = fn(acc, x)` starting from `initial`. Without `initial`, seeds with the first element (empty array → `nil`). `acc`/result may be any type the callback returns.	—
RemoveAt	`RemoveAt(arr:array, index:int) - bool`	Removes element at `index`, shifts remaining left. Mutates.	—
Reserve	`Reserve(arr:array, capacity:int) - bool`	Ensures capacity ≥ `capacity`. Does not affect length. Mutates capacity.	—
Reverse	`Reverse(arr:array) - array`	Reverses elements in-place. Returns same array. Mutates.	—
Select	`Select(arr:array, func:fn ) - array`	Returns new array by applying `fn(x)` to every element (map).	—

Function	Signature	Description	JIT
Shuffle	`Shuffle(arr:array) - array`	Randomly shuffles elements in-place (Fisher-Yates, CSPRNG). Returns same array. Mutates.	—
ShrinkToFit	`ShrinkToFit(arr:array) - bool`	Reduces capacity to match current length. Mutates capacity.	—
Skip	`Skip(arr:array, count:int) - array`	Returns new array with first `count` elements removed (a non-positive `count` keeps all).	—
Slice	`Slice(arr:array, start:int, end?:int) - array`	Returns a new array with elements `[start, end)` (to the end by default). Negative indices count from the end (Python-style); both are clamped, an empty range yields `[]`.	—
SequenceEqual	`SequenceEqual(a:array, b:array) - bool`	`true` if `a` and `b` have the same length and equal elements in order (strict value+type equality: `2 ≠ 2.0`).	—
Sort	`Sort(arr:array, fn?:fn) - array`	Sorts in-place. Optional comparator `fn(a,b)→bool`. Mutates.	—
SortedInsert	`SortedInsert(arr:array, value:any, less?:fn) - int`	Inserts `value` into a SORTED array keeping it sorted (after any equal run). Returns the insertion index, or `nil` on failure. Same optional `less` predicate as `Sort` / `BinarySearch`. Mutates.	—
SortBy	`SortBy(arr:array, func:fn) - array`	Sorts in-place by extracted key: calls `fn(x)` once per element and compares the results. Builtin functions (e.g. `String.Length`, `Type.Of`) use the fast path. Mutates.	—
Tally	`Tally(arr:array) - object`	Counts occurrences of each element (converted to string). Returns `{value: count}`.	—
Subset	`Subset(arr:array, from:int, count:int) - array`	Returns new array of `count` elements starting at index `from`.	—
Swap	`Swap(arr:array, i:int, j:int) - arr`	Swaps elements at indices `i` and `j`; returns the array (`nil` if either index is out of range). Mutates.	—
Take	`Take(arr:array, count:int) - array`	Returns new array of first `count` elements.	—
Where	`Where(arr:array, func:fn ) - array`	Returns new array of elements for which `fn(x)` is truthy (filter).	—
Zip	`Zip(a:array, b:array) - array`	Returns array of `[a[i], b[i]]` pairs. Length is `min(len(a), len(b))`.	—

Array instance method syntax

All `Array` functions where the array is the first argument can be called directly on an array value:

```
var a: array = [3, 1, 4, 1, 5];
a = a.Append(9)              // same as Array.Append(a, 9)
a = a.Sort()                 // same as Array.Sort(a)
a.First()                    // 3
a.Last()                     // 9
a.Contains(4)                // true
a.Average()                  // arithmetic mean (Sum/Min/Max live in the Math module)
a.Select(fn(x) { return x * 2; })
a.Where(fn(x) { return x > 3; })
a.Reverse()
```

Functions that do not take an array as their first argument (`Create`, and similar factories) are not available as instance methods. Both forms compile to identical bytecode when the variable is typed (`: array` or inferred). Untyped variables fall back to a runtime dispatch.

Buffer

Namespace: `Buffer`

Low-level binary memory operations. A buffer is `count` elements × `size` bytes per element.

Function	Signature	Description	JIT
ChangeToString	`ChangeToString(buf:block) - bool`	Reinterprets buffer in-place as `string`. Destructive - original block handle is invalid after call.	—
Copy	`Copy(src:block, srcOff:int, dst:block, dstOff:int, len:int) - bool`	Copies `len` bytes from `src` to `dst`. Offsets in bytes. Bounds-checked.	✓
CopyBytesToArray	`CopyBytesToArray(buf:block, arr:array, offset:int) - bool`	Writes `len(buf)` raw bytes from `buf` into `arr` starting at `offset`. Existing elements are updated in-place; elements past the current array length are appended. Optional `offset` defaults to 0.	—
CopyStringAt	`CopyStringAt(buf:block, offset:int, s:string, n:int) - bool`	Copies `n` bytes of `s` into `buf` at byte `offset`. `n` defaults to `len(s)`. Bounds-checked against the full byte extent (`count × size`); returns `false` if `offset + n` exceeds it, or on a non-block/non-string.	✓
WriteVarintAt	`WriteVarintAt(buf:block, offset:int, value:int) - int`	Writes `value` as an unsigned LEB128 varint (protobuf-style: 7 bits/byte, high bit = continuation, max 10 bytes). Returns the byte count written, or 0 if it would not fit. Zigzag-encode signed values first: `(n << 1) ^ (n >> 63)`.	✓
ReadVarintAt	`ReadVarintAt(buf:block, offset:int) - array`	Reads an unsigned LEB128 varint at `offset`. Returns `[value, bytesRead]`; `bytesRead` is 0 on out-of-range or malformed/truncated input (never reads past the block).	—
Create	`Create(count:int, size:int) - block`	Allocates new buffer: `count` elements, `size` bytes each (`size` in 1–255). `nil` if `count <= 0` or `size` is outside 1–255.	✓ owned[1]
Fill	`Fill(buf:block, value:int) - bool`	Fills entire buffer with byte `value` (0–255). Mutates.	✓
ProcessCallback	`ProcessCallback(fn:function, buf:block, len:int, blocksize:int, cb:function) - nil`	Processes `buf` with your worker function `fn(buf, len, blocksize)`, then calls `cb(buf, result)` when finished. Returns immediately. Runs on another CPU core when `fn` is simple enough (see the note below), otherwise runs normally — same result either way.	✓
ProcessEvent	`ProcessEvent(fn:function, buf:block, len:int, blocksize:int, eventId:int) - nil`	Like `ProcessCallback`, but signals event `eventId` with the result instead of calling a callback. Start several and wait for them all with `Event.WaitFor([ids])`. See the note below.	✓
FromArray	`FromArray(buf:block, arr:array) - bool`	Writes integer array into buffer elements. Element size must be 1, 2, 4, or 8. Mutates.	✓
FromString	`FromString(s:string) - block`	Copies the raw bytes of `s` into a new block (`count = len(s)`, `size = 1`). Non-destructive - both string and block remain valid.	✓ owned[1]
GetAddress	`GetAddress(buf:block) - int`	Returns raw memory address as integer (for FFI/Memory API use). **Requires `--unsafe`** (unsafe	✓

Function	Signature	Description	JIT
		mode); returns raw pointers, so it is gated like the `Memory.*` API.	
FillRange	`FillRange(buf:block, offset:int, length:int, value:int) - bool`	Sets `length` bytes starting at byte `offset` to byte `value` (0–255). Bounds-checked against the full backing span (`count × size`). `false` on a bad value or out-of-range span.	✓
IndexOf	`IndexOf(buf:block, needle:int|string, start?:int) - int`	Byte offset of the first match at or after `start` (default 0), or `-1`. `needle` is a byte (int 0–255) or a byte substring (string; empty string matches at `start`).	✓
Compare	`Compare(a:block, b:block) - int`	`memcmp`-style ordering (`-1` / `0` / `1`) over the two backing byte spans; the shorter buffer sorts first when one is a prefix of the other. `nil` if either argument is not a block.	✓
Equals	`Equals(a:block, b:block) - bool`	`true` if both backing spans are the same byte length and byte-for-byte equal.	✓
ReadI8At	`ReadI8At(buf:block, offset:int) - int`	Reads a sign-extended signed byte at `offset`. `0` if out of bounds.	✓
ReadI16At	`ReadI16At(buf:block, offset:int, bigEndian?:bool) - int`	Reads a sign-extended signed 2-byte integer at `offset`. Optional `bigEndian` defaults to `false`. `0` if out of bounds.	✓
ReadI32At	`ReadI32At(buf:block, offset:int, bigEndian?:bool) - int`	Reads a sign-extended signed 4-byte integer at `offset`. Optional `bigEndian` defaults to `false`. `0` if out of bounds.	✓
ReadFloat32At	`ReadFloat32At(buf:block, offset:int, bigEndian?:bool) - float`	Reads an IEEE 32-bit float at byte `offset`. Optional `bigEndian` defaults to `false`. `0.0` if out of bounds.	✓
ReadFloat64At	`ReadFloat64At(buf:block, offset:int, bigEndian?:bool) - float`	Reads an IEEE 64-bit double at byte `offset`. Optional `bigEndian` defaults to `false`. `0.0` if out of bounds.	✓
WriteFloat32At	`WriteFloat32At(buf:block, offset:int, value:float, bigEndian?:bool) - bool`	Stores an IEEE 32-bit float at byte `offset`. Optional `bigEndian` defaults to `false`. `false` if out of bounds.	✓
WriteFloat64At	`WriteFloat64At(buf:block, offset:int, value:float, bigEndian?:bool) - bool`	Stores an IEEE 64-bit double at byte `offset`. Optional `bigEndian` defaults to `false`. `false` if out of bounds.	✓
ReadU8At	`ReadU8At(buf:block, offset:int) - int`	Reads one byte at `offset`. Returns 0 if out of bounds.	✓
ReadU16At	`ReadU16At(buf:block, offset:int, bigEndian:bool) - int`	Reads 2-byte unsigned integer at `offset`. Optional `bigEndian` defaults to `false` (little-endian). Returns 0 if out of bounds.	✓
ReadU32At	`ReadU32At(buf:block, offset:int, bigEndian:bool) - int`	Reads 4-byte unsigned integer at `offset`. Optional `bigEndian` defaults to `false`. Returns 0 if out of bounds.	✓
ReadU64At	`ReadU64At(buf:block, offset:int, bigEndian:bool) - int`	Reads 8-byte integer at `offset`. Optional `bigEndian` defaults to `false`. Returns 0 if out of bounds.	✓
Reserve	`Reserve(buf:block, capacity:int) - bool`	Ensures `buf` holds at least `capacity` **elements** (`capacity × size` bytes). Grows the buffer in-place via `realloc` if needed; the block object is mutated and `len(buf)` reflects the new element	—

Function	Signature	Description	JIT
		count. No-op if already large enough; `false` on a non-block or zero capacity.	
Slice	`Slice(buf:block, offset:int, count:int) – block`	Returns new buffer: `count` elements starting at element index `offset`.	✓ owned[1]
StringToArray	`StringToArray(s:string, arr:array, offset:int) – int`	Writes the raw UTF-8 bytes of `s` directly into `arr` at `offset` without allocating an intermediate block. Returns the number of bytes written (`len(s)`). Optional `offset` defaults to 0.	—
ToArray	`ToArray(buf:block, offset:int, count:int) – object`	Returns object with array of integers from buffer elements. Element size must be 1, 2, 4, or 8.	—
ToString	`ToString(buf:block) – string`	Copies the raw bytes of `buf` into a new string. Non-destructive - block remains valid. Total bytes read = `count × size`.	✓ owned[1]
WriteU8At	`WriteU8At(buf:block, offset:int, v:int) – bool`	Writes `v & 0xFF` at byte `offset`. Returns `false` if out of bounds.	✓
WriteU16At	`WriteU16At(buf:block, offset:int, v:int, bigEndian:bool) – bool`	Writes 2-byte integer at `offset`. Optional `bigEndian` defaults to `false`. Returns `false` if out of bounds.	✓
WriteU32At	`WriteU32At(buf:block, offset:int, v:int, bigEndian:bool) – bool`	Writes 4-byte integer at `offset`. Optional `bigEndian` defaults to `false`. Returns `false` if out of bounds.	✓
WriteU64At	`WriteU64At(buf:block, offset:int, v:int, bigEndian:bool) – bool`	Writes 8-byte integer at `offset`. Optional `bigEndian` defaults to `false`. Returns `false` if out of bounds.	✓
Fill16	`Fill16(buf:block, value:int, bigEndian?:bool) – int`	Fills the whole buffer with the 2-byte encoding of `value` repeated. Returns the number of 2-byte units written; a trailing partial unit is left unchanged. Optional `bigEndian` defaults to `false`.	✓
Fill32	`Fill32(buf:block, value:int, bigEndian?:bool) – int`	Like `Fill16` with a 4-byte value.	✓
Fill64	`Fill64(buf:block, value:int, bigEndian?:bool) – int`	Like `Fill16` with an 8-byte value.	✓
FillPattern	`FillPattern(buf:block, pattern:block) – int`	Tiles `pattern`'s bytes across the whole buffer extent (the last copy is truncated to fit). Returns the number of bytes written (the buffer extent), or `0` on a non-block or empty pattern.	✓
Reverse	`Reverse(buf:block) – bool`	Reverses the buffer's elements in place (each element is `size` bytes; a byte buffer is reversed byte-for-byte). `false` on a non-block.	✓
CopyWithin	`CopyWithin(buf:block, destOff:int, srcOff:int, length:int) – bool`	Moves `length` bytes within the same buffer from byte `srcOff` to byte `destOff` (memmove; the spans may overlap). Bounds-checked against the byte extent; `false` if out of range.	✓
Concat	`Concat(a:block, b:block) – block`	Returns a new byte block (element size 1) holding `a`'s bytes followed by `b`'s bytes (each buffer's full extent). `nil` on a non-block.	✓ owned[1]
Append	`Append(buf:block, src:block|string) – int`	Grows `buf` in place and appends `src`'s bytes. Returns the new element count, or `0` on failure (non-block/-src, a byte count that is not a multiple of the element size, or a failed grow — on failure `buf` is left unchanged).	—

- **ProcessCallback / ProcessEvent** — process a buffer, optionally on another CPU core
  - You give it a **worker function** `fn(buf, len, blocksize)` that reads and writes the buffer, plus a `cb(buf, result)` that runs when the work is finished. `ProcessCallback` returns right away; `cb` is called later. (`ProcessEvent` is the same but signals an event instead of calling `cb` — see below.)
  - `len` and `blocksize` are just two numbers passed straight through to your worker function for it to use as it likes (typically the element count and the element bit-width, 8/16/32/64). The call does not split or reinterpret the buffer for you.
  - Your worker function returns an `int` or a `float`; that value arrives as the second argument of `cb(buf, result)`. If the work hits an error (e.g. an out-of-range index), `result` is `nil` — it never crashes your program.
  - **To make it run on another core, keep the worker function simple:** it must only read/write the buffer you passed and do plain number math — **no** creating strings/arrays/objects and **no** calling other functions from inside it. Its first parameter must be the buffer and any others must be `int`. A function that follows these rules runs in parallel; any other function still works correctly but runs normally (in line), so you get the same result either way.
  - `cb` **has no such restriction** — it is an ordinary function. Read the finished buffer, allocate, call anything.
  - **To actually run work in parallel, start several calls at once** over *different* buffers. Each returns immediately, so the jobs run side by side. A single call processes one buffer; it does not split one buffer across cores.
  - `ProcessEvent(fn, buf, len, blocksize, eventId)` delivers the result into an event instead of calling a callback. Start several jobs, each with its own event id, then wait for them all in one line with `Event.WaitFor([ids])` — it blocks until every job is done and returns the results as an array (see the example). Handy when you want to fan out and then continue once everything finishes.

```
// Process 4 buffers in parallel, then wait for all of them in one line.
fn kernel(b:block, len:int, _bs:int): int {
    var sum = 0; var i = 0;
    while (i < len) { b[i] = (b[i] * 2 + 1) % 256; sum = sum + b[i]; i = i + 1; }
    return sum;
}
fn async Main() {
    let bufs = [Buffer.Create(1024,1), Buffer.Create(1024,1),
                Buffer.Create(1024,1), Buffer.Create(1024,1)];
    let ids  = [Event.Create(), Event.Create(), Event.Create(), Event.Create()];
    for (let k = 0; k < 4; k++) { Buffer.ProcessEvent(kernel, bufs[k], 1024, 8, ids[k]); }
    let checksums = Event.WaitFor(ids);   // blocks until all four events are set
}
```

```
// Process 4 buffers in parallel, using a counter to tell when all are done.
global done = 0;
fn kernel(b:block, len:int, _bs:int): int {
    var sum = 0; var i = 0;
    while (i < len) { b[i] = (b[i] * 2 + 1) % 256; sum = sum + b[i]; i = i + 1; }
```

```
    return sum;               // delivered to cb as `result`
}
fn onDone(b:block, result:int) { done = done + 1; }   // called when a job finishes

fn async Main() {
    let bufs = [Buffer.Create(1024,1), Buffer.Create(1024,1),
                Buffer.Create(1024,1), Buffer.Create(1024,1)];
    for (let k = 0; k < 4; k++) { Buffer.ProcessCallback(kernel, bufs[k], 1024, 8, onDone); }
    while (done < 4) { Fiber.Sleep(1); }              // wait for all 4 callbacks
}
```

- **FromString / ToString**
  - Both are non-destructive copies - unlike `ChangeToString`, neither modifies the source object.
  - `ToString` reads `count × size` bytes, so a buffer created with `Buffer.Create(n, 4)` will produce a string of `n × 4` bytes. For raw byte buffers intended for string conversion, use `size = 1`.
  - Primary use case is passing string data to FFI functions and reading string results back:

```
    let buf    = Buffer.FromString(http_response.body);
    let result = my_ffi_fn(buf);
    Console.WriteLine(Buffer.ToString(result));
```

- **CopyBytesToArray / StringToArray**
  - These avoid the round-trip overhead of `ToArray` + a Flaris loop when writing into an existing byte array.
  - `StringToArray` reads directly from the string's internal bytes - no block is allocated.
  - Both are zero-copy for the destination: existing array slots (0–255 byte values) are updated in-place without heap allocation.

```
    // Write UTF-8 bytes of a string into a pre-allocated byte array at offset 10
    let n:int = Buffer.StringToArray(s, my_arr, 10);

    // Copy raw bytes from a block into an array
    let blk:block = Buffer.FromString(s);
    Buffer.CopyBytesToArray(blk, my_arr, 0);
```

Class

Namespace: **`Class`**

Type-level introspection, inheritance inspection, and dynamic reflection on user-defined classes.

Function	Signature	Description	JIT
InstanceOf	`InstanceOf(inst:instance, cls:class) – bool`	`true` if instance was created from `class` or any base class.	✓
SubclassOf	`SubclassOf(A:class, B:class) – bool`	`true` if `A` is `B` or `A` descends from `B`. Siblings (a shared parent) and same-named unrelated	✓

Function	Signature	Description	JIT
		classes are **not** subclasses.	
IsClass	`IsClass(value:any) - bool`	`true` if value is a class object.	✓
IsInstance	`IsInstance(value:any) - bool`	`true` if value is a class instance.	✓
Of	`Of(obj:instance|class) - class`	Runtime class of an instance; identity for a class. The unambiguous "get the type" primitive.	—
GetBase	`GetBase(obj:instance|class) - class`	For a **class**, its immediate base (or nil). For an **instance**, its own class — use `Of` when you want the type unambiguously.	—
GetName	`GetName(obj:instance|class) - string`	Class name string, or nil for other types.	—
Name	`Name(obj:instance|class) - string`	Alias of `GetName`. Allocation-free (returns the existing class-name string).	—
Fields	`Fields(inst:instance) - array`	Field name strings on an instance, inherited included.	—
InstanceMethods	`InstanceMethods(cls:class) - array`	Instance method names declared on the class (not inherited).	—
StaticMethods	`StaticMethods(cls:class) - array`	Static method names declared on the class (not inherited).	—
AllMethods	`AllMethods(obj:instance|class) - array`	Every resolved method name, inherited included (overrides appear once).	—
HasMethod	`HasMethod(target:instance|class, name:string) - bool`	`true` if `name` resolves to a method (inherited included).	—
GetMethod	`GetMethod(target:instance|class, name:string) - any`	Callable handle for method `name`: a bound method for an instance method, the raw function for a static; nil if absent or cross-context.	—
GetField	`GetField(inst:instance, name:string) - any`	Value of the declared field `name` (inherited included), or nil. Methods/consts are not fields.	—
SetField	`SetField(inst:instance, name:string, value:any) - bool`	Writes a declared field; returns `false` for an undeclared name or frozen instance. Never fabricates a field.	—
Invoke	`Invoke(target:instance|class, name:string, ...args) - any`	Dynamically calls method `name` on instance or class.	—
Instantiate	`Instantiate(name:string) - instance`	New instance of the named global class **without** running its Constructor (fields get declared defaults). Nil for an unknown or non-class name.	—
New	`New(target:class|string, ...args) - instance`	New instance of a class (object or global name) **with** its Constructor run against `args`. Nil for an unknown/non-class target.	—

Collections

Namespace: `Collections`

Factory functions for Stack, Queue, HashMap, PriorityQueue, MaxPriorityQueue, LinkedList, Set, and OrderedMap. All factories return `object`.

Stack - `Collections.Stack() - object`

Method	Description
`s.Push(v)`	Push value to top.
`s.Pop()`	Remove and return top, or `nil` if empty.
`s.Peek()`	Return top without removing, or `nil` if empty.

Method	Description
`s.IsEmpty()`	`true` if stack has no elements.
`s.Size()`	Number of elements.
`s.Clear()`	Remove all elements.
`s.ToArray()`	Return current elements as array (bottom-to-top).

Queue - `Collections.Queue() – object`

Method	Description
`q.Enqueue(v)`	Add value to back.
`q.Dequeue()`	Remove and return front, or `nil` if empty.
`q.Peek()`	Return front without removing, or `nil` if empty.
`q.IsEmpty()`	`true` if queue has no elements.
`q.Size()`	Number of elements.
`q.Clear()`	Remove all elements.
`q.ToArray()`	Return current elements as array (front-to-back).

HashMap - `Collections.HashMap(capacity?:int) – object`

Keys may be any hashable value (int, char, string, float, ...); lookup matches by hash, which is **by value** for numbers (int, char, bool, and float - equal values, however computed, hit the same entry) and for strings. Other heap objects (arrays, instances, ...) hash by identity, so only the same object matches. Pass an optional `capacity` to pre-size the map when the final key count is known, avoiding intermediate regrows. Iteration order (`Keys` / `Values`) is **not** stable across processes.

Method	Description
`m.Set(key, value)`	Set key to value.
`m.Get(key, default?)`	Return value for key, else `default` (if given) or `nil`.
`m.Has(key)`	`true` if key exists.
`m.Delete(key)`	Remove key; return `true` if it existed.
`m.Clear()`	Remove all entries.
`m.Size()`	Number of entries.
`m.Keys()`	Return array of all keys (unspecified order).
`m.Values()`	Return array of all values (unspecified order).

PriorityQueue - `Collections.PriorityQueue() – object`

Min-heap: `Dequeue()` always returns the smallest value.

Method	Description
`q.Enqueue(v, priority)`	Insert value with numeric priority.
`q.Dequeue()`	Remove and return the value with the lowest priority, or `nil`.
`q.Peek()`	Return the lowest-priority value without removing, or `nil`.
`q.IsEmpty()`	`true` if queue has no elements.
`q.Size()`	Number of elements.
`q.Clear()`	Remove all elements.
`q.ToArray()`	Return values as array ordered lowest-to-highest priority.

MaxPriorityQueue - `Collections.MaxPriorityQueue() – object`

Max-heap: `Dequeue()` always returns the largest value. Same API as `PriorityQueue`.

LinkedList - `Collections.LinkedList() – object`

Singly-linked list with a cached tail pointer. `PushFront`, `PushBack`, `PopFront`, `PeekFront`, and `PeekBack` are O(1). `PopBack` and `Clear` are O(n).

Method	Description
`l.PushFront(v)`	Insert at head. Returns `self` for chaining.
`l.PushBack(v)`	Insert at tail. Returns `self` for chaining.
`l.PopFront()`	Remove and return head value, or `nil` if empty.
`l.PopBack()`	Remove and return tail value, or `nil` if empty. O(n).
`l.PeekFront()`	Return head value without removing, or `nil`.
`l.PeekBack()`	Return tail value without removing, or `nil`.
`l.IsEmpty()`	`true` if list has no elements.
`l.Size()`	Number of elements.
`l.Clear()`	Remove all elements. O(n).
`l.ToArray()`	Return elements as array (front-to-back). O(n).

Set - `Collections.Set(arr?:array) – object`

Hash set: `Add`, `Has`, and `Remove` are one hash lookup each. Optionally seeded with the elements of an array (duplicates collapse). Membership follows the value hash: ints, chars, and strings compare by value; floats and other heap values by identity - use int/char/string elements for value semantics.

Method	Description
`s.Add(v)`	Insert `v`. Returns `true` if newly added, `false` if already present.
`s.Has(v)`	`true` if `v` is a member.
`s.Remove(v)`	Remove `v`; return `true` if it was present.
`s.Union(other)`	New Set with every element of both sets.
`s.Intersect(other)`	New Set with the elements present in both sets.
`s.Difference(other)`	New Set with the elements of `s` not in `other`.
`s.IsEmpty()`	`true` if the set has no elements.
`s.Size()`	Number of elements.
`s.Clear()`	Remove all elements.
`s.ToArray()`	Return the elements as an array (hash order, not insertion order).

```
let seen = Collections.Set();
if (seen.Add(id)) {
    // first time we see id
}
let common = Collections.Set(a).Intersect(Collections.Set(b));
```

OrderedMap - `Collections.OrderedMap() – object`

Insertion-ordered hash map (a LinkedHashMap): O(1) key lookup like `HashMap` plus deterministic insertion-order iteration. Internally a hash index over a doubly-linked node chain.
`Set` / `Get` / `Has` / `Delete` / `First` / `Last` / `PopFirst` / `PopLast` / `MoveToEnd` / `MoveToFront` / `Size` are O(1); `Keys` / `Values` / `Entries` / `ToArray` / `Clear` are O(n). Key equality follows the value hash (ints, chars, and strings compare by value; floats and other heap values by identity). Setting an existing key updates its value in place and leaves its position unchanged; use `MoveToEnd` / `MoveToFront` to reorder (e.g. to drive an LRU cache).

Method	Description
`m.Set(k, v)`	Insert or update. Existing keys keep their position. Returns `true`.
`m.Get(k, default?)`	Stored value, else `default` (if given) or `nil`.
`m.Has(k)`	`true` if the key is present.
`m.Delete(k)`	Remove the entry; `true` if it existed. O(1).
`m.First()`	Oldest value (head) without removing, or `nil`.
`m.Last()`	Newest value (tail) without removing, or `nil`.
`m.PopFirst()`	Remove and return the oldest value (FIFO/LRU eviction), or `nil`.
`m.PopLast()`	Remove and return the newest value, or `nil`.
`m.MoveToEnd(k)`	Move an existing key to newest position (LRU touch); `true` if present.
`m.MoveToFront(k)`	Move an existing key to oldest position; `true` if present.
`m.Keys()`	Keys as an array, in insertion order. O(n).
`m.Values()`	Values as an array, in insertion order. O(n).
`m.Entries()`	`[key, value]` pairs as an array, in insertion order. O(n).
`m.ToArray()`	Values in insertion order (alias of `Values`). O(n).
`m.IsEmpty()`	`true` if the map has no entries.
`m.Size()`	Number of entries.
`m.Clear()`	Remove all entries. O(n).

```
// LRU cache: evict the least-recently-used entry when over capacity.
let cache = Collections.OrderedMap();
fn touch(key, value) {
    cache.Set(key, value);
    cache.MoveToEnd(key);              // mark as most-recently-used
    if (cache.Size() > 100) cache.PopFirst();  // drop the oldest
}
```

Console

Namespace: **`Console`**

Terminal I/O, color, and cursor control.

Function	Signature	Description	JIT
Clear	`Clear() - nil`	Clear the entire terminal screen and move cursor to home (1,1).	—
ClearLine	`ClearLine() - nil`	Erase the current line and move cursor to column 1.	—
Error	`Error(...args:any) - nil`	Print to stderr in red.	—
GetCursor	`GetCursor() - object`	Returns `{x, y}` cursor position object.	—
IsTTY	`IsTTY() - bool`	`true` if stdout is a real terminal (not piped).	—
KeyPressed	`KeyPressed() - bool`	`true` if a key is waiting in the input buffer (non-blocking).	—
Ok	`Ok(...args:any) - nil`	Print to stdout in green.	—
ReadChar	`ReadChar() - char`	Read a single character from stdin (blocking).	—
ReadFloat	`ReadFloat() - float`	Read a float from stdin.	—
ReadInt	`ReadInt() - int`	Read an integer from stdin.	—
ReadKey	`ReadKey() - object`	Blocking read of one keypress. Returns `{Key:string, Char:char, IsSpecial:bool}`. `Key` is `"Up"`, `"Down"`, `"Left"`, `"Right"`, `"Home"`, `"End"`, `"PageUp"`, `"PageDown"`, `"Insert"`, `"Delete"`, `"Enter"`, `"Escape"`, `"Backspace"`, `"Tab"`, `"F1"`–`"F12"`, or a single printable character. `IsSpecial` is `true` for arrows, function keys, and the navigation cluster.	—

Function	Signature	Description	JIT
ReadLine	`ReadLine() - string`	Read a line from stdin (blocking, strips newline).	—
ReadPassword	`ReadPassword() - string`	Read without echo.	—
RestoreCursor	`RestoreCursor() - nil`	Restore previously saved cursor position.	—
SaveCursor	`SaveCursor() - nil`	Save current cursor position.	—
SetBackground	`SetBackground(color:int) - nil`	Set background color. Use `ConsoleColor.*` constants.	—
SetColor	`SetColor(color:int) - nil`	Set foreground color. Use `ConsoleColor.*` constants.	—
SetCursor	`SetCursor(x:int, y:int) - nil`	Move cursor to column `x`, row `y`.	—
SetRaw	`SetRaw(enable:bool) - nil`	Enable or disable terminal raw mode. When `true`: disables line buffering and echo so individual keypresses are available immediately without pressing Enter. Restore with `SetRaw(false)` before exit. Has no effect if not running on a TTY.	—
Size	`Size() - object`	Returns `{width, height}` of terminal.	—
TryReadChar	`TryReadChar() - char`	Non-blocking: returns `char` if key pressed, else `\0`.	—
Warn	`Warn(...args:any) - nil`	Print to stderr in yellow.	—
Write	`Write(...args:any) - nil`	Print to stdout without trailing newline.	—
WriteLines	`WriteLines(...args:any) - nil`	Print multiple values each on its own line.	—
WriteLine	`WriteLine(...args:any) - nil`	Print to stdout with trailing newline.	—

ConsoleColor constants: `Default`, `Red`, `Green`, `Blue`, `Yellow`, `Cyan`

Convert

Namespace: `Convert`

Type conversion and parsing with full range of integer widths.

Function	Signature	Description	JIT
ToBool	`ToBool(v:int|float|bool|char) - bool`	Convert numeric to bool.	✓
ToChar	`ToChar(v:int|float|bool|char) - char`	Convert numeric to char (Unicode codepoint).	✓
ToFloat	`ToFloat(v:int|float|bool|char) - float`	Convert numeric to float.	✓
ToInt	`ToInt(v:int|float|bool|char) - int`	Convert numeric to int.	✓
ToInt8	`ToInt8(v:int|float|bool|char) - int`	Truncate to signed 8-bit, stored as int.	✓
ToInt16	`ToInt16(v:int|float|bool|char) - int`	Truncate to signed 16-bit, stored as int.	✓
ToInt32	`ToInt32(v:int|float|bool|char) - int`	Truncate to signed 32-bit, stored as int.	✓
ToInt64	`ToInt64(v:int|float|bool|char) - int`	Truncate to signed 64-bit, stored as int.	✓
ToUInt8	`ToUInt8(v:int|float|bool|char) - int`	Truncate to unsigned 8-bit, stored as int.	✓
ToUInt16	`ToUInt16(v:int|float|bool|char) - int`	Truncate to unsigned 16-bit, stored as int.	✓
ToUInt32	`ToUInt32(v:int|float|bool|char) - int`	Truncate to unsigned 32-bit, stored as int.	✓
ToUInt64	`ToUInt64(v:int|float|bool|char) - int`	Truncate to unsigned 64-bit, stored as int.	✓
ToString	`ToString(v:int|float|bool|char|string) - string`	Precise, round-trippable string form (floats via `%.17g`). Chars/bools render as their numeric value (`ToString('A')` → `"65"`); use `str()` for the short display form.	✓ owned[1]
ToRadixString	`ToRadixString(n:int, base:int) - string|nil`	`n` written in an explicit base 2–36 (lowercase digits, `-` for negatives). `nil` if base is out of range. Inverse of `ParseRadix`.	✓ owned[1]

Function	Signature	Description	JIT
ParseBool	`ParseBool(s:string) – bool|nil`	Parse `"true"` / `"false"` (case-insensitive) or `"1"` / `"0"`. **Returns `nil`** on anything else - never raises.	✓
ParseChar	`ParseChar(s:string) – char|nil`	Parse a one-character string to a char. `nil` if empty, multi-character, or not valid UTF-8.	✓
ParseFloat	`ParseFloat(s:string) – float|nil`	Parse a decimal/exponent string. **Returns `nil`** on failure - never raises. (Accepts `inf` / `nan` and hex floats via C `strtod`.)	✓
ParseInt	`ParseInt(s:string) – int|nil`	Parse a signed integer (optional sign, `0x` / `0b` / `0o` prefix, surrounding whitespace). Full `Int64Min…Int64Max` range. **Returns `nil`** on failure - never raises.	✓
ParseUInt	`ParseUInt(s:string) – int|nil`	Parse a non-negative integer (prefixes allowed) that fits the signed 64-bit slot. **Returns `nil`** on failure - never raises.	✓
ParseRadix	`ParseRadix(s:string, base:int) – int|nil`	Parse `s` in an explicit base 2–36; no `0x` / `0b` / `0o` prefix is applied (so `"FF"` with base 16 is 255). `nil` on malformed input or out-of-range base.	✓
TryParseInt	`TryParseInt(x:any) – int|nil`	Coerce **any** value to int: numbers pass through, strings are parsed, anything else is `nil`. Returns the value or `nil`.	✓
TryParseUInt	`TryParseUInt(x:any) – int|nil`	As `TryParseInt`, rejecting negatives. Returns the value or `nil`.	✓
TryParseFloat	`TryParseFloat(x:any) – float|nil`	Coerce any value to float. Returns the value or `nil`.	✓
TryParseBool	`TryParseBool(s:string) – bool`	**Success predicate**: `true` if `s` parses as a bool (see `ParseBool`), else `false`. Use `ParseBool` to recover the value.	✓
TryParseChar	`TryParseChar(s:string) – bool`	**Success predicate**: `true` if `s` is exactly one valid character, else `false`. Use `ParseChar` to recover the value.	✓

> **Numeric range constants** (`Int32Max`, `FloatEpsilon`, …) live in the `Math` module, alongside `Math.PI` and `Math.NaN`.

Char Module

Namespace: **`Char`**

Character classification and conversion. All functions operate on the full 32-bit Unicode codepoint stored inside a `char` value. Classification functions are Unicode-aware - they recognise letters, digits, and whitespace across all major scripts, not just ASCII (see per-function notes below). 18 of the 20 functions run at native speed inside JIT functions (see R11).

Classification - `char – bool`

Function	Signature	Unicode-aware	Description	JIT
IsAlpha	`IsAlpha(c:char) – bool`	✓	Letter in any major script (Latin, Greek, Cyrillic, Arabic, CJK, Hangul, …).	✓
IsDigit	`IsDigit(c:char) – bool`	✓	Decimal digit (Unicode Nd category): ASCII 0–9 and script-native digits (Arabic-Indic, Devanagari, Thai, …).	✓
IsAlNum	`IsAlNum(c:char) – bool`	✓	Letter or decimal digit (Unicode-aware for both).	✓
IsUpper	`IsUpper(c:char) – bool`	✓	Uppercase letter (Latin, Greek, Cyrillic, Armenian, Georgian, Fullwidth, …).	✓
IsLower	`IsLower(c:char) – bool`	✓	Lowercase letter (same script coverage as `IsUpper`).	✓
IsCased	`IsCased(c:char) – bool`	✓	Has an upper/lower case distinction - equivalent to `IsUpper(c) || IsLower(c)`.	✓
IsSpace	`IsSpace(c:char) – bool`	✓	Whitespace: ASCII space/tab/newlines plus Unicode spaces (En Space, Em Space, No-Break Space, …).	✓
IsNewline	`IsNewline(c:char) – bool`	✓	Line-ending character: LF `\n`, CR `\r`, VT, FF, NEL U+0085, Line Separator U+2028, Paragraph Separator U+2029.	✓
IsAscii	`IsAscii(c:char) – bool`	-	Codepoint is in the ASCII range (U+0000–U+007F).	✓
IsPunct	`IsPunct(c:char) – bool`	-	ASCII punctuation only (`. , ! ? " ' ; : – ( ) [ ] { } …`).	✓
IsXDigit	`IsXDigit(c:char) – bool`	-	Hexadecimal digit: 0–9, A–F, a–f (ASCII only - hex is an ASCII concept).	✓
IsPrint	`IsPrint(c:char) – bool`	~	Printable: U+0020–U+007E plus codepoints above U+009F.	✓
IsControl	`IsControl(c:char) – bool`	~	C0 and C1 control characters: codepoint < U+0020 or U+007F–U+009F.	✓

Conversion - `char – char`

Function	Signature	Unicode-aware	Description	JIT
ToUpper	`ToUpper(c:char) – char`	✓	Uppercase mapping; non-letters returned unchanged.	✓
ToLower	`ToLower(c:char) – char`	✓	Lowercase mapping; non-letters returned unchanged.	✓
SwapCase	`SwapCase(c:char) – char`	✓	Upper-lower, lower-upper, all other characters unchanged.	✓

Numeric and UTF-8 helpers - `char – int`

Function	Signature	Description	JIT
DigitValue	`DigitValue(c:char) – int`	Decimal value of a Unicode digit (0–9), or **-1** if the character is not a digit. Works for any Nd script block (Arabic-Indic `'٣'` - 3, Devanagari `'९'` - 9, …).	✓
Utf8Len	`Utf8Len(c:char) – int`	Number of UTF-8 bytes needed to encode the codepoint: 1 (ASCII), 2 (U+0080–U+07FF), 3 (U+0800–U+FFFF), 4 (U+10000+).	✓

Codepoint accessors

Function	Signature	Description	JIT
Code	`Code(c:char) – int`	Raw Unicode codepoint as an integer (`'A'` - 65, `'𝄞'` - 0x4E2D).	✓
FromInt	`FromInt(n:int) – char`	Wrap an integer codepoint as a `char`. No range check performed.	✓

```
// Unicode-aware classification
Char.IsAlpha('ä')   // true  (Latin Extended)
Char.IsAlpha('  ')  // true  (CJK)
Char.IsDigit('٣')  // true  (Arabic-Indic digit 3)
Char.IsUpper('A')  // true  (Greek capital Alpha)

// Case conversion
Char.ToUpper('ä')   // 'Ä'
Char.ToLower('A')   // 'α'
Char.SwapCase('A')  // 'a'

// Numeric helpers
Char.DigitValue('٣')           // 3
Char.DigitValue('a')           // -1
Char.Utf8Len('a')              // 1
Char.Utf8Len('ä')              // 2
Char.Utf8Len('  ')             // 3
Char.Utf8Len(Char.FromInt(0x1F600))  // 4

// Char calls run natively inside JIT functions
fn count_letters(s: string, n: int): int {
    let count: int = 0;
    for (let i: int = 0; i < n; i++) {
        if (Char.IsAlpha(s[i])) { count++; }
    }
    return count;
}
```

Char instance method syntax

All `Char` functions can also be called directly on a `char` value:

```
let c = String.CharAt("hello", 0);   // 'h'
c.IsAlpha()    // true   - same as Char.IsAlpha(c)
c.Code()       // 104    - same as Char.Code(c)
c.ToUpper()    // 'H'    - same as Char.ToUpper(c)
```

Both forms compile to identical bytecode when the variable is typed (`: char` or inferred from a builtin return). Untyped variables fall back to a runtime dispatch.

Crypto

Namespace: **`Crypto`**

Authenticated encryption (XChaCha20-Poly1305), Ed25519 signatures, X25519 key exchange, Argon2id password hashing, HMAC, and a CSPRNG. Public keys and signatures are hex strings.

Function	Signature	Description	JIT
Argon2	`Argon2(password:string|block, salt:string|block, iterations?:int, memKiB?:int) - string`	Argon2id password hash (64-hex). `salt` >= 8 bytes (16 recommended); `iterations` default 3; `memKiB` memory cost in KiB, default 65536 (64 MiB), capped at 1 GiB.	—
ConstantTimeEquals	`ConstantTimeEquals(a:string|block, b:string|block) - bool`	Compare two byte sequences in constant time (no early exit). Use instead of `==` when verifying MACs, signatures, or tokens so equality checks don't leak timing. Only length inequality returns early.	✓
Decrypt	`Decrypt(cipher:string|block, cipherLen:int, key:string|block, nonce:string|block, tag:string) - object`	Decrypt `cipherLen` bytes of XChaCha20-Poly1305 `cipher` with the 32-byte `key`, 24-byte `nonce`, and hex `tag` returned by `Encrypt`. Returns `{Ok:bool, Plain:block}`. On authentication failure `Ok` is `false` and `Plain` is zeroed - always check `Ok` before using `Plain`.	—
Ed25519KeyPair	`Ed25519KeyPair() - object`	Generate an Ed25519 signing key pair. Returns `{PublicKey:string(64 hex), SecretKey:string(128 hex)}`.	—
Ed25519Sign	`Ed25519Sign(secretKey:string, message:string|block) - string`	Sign `message` with a hex secret key; returns a 128-hex signature.	✓ owned[1]
Ed25519Verify	`Ed25519Verify(signature:string, publicKey:string, message:string|block) - bool`	Verify a hex signature against a hex public key and message.	✓
Encrypt	`Encrypt(data:string|block, dataLen:int, key:string|block) - object`	Encrypt `dataLen` bytes of `data` with XChaCha20-Poly1305 under a 32-byte `key`. A fresh 24-byte nonce is generated per call (never supply your own). Returns `{Ok:bool, Cipher:block, Nonce:block, Tag:string(32 hex)}`. Max 256 MB.	—
HmacSha1	`HmacSha1(key:string|block, data:string|block) - string`	Compute HMAC-SHA1 (40-hex). Legacy/interop: TOTP/HOTP 2FA, OAuth1, AWS SigV2.	✓ owned[1]
HmacSha256	`HmacSha256(key:string|block, data:string|block) - string`	Compute HMAC-SHA256 (64-hex).	✓ owned[1]
HmacSha512	`HmacSha512(key:string|block, data:string|block) - string`	Compute HMAC-SHA512 (128-hex). E.g. JWT HS512.	✓ owned[1]
RandomBytes	`RandomBytes(count:int) - block`	Generate `count` cryptographically random bytes (CSPRNG). Raises if `count` is not in `1..67108864`.	—
X25519KeyPair	`X25519KeyPair() - object`	Generate an X25519 key-exchange key pair. Returns `{PublicKey:string(64 hex), SecretKey:string(64 hex)}`.	—
X25519Shared	`X25519Shared(secretKey:string, peerPublicKey:string) - string`	X25519 ECDH; returns the 64-hex raw shared secret. Hash it (e.g. `Hash.Blake2b`) before using it as a key.	✓ owned[1]

Debug

Namespace: `Debug`

Runtime introspection and assertion tools. Available in all builds; overhead is negligible for non-tracing calls.

Function	Signature	Description	JIT
Assert	`Assert(left:any, right:any, ?message:any) - bool`	Verify `left == right`. On mismatch prints both values (plus optional `message`) and a stack trace, then **aborts the VM** (`EXIT_CODE_ERR_ASSERT`) - not a catchable raise. Returns `true` when it holds.	—
AssertTrue	`AssertTrue(condition:any, ?message:any) - bool`	Verify `condition` is truthy. On failure prints it (plus optional `message`) and a stack trace, then **aborts the VM**. Returns `true` when it holds.	—
GuardAddress	`GuardAddress(addr:int)`	Installs an allocator watchpoint that traps when `addr` is touched.	—
Here	`Here(?label:string)`	Prints the current file, line, and function name to stdout (with an optional `label`).	—
Pool	`Pool()`	Prints SLAB allocator statistics and a per-slab dump.	—
Refs	`Refs(value:any) - int`	Returns `value`'s external reference count (`-1` for tagged immediates, which have none).	—
Stack	`Stack()`	Prints the current operand stack to stdout.	—
StackPtr	`StackPtr() - int`	Returns the current operand-stack depth.	—
Value	`Value(v:any)`	Prints a detailed internal representation of a value.	—

Directory

Namespace: `Directory`

Filesystem directory operations.

Function	Signature	Description	JIT
Copy	`Copy(src:string, dst:string) - bool`	Recursively copy directory `src` into `dst` (created if missing; existing `dst` is merged into). Symlinks are copied as links, never dereferenced. `false` if `src` is not a directory or any entry could not be copied.	—
Create	`Create(path:string) - bool`	Create directory at `path`. Returns `false` if already exists.	—
CreateTemp	`CreateTemp(prefix?:string) - string`	Atomically create a uniquely-named directory in the system temp dir and return its path (`mkdtemp`). Optional name prefix (default `"flaris"`) must not contain path separators. Not auto-removed - pair with `DeleteAll`. `nil` on failure.	—
Delete	`Delete(path:string) - bool`	Delete directory (must be empty).	—
DeleteAll	`DeleteAll(path:string) - bool`	Recursively delete a directory and everything under it (`rm -rf`). Symlinks are removed as links, never followed out of the tree. Refuses an empty path or the filesystem root. `true` only if every entry was removed.	—
Ensure	`Ensure(path:string) - bool`	Create directory (and parents) if not exists; no-op if exists.	—
Exists	`Exists(path:string) - bool`	`true` if path exists and is a directory.	—
GetModifiedTime	`GetModifiedTime(path:string) - int`	Returns Unix timestamp of last modification.	—
Glob	`Glob(pattern:string, followSymlinks?:bool) - array`	Recursive glob: each `/`-separated segment supports `*`, `?`, and `[set]`; a bare `**` segment matches zero or more directory levels (`"src/**/*.fls"`). Returns matching paths (files and directories) relative to the current directory (or absolute if the pattern is). Case-sensitive; wildcards do not match a leading `.` (name the dot explicitly). Depth capped at 64. `followSymlinks` defaults to `true`; pass `false` to	—

Function	Signature	Description	JIT
		keep `**` from descending through symlinked directories (POSIX). Explicitly named path components are always followed - only `**` wildcard descent is affected.	
List	`List(path:string) - array`	Returns array of all entry names (files + dirs) in `path`.	—
Move	`Move(src:string, dst:string) - bool`	Rename a directory (atomic within one filesystem; fails across filesystems with no copy+delete fallback).	—
ListDirs	`ListDirs(path:string) - array`	Returns array of subdirectory names only.	—
ListFiles	`ListFiles(path:string, filters?:string) - array`	Returns array of immediate **non-directory** entry names (`stat`'d to exclude subdirectories - the complement of `ListDirs`; use `List` for every entry regardless of kind). Optionally filtered by a comma-separated glob list (`"*.txt, *.md"` - full `*` / `?` / `[set]` wildcards, case-insensitive). Empty array if the directory can't be opened. Non-recursive.	—

Event

Namespace: `Event`

Signals fibers can wait on. Max 64 events (a shared, fixed pool).

Function	Signature	Description	JIT
Create	`Create() - int`	Claims a new event id (`0..63`), or `-1` if the 64-id pool is exhausted. Always check for `-1`.	—
Set	`Set(id:int, value?:any)`	Signal the event; optionally attach a value. Wakes **every** fiber waiting on it (broadcast). Raises `IndexOutOfBounds` for an id outside `0..63`, or `InvalidArgs` for an id that was never created.	—
WaitOne	`WaitOne(id:int, timeoutMs?:int) - any`	Parks the caller until `Set` is called on this event, then returns the value it was set with. With a positive `timeoutMs` it resumes with `nil` once the deadline passes; `0`/omitted waits forever. Raises `IndexOutOfBounds` for an out-of-range id.	—
WaitFor	`WaitFor(ids:array, timeoutMs?:int) - array`	Waits until **all** of the events in `ids` have been set, then returns their values as an array. Use it to start several `ProcessEvent` / `Set` jobs and continue once they've all finished. (An empty list returns `[]` right away.) With a positive `timeoutMs`, if the deadline passes before every event fires it returns `nil` and stops waiting; `0` or omitted waits forever. Raises `IndexOutOfBounds` for an out-of-range id or `InvalidArgs` for a non-int element.	—
Free	`Free(id:int)`	Releases an id back to the pool. Use to reclaim an event you created but never delivered (a dropped job) so the fixed pool is not exhausted. Raises `IndexOutOfBounds` for an out-of-range id.	—
Reset	`Reset(id:int)`	Clears a claimed event's signalled state and value so it can be `Set` and waited on again, without returning the id to the pool. Raises `IndexOutOfBounds` for an out-of-range id.	—

Lifecycle: an event is reclaimed automatically once the last fiber waiting on it has been served, so a fan-out `WaitFor` join does not leak ids. An event you `Create` but never deliver stays claimed until you `Free` it. A `Set` is broadcast to every fiber already parked on the event; a fiber that starts waiting **after** the last waiter was served and the id reclaimed will block (one-shot events are not retained) — subscribe your waiters before signalling.

Note: `WaitFor` returns the values sorted by event id, which may differ from the order you listed the ids in. If you need to tell them apart, use the values themselves or the buffer each job wrote — not the array position.

On timeout the result is `nil` (distinct from the always-non-nil array a successful join returns).

FFI

Namespace: **`Ffi`**

Dynamic loading of native shared libraries (`.so`, `.dylib`). Requires `--unsafe` flag. FFI calls cross a clean isolation boundary - arguments and return values are marshalled through a structured type system; no VM internals are exposed to native code.

Function	Signature	Description	JIT
Load	`Load(path:string, sha256:string|nil, flags?:int) - pointer`	Load shared library at `path`. When `sha256` is a 64-character hex string, the SHA-256 fingerprint of the library is verified before loading - pass `nil` to skip the check. A digest that is not 64 hex characters is a malformed pin and raises `Exception.ChecksumError` rather than loading unverified, as does a mismatch. SHA verification is optional security hardening; libraries distributed without a known hash should pass `nil`. `flags` may combine `Ffi.LAZY` (default), `Ffi.NOW` and `Ffi.GLOBAL`; ignored on Windows. The plugin's `flaris_abi_version` (see `FLARIS_PLUGIN_ABI()` in ffi_object.h) is verified; an unknown ABI raises `Exception.ChecksumError`, and a library exporting none is loaded with a warning. Returns a handle, or `nil` on failure - see `Ffi.LastError`. Same library is never loaded twice - returns cached handle on repeated calls, and loaded libraries stay mapped for the lifetime of the process.	—
GetFunction	`GetFunction(handle:pointer, name:string, sig?:string) - ffi_function`	Retrieve a named symbol from a loaded library as a callable value. When `sig` is provided, argument count, argument types and return type are validated on every call; a mismatched return raises `Exception.TypeMismatch`, and `nil` is always accepted as a return value regardless of the declared type. Returns `nil` if the symbol is missing or `sig` is malformed - see `Ffi.LastError`. Omitting `sig` disables validation (all types accepted).	—
HasFunction	`HasFunction(handle:pointer, name:string) - bool`	Returns `true` if the named symbol exists in the library. Does not allocate. Use to guard optional symbols.	—
SetCallback	`SetCallback(handle:pointer, name:string, fn:fn) - bool`	Register a Flaris function as a named callback. The library must export `flaris_set_callback`. Callbacks are global - use a lib-specific prefix to avoid name collisions across plugins (e.g. `"mylib_ondata"`).	—
RemoveCallback	`RemoveCallback(handle:pointer, name:string) - bool`	Unregister a previously registered callback by name without unloading the library. Useful for one-shot callbacks.	—
GetVersion	`GetVersion(handle:pointer) - int`	Returns the plugin's version as a `uint32` in `0xMMmmppbb` format. Plugin must export `uint32_t flaris_version(void)`. Returns `0` if symbol not found.	—
Call	`Call(handle:pointer, name:string, ...args) - any`	Resolve and call a named symbol in one step without allocating an `ffi_function` object. Accepts up to 4 extra arguments. Calls `dlsym` on every invocation - use `GetFunction` for hot paths.	—

Function	Signature	Description	JIT
CallAsync	`CallAsync(fn:ffi_function, args:array) - any`	Runs a **pure** FFI function (from `GetFunction`) on the I/O thread pool so the calling fiber suspends instead of blocking the scheduler. Must be `await`ed. `args` is the call's argument array; block arguments are refused (they alias VM memory and cannot cross threads). The function must be side-effect-free with respect to the VM: no callbacks, no shared mutable state, thread-safe. A declared return type that does not match resolves to `nil`.	—
LastError	`LastError() - string|nil`	Text of the most recent `Load` / `GetFunction` / `Call` failure, or `nil` if the last one succeeded. Those three clear it on entry, so it always describes the most recent call.	—
LAZY	`LAZY - int`	`dlopen` flag: resolve symbols on first use (the default).	—
NOW	`NOW - int`	`dlopen` flag: resolve every symbol at load, so a missing symbol fails `Load` instead of crashing on first call.	—
GLOBAL	`GLOBAL - int`	`dlopen` flag: make the library's symbols available to subsequently loaded libraries.	—

`Ffi.Load` path resolution. The name resolves in order: (1) the path as given if it exists; (2) the per-user FFI dir - `~/.flaris/ffi` (POSIX) / `%USERPROFILE%\.flaris\ffi` (Windows), or `$FLARIS_FFI` - where a **bare name** gets the platform extension appended (`Ffi.Load("myplugin")` → `~/.flaris/ffi/myplugin.{dylib,so,dll}`); (3) `<name><ext>` in the working directory; (4) the system loader (so `"libsqlite3.so"` still works). Prefer a bare name and install the library to `~/.flaris/ffi` for portable, path-free loading. See ffi.md.

FFI Type Mapping

Flaris type	FfiObject	Notes
`nil`	`FFIVAL_NIL`	Also what an unsupported or too-deeply-nested value marshals to.
`int`	`FFIVAL_INT` (`ival`)	`int64_t`.
`float`	`FFIVAL_FLOAT` (`fval`)	`double`.
`bool`	`FFIVAL_BOOL` (`ival`)	`0` / `1`.
`char`	`FFIVAL_CHAR` (`ival`)	Unicode code point.
`string`	`FFIVAL_STRING` (`string.data`, `string.length`)	**Arguments borrow** the VM's buffer (`FFI_FLAG_BORROWED`): read it freely during the call, never write it or free it, copy it if you need it afterwards. A string **returned** to the VM must be `malloc`'d - the VM takes ownership.
`block`	`FFIVAL_BLOCK` (`block.memory`)	Zero-copy pointer into VM memory, writable by convention. Never free or reallocate. Size is `block_size * block_count`.
`pointer`	`FFIVAL_POINTER`-shaped `ptr`	Opaque handle (e.g. from `Ffi.Load`).
`array`	`FFIVAL_ARRAY` (`array.elements`)	Contiguous `FfiObject` **values**, not pointers. Deep-copied. Avoid in hot paths.
`object`	`FFIVAL_OBJECT` (`object.pairs`)	Contiguous `FfiKV` pairs, values embedded. Deep-copied. **Argument keys borrow** the VM's key strings under the same rules as string arguments. Avoid in hot paths.
`array` of objects	`FFIVAL_TABLE` (return only)	Rowset: column names once + row-major cells. Converts to the same array-of-row-objects shape, hashing each column name once per table instead of once per row (~1.5x faster to marshal). Prefer for anything row-shaped.
`ffi_function`	`FFIVAL_FUNC` (`fn`)	A C function pointer handed back to script. It carries no signature, so it is always called fully dynamically.

Prefer `int` and `float` for high-frequency callback arguments. Use `block` for bulk data. `string` is acceptable for low-frequency events (errors, lifecycle). `array` and `object` should be avoided in callbacks

called at high rate.

Function Signature Format

Signatures passed to `GetFunction` use the syntax:

```
fn(arg_type, arg_type, ...):return_type
```

Types may be unioned with `|`. Zero-argument functions use `fn():return_type`.

Type name	Matches
`any`	Any value
`nil`	Nil
`bool`	Boolean
`int`	Integer (`u8` , `u16` , `u32` , `u64` , `i8` , `i16` , `i32` , `i64` are aliases)
`float`	Float
`char`	Character
`string`	String
`array`	Array
`object`	Object
`block`	Block (raw memory buffer)
`pointer` / `ptr`	Pointer
`function` / `fn`	Flaris function (does **not** match an FFI function - use `callable` or `ffi`)
`builtin`	Builtin function
`ffi`	FFI function
`number` / `numeric`	`int|float`
`callable`	`fn|builtin|ffi`

Examples:

```
let fn = Ffi.GetFunction(lib, "add", "fn(int, int):int");
let fn = Ffi.GetFunction(lib, "process", "fn(string, pointer):bool");
let fn = Ffi.GetFunction(lib, "compute", "fn(number):float");
let fn = Ffi.GetFunction(lib, "reset", "fn():nil");
let fn = Ffi.GetFunction(lib, "anything", "fn(any):any");
```

For a complete plugin development guide including helpers, examples, and ownership rules, see `ffi.md`.

Fiber

Namespace: **`Fiber`**

Cooperative green-thread creation, communication, and lifecycle management.

Function	Signature	Description	JIT
Cancel	`Cancel(f:fiber) - bool`	Request cancellation of a running fiber.	—
Cancello	`CancelIo(f:fiber) - bool`	Abandon `f` 's pending async I/O **without** killing the fiber: it wakes from its `await` with `nil` and `Stream.LastError()` reporting `7` (cancelled), then carries on running. Use it to drop a stalled read while	—

Function	Signature	Description	JIT
		still answering the client — `Cancel` by contrast stops the fiber for good. `false` if `f` was not waiting on I/O.	
FromId	`FromId(id:int) - fiber`	Returns fiber handle from its integer ID.	—
GetMessage	`GetMessage(f:fiber) - any`	Consume and return the oldest queued message (FIFO), or `nil` if the mailbox is empty.	—
HasMessage	`HasMessage(f:fiber) - bool`	`true` if at least one message is queued for `f`.	—
Id	`Id() - int`	Returns current fiber's integer ID.	—
MessageCount	`MessageCount(f:fiber) - int`	Number of messages currently queued in `f`'s mailbox.	—
IsDone	`IsDone(f:fiber) - bool`	`true` if fiber has completed or been cancelled.	—
New	`New(func:fn ) - fiber`	Create a new fiber (not yet started).	—
Resume	`Resume(f:fiber|nil, detached?:bool) - any`	Schedule `f` to run and yield self. Default (attached): the caller parks until `f` yields/finishes and that value is routed back. `detached=true`: the caller just suspends and `f` runs independently. Passing `nil` (or a finished `f`) returns `nil`. The resume value is delivered later by the scheduler.	—
Run	`Run(func:fn ) - int`	Create and immediately start a detached fiber. Returns its integer ID.	—
SendMessage	`SendMessage(f:fiber, msg:any) - bool`	Append `msg` to `f`'s bounded FIFO mailbox (depth 64) and wake it if idle/sleeping. `false` if `f` is missing, finished, or its mailbox is full (older messages are kept, not overwritten); `true` otherwise. Non-blocking.	—
SetQuantum	`SetQuantum(f:fiber, quantum:int) - bool`	Override the scheduling quantum for `f` - the number of checkpoints (loop back-edges and call/return boundaries) before yielding to the scheduler. Clamped to `[0, 10×FIBER_QUANTUM]`; `0` means "use the default".	—
Sleep	`Sleep(ms:int) - nil`	Suspend current fiber for at least `ms` milliseconds (other fibers keep running). Negative `ms` yields immediately.	—
Status	`Status(f:fiber) - int`	Returns status code. Constants: `Fiber.Status.New=0`, `Running=1`, `Suspended=2`, `Finished=3`, `WaitIO=4`, `Yield=5`, `WaitFiber=6`.	—

Fiber instance method syntax

All `Fiber` functions where the fiber is the first argument can be called directly on a `fiber` value:

```
let f = Fiber.New(fn() { /* ... */ });
f.Resume()            // same as Fiber.Resume(f)
f.IsDone()            // same as Fiber.IsDone(f)
f.Status()            // same as Fiber.Status(f)
f.Cancel()            // same as Fiber.Cancel(f)
f.HasMessage()        // same as Fiber.HasMessage(f)
f.GetMessage()        // same as Fiber.GetMessage(f)
f.MessageCount()      // same as Fiber.MessageCount(f)
f.SendMessage("hi")   // same as Fiber.SendMessage(f, "hi")
```

Factory functions (`New`, `Run`, `Id`, `Sleep`) are not available as instance methods. Both forms compile to identical bytecode when the variable is typed (`: fiber` or inferred from a builtin return). Untyped variables fall back to a runtime dispatch.

File

Namespace: `File`

File read, write, and metadata operations.

Function	Signature	Description	JIT
AppendLines	`AppendLines(path:string, lines:array) - bool`	Append array of strings as lines to file.	—
AppendText	`AppendText(path:string, text:string) - bool`	Append text string to file.	—
Copy	`Copy(src:string, dst:string) - bool`	Copy file from `src` to `dst`.	—
CopyAsync	`CopyAsync(src:string, dst:string) - fiber`	Non-blocking `Copy`. Use with `await`; resolves to `true` on success / `nil` on error. Other fibers run while the file is copied.	—
CreateHardLink	`CreateHardLink(target:string, linkPath:string) - bool`	Create a hard link at `linkPath` sharing `target`'s inode. `target` must exist and be on the same filesystem.	—
CreateSymlink	`CreateSymlink(target:string, linkPath:string) - bool`	Create a symbolic link at `linkPath` pointing to `target` (need not exist). On Windows requires Developer Mode / admin.	—
Delete	`Delete(path:string) - bool`	Delete file at `path`.	—
Exists	`Exists(path:string) - bool`	`true` if path exists and is a regular file (symlinks followed). Directories report `false` - use `Directory.Exists`.	—
GetModifiedTime	`GetModifiedTime(path:string) - int`	Returns Unix timestamp of last modification.	—
Lines	`Lines(path:string, callback:fn) - int`	Stream the file line by line, calling `callback(line)` for each (trailing CR/LF stripped) without building an array - the memory-cheap alternative to `ReadLines` for large files. The callback may return `false` to stop early. Returns the number of lines processed, or `nil` if the file can't be opened or the callback raises.	—
Move	`Move(src:string, dst:string) - bool`	Move/rename file.	—
ReadAllBytes	`ReadAllBytes(path:string) - block`	Read entire file as binary block.	—
ReadAllBytesAsync	`ReadAllBytesAsync(path:string) - fiber`	Non-blocking `ReadAllBytes`. Use with `await`. Other fibers run while the file is read.	—
ReadLines	`ReadLines(path:string) - array`	Read file into array of strings (one per line).	—
ReadLink	`ReadLink(path:string) - string`	The target a symbolic link points to (the raw stored target, not a resolved path), or `nil` if not a symlink / unreadable. POSIX only - `nil` on Windows.	—
ReadText	`ReadText(path:string) - string`	Read entire file as UTF-8 string.	—
ReadTextAsync	`ReadTextAsync(path:string) - fiber`	Non-blocking `ReadText`. Use with `await`. Other fibers run while the file is read.	—
Sha256	`Sha256(path:string, raw?:bool) - string`	Compute SHA-256 of file contents. Optional `raw` for binary output.	—
Sha256Async	`Sha256Async(path:string, raw?:bool) - fiber`	Non-blocking `Sha256`. Use with `await`; resolves to the hex string (or 32-byte block if `raw`), or `nil` if the file can't be opened.	—

Function	Signature	Description	JIT
SetMode	`SetMode(path:string, mode:int) – bool`	Set permission bits (POSIX `chmod`, low `07777` of `mode`). On Windows only the write bit is honored. `nil` if it can't be applied.	—
SetModifiedTime	`SetModifiedTime(path:string, unixSeconds:int) – bool`	Set the file's modification time to a Unix timestamp; access time is preserved where the platform allows. `nil` on failure.	—
Size	`Size(path:string) – int`	File size in bytes, or `nil` if the file can't be opened.	—
Stat	`Stat(path:string) – object`	One-syscall metadata: `{Size, Modified, Accessed, Created, IsDir, IsFile, IsSymlink, Mode}`. Times are Unix timestamps; `Mode` is the permission bits (octal 7777 mask); `Created` uses birth time on macOS/BSD, else ctime. Returns `nil` if the path does not exist. A dangling symlink stats as the link itself.	—
Touch	`Touch(path:string) – bool`	Create file if missing; set access + modification time to now (like POSIX `touch`).	—
Truncate	`Truncate(path:string, size:int) – bool`	Resize the file to exactly `size` bytes (extension zero-fills). `false` if `size` is negative or the file can't be resized.	—
WriteAllBytes	`WriteAllBytes(path:string, data:string|block) – bool`	Write binary data to file (overwrite). `false` on open/write failure.	—
WriteAtomic	`WriteAtomic(path:string, data:string|block) – bool`	Durable write: stream into a temp file in the same directory, `fsync`, then `rename()` over the target - a reader never sees a half-written file, and a crash leaves either the old or new content, never a truncated mix. `false` on any failure (temp file removed). Atomic only within one filesystem.	—
WriteLines	`WriteLines(path:string, lines:array) – bool`	Write array of strings as lines (overwrite). `nil` on open/write failure.	—
WriteText	`WriteText(path:string, text:string) – bool`	Write text string to file (overwrite). `nil` on open/write failure.	—
WriteTextAsync	`WriteTextAsync(path:string, text:string) – fiber`	Non-blocking `WriteText`. Use with `await`. Other fibers run while the file is written.	—

FileWatch

Namespace: **`FileWatch`**

File and directory event monitoring. Up to 64 concurrent watchers. Uses `inotify` on Linux, `kqueue` on macOS/BSD, and `ReadDirectoryChangesW` on Windows. The callback fires on every scheduler tick where an event is pending - no fiber is blocked.

Function	Signature	Description	JIT
Open	`Open(path:string, callback:fn) – int`	Watch `path` for changes using the default mode (`"write,create,delete"`). Returns a slot id (≥ 0) or `-1` on failure.	—
Open	`Open(path:string, mode:string, callback:fn) – int`	Watch `path` with explicit mode.	—
Close	`Close(id:int) – bool`	Stop watching and release the slot.	—

Mode tokens - comma-separated, any combination:

Token	Description	Linux	macOS	Windows
`"write"`	File content modified or saved	✓	✓	✓
`"create"`	New files/dirs created or moved in	✓	✓	✓
`"delete"`	File deleted or renamed away	✓	✓	✓
`"attrib"`	Metadata changed (chmod, chown, timestamps)	✓	✓	✓
`"open"`	File opened for reading or writing	✓	-	-
`"all"`	All of the above	✓	✓ (except open)	✓ (except open)

Default mode (no mode argument, or no recognized token) is `"write,create,delete"`. Unrecognized tokens are ignored, not errors - `"open"` on a platform without it simply falls back to whatever else the string names (or the default).

Callback signature: `fn(event:object, id:int)`

Field	Type	Description
`event.event`	string	`"modify"`, `"create"`, `"delete"`, `"rename"`, `"attrib"`, `"open"`, or `"unknown"`
`event.name`	string	Linux/Windows: the changed entry's name relative to the watched directory (may be empty). macOS/BSD: the watched path as passed to `Open`.

Notes:

- The file must exist when `Open` is called. Watching a non-existent path returns `-1`.
- On macOS, editors like `nano` and `vim` save via rename - the file's inode is replaced. After receiving a `"delete"` event, close and re-open the watch to re-attach to the new inode.
- `"open"` mode is Linux-only; elsewhere it is ignored.

Example:

```
var id = FileWatch.Open("./config.json", fn(ev, id) {
    Console.WriteLine(ev.event + " " + ev.name)
})

Fiber.Sleep(30_000)
FileWatch.Close(id)
```

Hash

Namespace: **`Hash`**

Non-cryptographic and cryptographic hashes. Input accepts `string` or `block`. Max input 1 GB.

Function	Signature	Description	JIT
Adler32	`Adler32(data:string|block) - int`	Adler-32 checksum.	✓
Blake2b	`Blake2b(data:string|block, outLen?:int) - string`	BLAKE2b hash (hex string). `outLen` 1-64 bytes, default 32.	✓ owned[1]
Blake2s	`Blake2s(data:string|block) - string`	BLAKE2s hash (hex string).	✓ owned[1]
Crc32	`Crc32(data:string|block) - int`	CRC-32 checksum.	✓
Crc32c	`Crc32c(data:string|block) - int`	CRC-32C (Castagnoli) checksum.	✓

Function	Signature	Description	JIT
Md5	`Md5(data:string|block) - string`	MD5 (hex string). Legacy/interop checksums only (Content-MD5, ETags); **not** for security.	✓ owned[1]
Sha1	`Sha1(data:string|block) - string`	SHA-1 hash (hex string). Legacy; prefer SHA-256/BLAKE2 for security.	✓ owned[1]
Sha256	`Sha256(data:string|block) - string`	SHA-256 hash (hex string).	✓ owned[1]
Sha512	`Sha512(data:string|block) - string`	SHA-512 hash (hex string).	✓ owned[1]
SipHash24	`SipHash24(data:string|block, key:string) - int`	SipHash-2-4 with 16-byte key.	✓
Xxhash32	`Xxhash32(data:string|block, seed?:int) - int`	xxHash-32. Optional seed.	✓
Xxhash64	`Xxhash64(data:string|block, seed?:int) - int`	xxHash-64. Optional seed.	✓

Json

Namespace: `Json`

JSON parsing, serialization, path-based query and mutation, and NDJSON.

Function	Signature	Description	JIT
Canonicalize	`Canonicalize(value:any) - string`	Serialize to compact JSON with object (and instance) keys in a deterministic **sorted** order — for hashing/signing, where equal values must produce identical bytes (hashmap iteration order is not stable across processes). Keys ordered by UTF-8 byte value. Same cycle/depth guarantees as `Stringify`.	—
Deserialize	`Deserialize(json:string, cls:class) - instance|array`	Parse `json` and map it onto instances of `cls` by declared field name: an object yields one instance, an array of objects yields an array of instances. Unknown keys are ignored; absent fields keep their class defaults; inherited fields are included. The `Constructor` is **not** called - values are written directly to the field slots. Nested objects stay plain objects (fields are **not** recursively hydrated into their declared class type). Returns `nil` for malformed JSON, a scalar top level, or an array containing a non-object element.	—
Diff	`Diff(a:object, b:object) - object`	Produce an RFC 7386 merge-patch such that `Merge(copy-of-a, patch)` yields `b`: changed/added keys carry b's value, removed keys map to `null`, nested objects recurse. `nil` if either argument is not an object. Cannot represent a key whose value in `b` is genuinely `null` (null means delete).	—
Exists	`Exists(path:string, value:any) - bool`	`true` if path resolves to a node within `value` (a parsed object/array, or a class instance for key steps).	✓
FastSelect	`FastSelect(path:string, json:string) - any`	Fast path extraction from raw JSON text (no full parse). Dot/bracket notation; `nil` if absent. On duplicate object keys it returns the last (matching `Parse`).	—
Find	`Find(path:string, value:any) - any`	Navigate a parsed value by path. Supports wildcards (`*`, `[*]`). Returns the matched node or `nil`. Key steps descend into class instances (by field name); wildcards apply to plain objects/arrays only.	—
IsValid	`IsValid(json:string) - bool`	`true` if the string is exactly one well-formed JSON document. Disambiguates the case `Parse` cannot (it returns `nil` for both	✓

Function	Signature	Description	JIT
		invalid input and a literal `null`).	
Merge	`Merge(target:object, patch:object) - bool`	Apply an RFC 7386 merge-patch to `target` **in place**: a `null` patch value deletes that key, an object value merges recursively, any other value (including arrays) replaces. `false` if either argument is not an object. Because `null` deletes, a patch cannot set a key to `null`.	—
Parse	`Parse(json:string) - any`	Parse JSON string into a Flaris value (object/array/string/int/float/bool/nil). `nil` on malformed input. Duplicate object keys resolve last-wins; a `\u0000` escape is rejected (Flaris strings cannot hold an embedded NUL). Max depth 256; raises if the document exceeds the internal node cap (`MAX_JSON_NODES`).	—
ParseLines	`ParseLines(text:string) - array`	Parse NDJSON / JSON Lines: one JSON value per line. Blank lines skipped; `nil` if any non-empty line is invalid.	—
Remove	`Remove(root:object|array, path:string) - bool`	Remove the value at `path` (object key or array index, in place). `false` on a missing node or type mismatch.	—
Set	`Set(root:object|array, path:string, value:any) - bool`	Set `value` at `path` in place, creating missing intermediate objects/arrays. `false` on a wrong-type intermediate (never clobbered) or an out-of-range array index (only append-at-length allowed).	—
Stringify	`Stringify(value:any, pretty?:int) - string`	Serialize to JSON. Truthy `pretty` enables 2-space indenting. Non-finite floats (NaN/Inf) serialize as `null`. Class instances serialize as objects with every declared field (own + inherited) by name - the inverse of `Deserialize`. Raises (rather than emit malformed JSON) if `value` contains a reference cycle or nests past the depth cap.	—
StringifyLines	`StringifyLines(array:array) - string`	Serialize each element as a compact JSON value on its own line (NDJSON); inverse of `ParseLines`. Raises if any element is cyclic or over-deep.	—

Math

Namespace: `Math`

Full mathematical library. `number` accepts `int` or `float`.

Domain and pole errors (IEEE). Scalar float functions return the IEEE special value on an out-of-domain or pole input rather than `nil`: domain errors yield `NaN` (`Sqrt(-1)`, `Log(-1)`, `Asin(2)`, `Acosh(0.5)`, `Acoth(0.5)`), and poles yield `±Inf` (`Log(0)` → `-Inf`, `Csc(0)` / `Sec` / `Cot` / `Csch` → `±Inf`, `Atanh(1)` → `Inf`, `LogBase(x,1)` → `Inf`). Detect them with `Math.IsNaN` / `Math.IsFinite`.

Function	Signature	Description	JIT
Abs	`Abs(x:int|float) - int`	Absolute value as integer.	✓
AbsF	`AbsF(x:int|float) - float`	Absolute value as float.	✓
Acos	`Acos(x:int|float) - float`	Arc cosine (radians).	✓
Acosh	`Acosh(x:int|float) - float`	Inverse hyperbolic cosine.	✓
Acot	`Acot(x:int|float) - float`	Arc cotangent.	✓
Acoth	`Acoth(x:int|float) - float`	Inverse hyperbolic cotangent.	✓
Asin	`Asin(x:int|float) - float`	Arc sine (radians).	✓
Asinh	`Asinh(x:int|float) - float`	Inverse hyperbolic sine.	✓
Atan	`Atan(x:int|float) - float`	Arc tangent (radians).	✓

Function	Signature	Description	JIT
Atan2	`Atan2(y:int|float, x:int|float) - float`	Two-argument arc tangent.	✓
Atanh	`Atanh(x:int|float) - float`	Inverse hyperbolic tangent.	✓
Cbrt	`Cbrt(x:int|float) - float`	Cube root.	✓
Ceil	`Ceil(x:int|float) - int`	Ceiling (round up).	✓
Clamp	`Clamp(x:int|float, min:int|float, max:int|float) - float`	Clamp x to [min, max].	✓
CopySign	`CopySign(x:int|float, y:int|float) - float`	Returns magnitude of x with sign of y.	✓
Cos	`Cos(x:int|float) - float`	Cosine.	✓
Cosh	`Cosh(x:int|float) - float`	Hyperbolic cosine.	✓
Cot	`Cot(x:int|float) - float`	Cotangent.	✓
Csc	`Csc(x:int|float) - float`	Cosecant.	✓
Csch	`Csch(x:int|float) - float`	Hyperbolic cosecant.	✓
Deg	`Deg(x:int|float) - float`	Convert radians to degrees.	✓
Exp	`Exp(x:int|float) - float`	e^x.	✓
Floor	`Floor(x:int|float) - int`	Floor (round down).	✓
Gcd	`Gcd(a:int|float, b:int|float) - int`	Greatest common divisor (>= 0; signs ignored). Gcd(0,0)=0.	✓
Hypot	`Hypot(x:int|float, y:int|float) - float`	Euclidean distance sqrt(x²+y²).	✓
IsEven	`IsEven(x:int|float) - bool`	`true` if x is even.	✓
IsFinite	`IsFinite(x:int|float) - bool`	`true` if x is finite (not NaN or Inf). Note: return type is float in registry; treat as bool.	✓
IsInteger	`IsInteger(x:int|float) - bool`	`true` if x has no fractional part. Note: return type is float in registry; treat as bool.	✓
IsNaN	`IsNaN(x:int|float) - bool`	`true` if x is NaN.	✓
IsNegative	`IsNegative(x:int|float) - bool`	`true` if x < 0.	✓
IsOdd	`IsOdd(x:int|float) - bool`	`true` if x is odd.	✓
IsPositive	`IsPositive(x:int|float) - bool`	`true` if x > 0.	✓
Lcm	`Lcm(a:int|float, b:int|float) - int`	Least common multiple (>= 0). Lcm(x,0)=0. May overflow int64 for large inputs.	✓
Log	`Log(x:int|float) - float`	Natural logarithm.	✓
Log10	`Log10(x:int|float) - float`	Base-10 logarithm.	✓
LogBase	`LogBase(x:int|float, base:int|float) - float`	Logarithm with arbitrary base.	✓
Max	`Max(a:int|float, b:int|float) - int`	Larger of a, b as integer.	✓
MaxF	`MaxF(a:int|float, b:int|float) - float`	Larger of a, b as float.	✓
MaxOf	`MaxOf(arr:array) - int|float`	Largest element of a numeric array (type preserved). `nil` if empty or non-numeric.	—
Mean	`Mean(arr:array) - float`	Arithmetic mean of array elements.	✓
Median	`Median(arr:array) - float`	Median of array elements.	✓
Min	`Min(a:int|float, b:int|float) - int`	Smaller of a, b as integer.	✓
MinF	`MinF(a:int|float, b:int|float) - float`	Smaller of a, b as float.	✓
MinOf	`MinOf(arr:array) - int|float`	Smallest element of a numeric array (type preserved). `nil` if empty or non-numeric.	—
Mod	`Mod(a:int|float, b:int|float) - int|float`	Euclidean/floored modulo: result takes sign of b, so Mod(-1,5)=4. `nil` if b=0.	—
Mode	`Mode(arr:array) - float`	Mode of array elements.	✓
Pow	`Pow(base:int|float, exp:int|float) - float`	base^exp.	✓
Product	`Product(arr:array) - int|float`	Product of array elements (int if all-int, else float). `1` if empty; `nil` if non-numeric.	—
Rad	`Rad(x:int|float) - float`	Convert degrees to radians.	✓

Function	Signature	Description	JIT
RandBool	`RandBool() - bool`	Random boolean.	—
RandChoice	`RandChoice(arr:array) - any`	Random element from array.	—
RandFloat	`RandFloat(min:int|float, max:int|float) - float`	Random float in [min, max].	✓
Random	`Random() - float`	Random float in [0.0, 1.0).	—
RandomInt	`RandomInt(min:int|float, max:int|float) - int`	Random integer in [min, max].	✓
RandomNormal	`RandomNormal(mean:int|float, stddev:int|float) - float`	Normal (Gaussian) distribution sample.	✓
Round	`Round(x:int|float) - int`	Round to nearest integer (half-up).	✓
RoundTo	`RoundTo(x:int|float, decimals:int) - float`	Round to `decimals` fractional digits (clamped 0..15).	✓
Sec	`Sec(x:int|float) - float`	Secant.	✓
Sech	`Sech(x:int|float) - float`	Hyperbolic secant.	✓
Sign	`Sign(x:int|float) - int`	Returns -1, 0, or 1.	✓
Sin	`Sin(x:int|float) - float`	Sine.	✓
Sinh	`Sinh(x:int|float) - float`	Hyperbolic sine.	✓
Sqrt	`Sqrt(x:int|float) - float`	Square root.	✓
Stddev	`Stddev(arr:array) - float`	Standard deviation of array elements.	✓
Sum	`Sum(arr:array) - int|float`	Sum of array elements (int if all-int, else float). `0` if empty; `nil` if non-numeric.	—
Tan	`Tan(x:int|float) - float`	Tangent.	✓
Tanh	`Tanh(x:int|float) - float`	Hyperbolic tangent.	✓
Trunc	`Trunc(x:int|float) - float`	Truncate toward zero.	✓
Variance	`Variance(arr:array) - float`	Variance of array elements.	✓
Wrap	`Wrap(x:int|float, min:int|float, max:int|float) - float`	Wrap x into [min, max] range.	✓

Additional scalar functions (less common):

Function	Signature	Description	JIT
Clamp01	`Clamp01(x:int|float) - float`	Clamp to [0.0, 1.0].	✓
Combination	`Combination(n:int|float, r:int|float) - int`	Binomial coefficient C(n,r).	✓
Distance	`Distance(x1:int|float, y1:int|float, x2:int|float, y2:int|float) - float`	2D Euclidean distance.	✓
Expm1	`Expm1(x:int|float) - float`	e^x – 1 (accurate near zero).	✓
Factorial	`Factorial(n:int|float) - int`	n!	✓
Fract	`Fract(x:int|float) - float`	Fractional part of x.	✓
IEEERemainder	`IEEERemainder(x:int|float, y:int|float) - float`	IEEE 754 remainder.	✓
IsClose	`IsClose(a:int|float, b:int|float) - bool`	Approximate equality check.	✓
Lerp	`Lerp(a:int|float, b:int|float, t:int|float) - float`	Linear interpolation.	✓
Log1p	`Log1p(x:int|float) - float`	ln(1 + x) (accurate near zero).	✓
Log2	`Log2(x:int|float) - float`	Base-2 logarithm.	✓
Permutation	`Permutation(n:int|float, r:int|float) - int`	P(n,r) ordered permutations.	✓

Special functions (`Gamma`, `LnGamma`, `Erf`, `Erfc` run at native speed inside JIT functions):

Function	Signature	Description	JIT
Gamma	`Gamma(x:int|float) - float`	Gamma function Γ(x).	✓
LnGamma	`LnGamma(x:int|float) - float`	Natural log of |Γ(x)| (no overflow for large x).	✓
Digamma	`Digamma(x:int|float) - float`	Digamma ψ(x) = Γ'(x)/Γ(x).	✓
Beta	`Beta(a:int|float, b:int|float) - float`	Beta function B(a,b) = Γ(a)Γ(b)/Γ(a+b).	✓
LnBeta	`LnBeta(a:int|float, b:int|float) - float`	Natural log of B(a,b).	✓
Erf	`Erf(x:int|float) - float`	Error function.	✓
Erfc	`Erfc(x:int|float) - float`	Complementary error function 1−erf(x).	✓
GammaP	`GammaP(a:int|float, x:int|float) - float`	Regularized lower incomplete gamma P(a,x) ∈ [0,1]. Chi²/gamma CDFs.	✓
GammaQ	`GammaQ(a:int|float, x:int|float) - float`	Regularized upper incomplete gamma Q(a,x) = 1−P(a,x).	✓
BetaInc	`BetaInc(a:int|float, b:int|float, x:int|float) - float`	Regularized incomplete beta $I_x(a,b)$ ∈ [0,1]. Student-t/F/beta CDFs.	✓

Bit intrinsics (integer operands; all four run at native speed inside JIT functions):

Function	Signature	Description	JIT
BitCount	`BitCount(n:int) - int`	Number of set bits in the 64-bit two's-complement representation of n.	✓
BitLength	`BitLength(n:int) - int`	Minimum bits to represent |n| (0 for n=0). Equivalent to Python `int.bit_length()`.	✓
LeadingZeros	`LeadingZeros(n:int) - int`	Number of leading zero bits in the 64-bit representation (64 when n=0).	✓
TrailingZeros	`TrailingZeros(n:int) - int`	Number of trailing zero bits in the 64-bit representation (64 when n=0).	✓

Constants:

Constant	Value
`Math.PI`	`3.14159265358979323846`
`Math.TAU`	`6.28318530717958647692`
`Math.E`	`2.71828182845904523536`
`Math.Infinity`	`+Inf`
`Math.NaN`	`NaN`

Range constants: `Int8Min/Max`, `Int16Min/Max`, `Int32Min/Max`, `Int64Min/Max`, `IntMin/Max`, `UInt8Min/Max`, `UInt16Min/Max`, `UInt32Min/Max`, `UInt64Min/Max`, `Float32Min/Max/Epsilon`, `Float64Min/Max/Epsilon`, `FloatMin/Max/Epsilon`, `Float32Lowest`, `Float64Lowest`, `FloatLowest`.

> **Notes.** `UInt64Max` is capped at `Int64Max` (`9223372036854775807`) because constants live in a signed 64-bit slot. For floats, `*Min` is the smallest *positive normal* (like C's `FLT_MIN` / `DBL_MIN`), **not** the most-negative value - use `*Lowest` (= −`*Max`) for that.

Random

Namespace: **`Random`**

Seeded, deterministic pseudo-random streams (PCG32). The same seed always yields the same sequence - use for reproducible tests, simulations, and procedural generation. Each generator is an independent stream;

`Math.Random` and friends stay on the global CSPRNG. **Not cryptographic** - use `Crypto.RandomBytes` for keys and tokens.

Function	Signature	Description	JIT
New	`New(seed?:int) - object`	Create a generator. Without a seed the stream is seeded from the OS CSPRNG (fast but non-reproducible).	—

Generator methods:

Method	Description
`r.Next()`	Uniform int in `[0, 2^32)` - one raw PCG32 output.
`r.NextInt(min, max)`	Uniform int in `[min, max]` (inclusive, like `Math.RandomInt`). Unbiased (rejection sampling). `nil` if `min > max`.
`r.NextFloat()`	Uniform float in `[0, 1)` with 53-bit precision.
`r.NextNormal(mean?, stddev?)`	Normally distributed float (Box-Muller). Defaults: mean 0, stddev 1.

```
let r = Random.New(42);
r.NextInt(1, 6);     // same die roll on every run
r.NextFloat();       // deterministic [0,1)
```

Memory

Namespace: `Memory`

Unsafe raw memory access. Requires `--unsafe` flag. `ptr` = `int\|string\|block\|pointer` (any pointer-like value - see legend).

> **Region requirement.** Every access is guarded: the `[ptr, ptr+width)` span must lie inside a **registered** region, otherwise the call returns `nil`/no-op (or raises `Forbidden/Invalid memory-access`). Registered regions come from `Memory.Alloc`, `Memory.Pin`, or a `block`'s backing store (`Buffer.Create`/`GetAddress`). A plain `string` address is **not** registered, so passing a bare string is rejected — copy it into a block (`Buffer.FromString`) or `Pin` its address first.

Function	Signature	Description	JIT
Alloc	`Alloc(size:int) - int`	Allocate `size` bytes; returns the raw address. `nil` if `size <= 0` or exceeds the 2 GiB per-allocation cap. Caller must `Free`.	—
BitClear	`BitClear(ptr:int|string|block|pointer, bit:int) - nil`	Clear bit `bit` at address.	✓ `--unsafe`
BitSet	`BitSet(ptr:int|string|block|pointer, bit:int) - nil`	Set bit `bit` at address.	✓ `--unsafe`
BitTest	`BitTest(ptr:int|string|block|pointer, bit:int) - int`	Return 0 or 1 for bit at address.	✓ `--unsafe`
BitToggle	`BitToggle(ptr:int|string|block|pointer, bit:int) - nil`	Toggle bit at address.	✓ `--unsafe`
Compare	`Compare(a:int|string|block|pointer, b:int|string|block|pointer, len:int) - int`	memcmp of `len` bytes. Returns <0, 0, >0.	✓ `--unsafe`

Function	Signature	Description	JIT
Copy	`Copy(dst:int|string|block|pointer, src:int|string|block|pointer, len:int) - nil`	`memcpy` `len` bytes — the spans must **not** overlap (use `Move` if they might). Region-guarded.	✓ `--unsafe`
Move	`Move(dst:int|string|block|pointer, src:int|string|block|pointer, len:int) - nil`	`memmove` `len` bytes; the `dst` and `src` spans **may** overlap. Region-guarded.	✓ `--unsafe`
Fill	`Fill(addr:int|string|block|pointer, value:int, len:int) - nil`	Set `len` bytes at `addr` to byte `value` (0–255). Region-guarded; `nil` /no-op out of region or on a bad value.	✓ `--unsafe`
Free	`Free(addr:int) - nil`	Free previously allocated memory.	—
IsLittleEndian	`IsLittleEndian() - bool`	`true` if host is little-endian.	—
Pin	`Pin(addr:int, size:int) - int`	Register an externally-owned region (e.g. an FFI buffer) so the peek/poke guards accept it; returns `addr`. Does not allocate or take ownership. **Must be paired with `Unpin`** — a region left pinned at exit is reported as a leaked block region (and fails under `--mem`).	—
Process	`Process(addr:int, size:int, chunkSize:int, func:fn ) - bool`	Iterate memory in chunks calling `fn(ptr, len)`.	—
ProcessCallback	`ProcessCallback(fn, address:int, start:int, len:int, cb)`	Processes a raw memory range with your worker function `fn(address, start, len)` (which reads/writes it via `Memory.Read*` / `Write*`), then calls `cb(address, result)` when finished. Runs on another CPU core when `fn` is simple enough (only `Memory.Read*` / `Write*` on that address, plain number math, no calls or allocations) — otherwise runs normally, same result. **Don't allocate or free memory while the job is running.**	✓ `--unsafe`
ProcessEvent	`ProcessEvent(fn, address:int, start:int, len:int, eventId:int)`	Like `ProcessCallback`, but signals event `eventId` with the result instead of calling a callback. Wait for several with `Event.WaitFor([ids])`.	✓ `--unsafe`
Read8	`Read8(ptr:int|string|block|pointer, offset?:int) - int`	Read 1 byte at ptr+offset.	✓ `--unsafe`
Read16	`Read16(ptr:int|string|block|pointer, offset?:int) - int`	Read 2 bytes (little-endian).	✓ `--unsafe`
Read32	`Read32(ptr:int|string|block|pointer, offset?:int) - int`	Read 4 bytes (little-endian).	✓ `--unsafe`
Read64	`Read64(ptr:int|string|block|pointer, offset?:int) - int`	Read 8 bytes (little-endian).	✓ `--unsafe`
ReadBytes	`ReadBytes(ptr:int|string|block|pointer, offset?:int) - block`	Read a NUL-terminated byte run at `ptr` (+ `offset`) into a new block (excluding the terminator). The scan is bounded to the registered region. `nil` out of region or with no terminator inside it.	✓ owned[1] `--unsafe`
ReadFloat32	`ReadFloat32(ptr:int|string|block|pointer, offset?:int) - float`	Read IEEE-754 single.	✓ `--unsafe`

Function	Signature	Description	JIT
ReadFloat64	`ReadFloat64(ptr:int|string|block|pointer, offset?:int) - float`	Read IEEE-754 double.	✓ `--unsafe`
ReadString	`ReadString(ptr:int|string|block|pointer, offset?:int) - string`	Read a NUL-terminated string from `ptr` (+ `offset`). The scan is bounded to the registered region (never runs off the allocation). `nil` out of region or with no terminator inside it.	✓ owned[1] `--unsafe`
Swap16	`Swap16(v:int) - int`	Byte-swap 16-bit value.	✓ `--unsafe`
Swap32	`Swap32(v:int) - int`	Byte-swap 32-bit value.	✓ `--unsafe`
Swap64	`Swap64(v:int) - int`	Byte-swap 64-bit value.	✓ `--unsafe`
Unpin	`Unpin(addr:int) - bool`	Remove a region previously registered with `Pin`; returns `true` if it was tracked. Does not free the underlying memory.	—
Write8	`Write8(ptr:int|string|block|pointer, value:int, offset?:int) - nil`	Write 1 byte.	✓ `--unsafe`
Write16	`Write16(ptr:int|string|block|pointer, value:int, offset?:int) - nil`	Write 2 bytes (little-endian).	✓ `--unsafe`
Write32	`Write32(ptr:int|string|block|pointer, value:int, offset?:int) - nil`	Write 4 bytes (little-endian).	✓ `--unsafe`
Write64	`Write64(ptr:int|string|block|pointer, value:int, offset?:int) - nil`	Write 8 bytes (little-endian).	✓ `--unsafe`
WriteBytes	`WriteBytes(ptr:int|string|block|pointer, s:string, offset?:int) - nil`	Write `s`'s bytes (WITHOUT a NUL terminator) at `ptr` (+ `offset`). The byte count comes from `strlen`, so an embedded NUL truncates. Region-guarded.	✓ `--unsafe`
WriteFloat32	`WriteFloat32(ptr:int|string|block|pointer, value:float, offset?:int) - nil`	Write IEEE-754 single.	✓ `--unsafe`
WriteFloat64	`WriteFloat64(ptr:int|string|block|pointer, value:float, offset?:int) - nil`	Write IEEE-754 double.	✓ `--unsafe`
WriteString	`WriteString(ptr:int|string|block|pointer, s:string, offset?:int) - nil`	Write `s`'s bytes plus a NUL terminator at `ptr` (+ `offset`). Region-guarded.	✓ `--unsafe`

Net

Namespace: **`Net`**

Network utility functions: DNS resolution, IP parsing/conversion, address classification, CIDR math, and connectivity checks. Available on all platforms. Synchronous resolvers (`ResolveDNS`, `ResolveIPv4`, `ResolveIPv6`, `ReverseDNS`) block the single-threaded VM for the duration of the lookup — prefer the `*Async` forms on hot/UI paths.

Function	Signature	Description	JIT
BytesToIP	`BytesToIP(bytes:array) - string|nil`	Convert a 4- (IPv4) or 16-element (IPv6) byte array to an IP string. `nil` for any other length.	—
GetHostname	`GetHostname() - string|nil`	Returns machine hostname, or `nil` on error.	—
GetLocalIP	`GetLocalIP() - string|nil`	Returns primary local IP address (IPv4 preferred; falls back to IPv6). `nil` if fully offline.	—

Function	Signature	Description	JIT
IpInRange	`IpInRange(ip:string, cidr:string) - bool`	`true` if `ip` falls inside `cidr` (e.g. `"10.1.2.3", "10.0.0.0/8"`). A v4 address is never inside a v6 range. `false` on malformed input.	—
IpToBytes	`IpToBytes(ip:string) - array|nil`	Convert an IP string to its byte array (4 or 16 ints). `nil` if not a valid literal.	—
IPVersion	`IPVersion(ip:string) - int`	`4`, `6`, or `0` if `ip` is not a valid literal.	—
IsLinkLocal	`IsLinkLocal(ip:string) - bool`	`true` for `169.254.0.0/16` or `fe80::/10`.	—
IsLoopback	`IsLoopback(ip:string) - bool`	`true` for `127.0.0.0/8` or `::1`.	—
IsMulticast	`IsMulticast(ip:string) - bool`	`true` for `224.0.0.0/4` or `ff00::/8`.	—
IsPrivate	`IsPrivate(ip:string) - bool`	`true` for RFC1918 (`10/8`, `172.16/12`, `192.168/16`) or IPv6 ULA `fc00::/7`. IPv4-mapped IPv6 is unwrapped first.	—
IsReachable	`IsReachable(host:string, port:int, timeoutMs?:int) - bool`	TCP reachability check. Resolves both IPv4 and IPv6 and tries each under one total deadline. `timeoutMs` defaults to 1000 (clamped to a sane range).	—
IsReachableAsync	`IsReachableAsync(host:string, port:int, timeoutMs?:int) - fiber`	Non-blocking `IsReachable`. Use with `await`; resolves to a bool.	—
IsValidIP	`IsValidIP(s:string) - bool`	`true` if string is a valid IPv4 or IPv6 literal.	—
IsValidIPv4	`IsValidIPv4(s:string) - bool`	`true` only for a valid IPv4 literal.	—
IsValidIPv6	`IsValidIPv6(s:string) - bool`	`true` only for a valid IPv6 literal.	—
ParseCIDR	`ParseCIDR(cidr:string) - object|nil`	Parse `addr/prefix` into `{ network, prefix, version }` (plus `netmask`, `broadcast` for IPv4). `nil` on malformed input or out-of-range prefix.	—
ResolveDNS	`ResolveDNS(host:string) - array`	Resolve hostname to array of IP strings (IPv4 and IPv6), capped at 32. Empty array on failure.	—
ResolveDNSAsync	`ResolveDNSAsync(host:string) - fiber`	Non-blocking `ResolveDNS`. Use with `await`. Other fibers run during the DNS lookup.	—
ResolveIPv4	`ResolveIPv4(host:string) - array`	Resolve to array of IPv4 address strings.	—
ResolveIPv4Async	`ResolveIPv4Async(host:string) - fiber`	Non-blocking `ResolveIPv4`. Use with `await`. Other fibers run during the DNS lookup.	—
ResolveIPv6	`ResolveIPv6(host:string) - array`	Resolve to array of IPv6 address strings.	—
ResolveIPv6Async	`ResolveIPv6Async(host:string) - fiber`	Non-blocking `ResolveIPv6`. Use with `await`. Other fibers run during the DNS lookup.	—
ReverseDNS	`ReverseDNS(ip:string) - string|nil`	Reverse lookup: IP → hostname. `nil` on a bad literal / no PTR record.	—
ReverseDNSAsync	`ReverseDNSAsync(ip:string) - fiber`	Non-blocking `ReverseDNS`. Use with `await`; resolves to the hostname string, or `nil` on a bad literal / no PTR record.	—

Object

Namespace: **`Object`**

Introspection and manipulation of `object` values, plus read-only reflection over `instance` and `class` values.

`Keys`, `Values`, `Count` and `Entries` accept an `instance` or `class` and reflect its declared **fields** (base-first, in declaration order): the field names, an instance's current field values (a class's field defaults), and the field

count. The mutating and higher-order helpers (`Merge`, `Delete`, `Clear`, `Select`, `Where`, `Pick`, `MapKeys`, `Invert`, `Reduce`, `Any`, `All`) are **object-only** — a class's schema is sealed.

Property iteration order follows the hashmap's bucket order, which is salted per process (hash-flood defense) and therefore **not stable across runs**; use `Collections.OrderedMap` when insertion order matters. Field reflection order is the class's declaration order.

Function	Signature	Description	JIT
Clear	`Clear(obj:object) - bool`	Remove all properties from `obj`. Mutates. Raises if `obj` is frozen.	—
Clone	`Clone(value:any) - any`	Deep clone: scalars, strings, arrays (including flat `[float]`), objects, classes and instances are copied recursively; functions, builtins, modules and exceptions are shared (ref-counted, not copied). A **cyclic** value (or nesting past the clone-depth limit) raises `Exception.NestingError` rather than looping forever.	—
Count	`Count(obj:object|instance|class) - int`	Number of properties (object) or declared fields (instance/class).	✓
Delete	`Delete(obj:object, key:string) - bool`	Delete property `key`. Returns `true` if existed. Raises if `obj` is frozen.	—
Entries	`Entries(obj:object|instance|class) - array`	Array of `[key, value]` pairs (object properties, or field name → value).	—
Freeze	`Freeze(obj:object) - bool`	Makes `obj` read-only. Any subsequent write throws — including via `Merge` / `Delete` / `Clear`. Shallow only. Returns `false` for a non-object. Query with `IsFrozen`.	—
FromEntries	`FromEntries(pairs:array) - object`	Inverse of `Entries`: builds an object from an array of `[key, value]` pairs (string keys). A later duplicate key overwrites the earlier. `nil` for a malformed entry.	—
FromKeys	`FromKeys(arr:array, value:any) - object`	Creates new object from string key array, all set to `value`.	—
All	`All(obj:object, func:fn) - bool`	Returns `true` if `fn(key, value)` is truthy for every entry. Returns `true` for an empty object.	—
Any	`Any(obj:object, func:fn) - bool`	Returns `true` if `fn(key, value)` is truthy for at least one entry. Returns `false` for an empty object.	—
HasKey	`HasKey(obj:object, key:string) - bool`	`true` if property `key` exists.	✓
Invert	`Invert(obj:object) - object`	Returns a new object with keys and values swapped. Values are coerced to string to become keys.	—
IsFrozen	`IsFrozen(obj:object) - bool`	`true` if `obj` was frozen by `Freeze`. `false` for an unfrozen object or a non-object.	✓
IsNil	`IsNil(val:any) - bool`	`true` if `val` is `nil`. Single-arg; usable as a fast-path callback to `Array.Where`, `Array.Find`, `Array.Count`, etc.	✓
Keys	`Keys(obj:object|instance|class) - array`	Array of property names (object) or field names (instance/class).	—
MapKeys	`MapKeys(obj:object, func:fn) - object`	Returns a new object with keys transformed by `fn(key)`. Values are kept; new keys are coerced to string. Builtin functions (e.g. `String.ToUpper`) use the fast path.	—
Merge	`Merge(dst:object, src:object) - bool`	Copy all `src` properties into `dst`. Mutates `dst`. Shallow. Raises if `dst` is frozen.	—
NotNil	`NotNil(val:any) - bool`	`true` if `val` is not `nil`. Single-arg; usable as a fast-path callback to `Array.Where`, `Array.Find`, `Array.Count`, etc.	✓

Function	Signature	Description	JIT
Pick	`Pick(obj:object, keys:array) - object`	Returns new object with only the listed keys.	—
Reduce	`Reduce(obj:object, func:fn, initial:any) - any`	Folds over entries: `acc = fn(acc, key, value)` starting from `initial`.	—
Select	`Select(obj:object, func:fn ) - object`	Returns new object with values transformed by `fn(key, value)`.	—
TryGet	`TryGet(obj:object, key:string, default:any) - any`	Returns `obj[key]` if present, otherwise `default`.	—
Values	`Values(obj:object|instance|class) - array`	Array of property values (object) or field values (instance/class).	—
Where	`Where(obj:object, func:fn ) - object`	Returns new object containing only keys where `fn(key, value)` is truthy.	—

Object instance method syntax

All `Object` functions where the object is the first argument can be called directly on an `object` value:

```
var o = { name: "Alice", age: 30 };
o.Keys()                          // ["name", "age"]  - same as Object.Keys(o)
o.Values()                        // ["Alice", 30]
o.HasKey("age")                   // true
o.Count()                         // 2
o.Delete("age")                   // same as Object.Delete(o, "age")
o.Merge({ city: "Stockholm" })    // same as Object.Merge(o, ...)
o.Where(fn(k, v) { return v != nil; })
o.Select(fn(k, v) { return String.ToUpper(v); })
```

Functions that do not take an object as first argument (`FromKeys`, `FromEntries`) are not available as instance methods. Both forms compile to identical bytecode when the variable is typed (`: object` or inferred). Untyped variables fall back to a runtime dispatch.

Type

Namespace: **`Type`**

Integer constants representing every runtime type. Used with the `type()` built-in, which returns the type of a value as an `int` bitmask.

```
type(42)      == Type.Int;       // true
type("hello") == Type.String;    // true
(type(x) & Type.Callable ) != 0; // true for functions, builtins, bound methods, FFI
(type(x) & Type.Number) != 0;    // true for int or float

Type.Is(x, Type.Callable);               // true for any function type
Type.Is(x, Type.Int | Type.Float);       // ad-hoc union without a named composite

Type.Name(42);                           // "int"
```

```
Type.Name([]);                          // "array"

Type.Assert(x, Type.Int);              // throws if x is not int
Type.Assert(x, Type.Number, "expected a number");
```

Functions

Function	Signature	Description	JIT
Assert	`Assert(x:any, mask:int, msg?:string) - bool`	Throws `InvalidArgs` if `type(x)` does not match `mask`. Optional custom message. Returns `true` on success.	✓
Is	`Is(x:any, mask:int) - bool`	Returns `true` if `type(x)` matches any bit in `mask`. Use with composite constants.	✓
IsArray	`IsArray(x:any) - bool`	`true` if `x` is an array. Single-arg; usable as a fast-path callback to `Array.Where`, `Array.Select`, `Array.Count`, etc.	✓
IsBlock	`IsBlock(x:any) - bool`	`true` if `x` is a memory block/typed buffer.	✓
IsBool	`IsBool(x:any) - bool`	`true` if `x` is a bool.	✓
IsCallable	`IsCallable(x:any) - bool`	`true` if `x` is any callable: function, builtin, **bound method**, or FFI. Superset of `IsFunction`.	✓
IsChar	`IsChar(x:any) - bool`	`true` if `x` is a character.	✓
IsClass	`IsClass(x:any) - bool`	`true` if `x` is a class definition. For "is `x` an instance of class `C`" use `Class.InstanceOf`.	✓
IsException	`IsException(x:any) - bool`	`true` if `x` is an exception object.	✓
IsFiber	`IsFiber(x:any) - bool`	`true` if `x` is a fiber.	✓
IsFloat	`IsFloat(x:any) - bool`	`true` if `x` is a float.	✓
IsFunction	`IsFunction(x:any) - bool`	`true` if `x` is a plain function, builtin, or FFI function. Does **not** include bound methods - use `IsCallable` for those.	✓
IsInstance	`IsInstance(x:any) - bool`	`true` if `x` is a class instance (of any class). For a specific class use `Class.InstanceOf`.	✓
IsInt	`IsInt(x:any) - bool`	`true` if `x` is an integer.	✓
IsModule	`IsModule(x:any) - bool`	`true` if `x` is a built-in module.	✓
IsNil	`IsNil(x:any) - bool`	`true` if `x` is `nil`.	✓
IsNumber	`IsNumber(x:any) - bool`	`true` if `x` is an integer or float.	✓
IsObject	`IsObject(x:any) - bool`	`true` if `x` is a plain object.	✓
IsStream	`IsStream(x:any) - bool`	`true` if `x` is a file/network stream.	✓
IsString	`IsString(x:any) - bool`	`true` if `x` is a string.	✓
Name	`Name(x:any) - string`	Returns the type as a human-readable string: `"int"`, `"array"`, `"fiber"`, etc. Single-arg; usable as a callback.	✓ owned[1]
NameOf	`NameOf(code:int) - string`	Returns the readable name of a `Type.*` code or a composite mask, e.g. `NameOf(Type.Number)` - `"int|float"`. The inverse of `Of`.	✓ owned[1]
Of	`Of(x:any) - int`	Returns the type bitmask of `x` - the referenceable equivalent of the `type()` built-in. Single-arg; usable as a callback (e.g. `Array.SortBy(items, Type.Of)`).	✓

> The numeric values below are implementation-defined bit flags. Always compare against the `Type.*` constants (`type(x) == Type.Int`, `Type.Is(x, Type.Number)`), never against hardcoded hex - the underlying bits may change between releases.

Primitive Types

Constant	Value	Matches
`Type.Nil`	`0x000001`	`nil`
`Type.Bool`	`0x000080`	`true`, `false`
`Type.Char`	`0x000002`	character literals
`Type.Int`	`0x000008`	integers
`Type.Float`	`0x000004`	floating-point numbers
`Type.String`	`0x000010`	strings

Collection Types

Constant	Value	Matches
`Type.Array`	`0x000040`	arrays
`Type.Object`	`0x000020`	plain objects

Callable Types

Constant	Value	Matches
`Type.Function`	`0x0010000`	user-defined functions
`Type.BuiltinFunction`	`0x0020000`	built-in functions
`Type.FFIFunction`	`0x0800000`	FFI-loaded native functions
`Type.BoundMethod`	`0x0100000`	bound instance methods

Object-Oriented Types

Constant	Value	Matches
`Type.Class`	`0x1000000`	class definitions
`Type.Instance`	`0x2000000`	class instances
`Type.Module`	`0x0040000`	built-in modules

Runtime Types

Constant	Value	Matches
`Type.Fiber`	`0x0400000`	fibers
`Type.Exception`	`0x0080000`	exception objects
`Type.Stream`	`0x4000000`	file/network streams
`Type.Block`	`0x0000100`	memory blocks
`Type.Pointer`	`0x0200000`	raw pointers (internal)

Composite Constants

These are bitmask unions - use `&` not `==`.

Constant	Matches
`Type.Number`	`int` or `float`
`Type.Numeric`	`int`, `float`, `bool`, or `char`
`Type.Callable`	any callable: fn, builtin, bound method, FFI
`Type.ObjectLike`	`object`, `instance`, `class`, or `module`

Os

Namespace: `Os`

Process, environment, and system interface. Functions marked **unsafe** require the `--unsafe` flag.

Function	Signature	Description	JIT
Arch	`Arch() - string`	Returns the CPU architecture: `"x86_64"`, `"x86"`, `"mips"`, `"arm64"`, `"arm"`, or `"unknown"`.	—
Args	`Args() - array`	Returns CLI argument strings as passed to the script.	—
Bits	`Bits() - int`	Returns the pointer width of the platform in bits. Flaris ships 64-bit-only, so this currently always returns `64`.	—
Chdir	`Chdir(path:string) - bool`	Change working directory. Process-global - affects every fiber.	—
GetCwd	`GetCwd() - string`	Returns the current working directory, or `nil` if it cannot be read (e.g. the directory was removed, or its path exceeds 4096 bytes). Pairs with `Chdir`.	—
Exec	`Exec(path:string, args?:array) - nil`	Replace the current process image with `path` via `execvp`. Only returns on error. Does not require `--unsafe`.	—
Execute	`Execute(cmd:string) - string`	Run a shell command via `popen`; returns stdout+stderr merged as a string. Returns `nil` on `popen` failure, output exceeding 16 MB, or a `pclose` error; the command's own non-zero exit status does **not** map to `nil` (the captured output is returned regardless). Does **not** require `--unsafe`. **The command string is passed directly to the shell with no escaping or sanitization - never pass untrusted or externally-supplied input** (command-injection risk). To run a program with separate arguments and no shell, use `Os.Spawn` / `Os.RunEx` (which capture output via `execvp`) or `Os.Exec` (which replaces the image).	—
ExecuteAsync	`ExecuteAsync(cmd:string) - fiber`	Non-blocking `Execute`. Use with `await`. Other fibers run while the command runs. Same security warning as `Execute` applies.	—
Getenv	`Getenv(name:string) - string`	Read environment variable. Returns `nil` if not set.	—
GetArgValue	`GetArgValue(name:string, default?:string) - string`	Extract value from CLI arg matching `name=value`. Returns `default` (or `nil`) if not found.	—
Gid	`Gid() - int`	Returns the real group ID of the process.	—
IsRoot	`IsRoot() - bool`	`true` if the process is running as root (UID 0).	—
Kill	`Kill(pid:int, signal:int) - bool`	Send a signal to a process. Use standard signal numbers (e.g. `15` for SIGTERM, `9` for SIGKILL). Requires `--unsafe`.	—
Name	`Name() - string`	Returns OS name: `"Windows"`, `"Linux"` or `"macOS"`.	—
Pid	`Pid() - int`	Returns the current process ID.	—
Ppid	`Ppid() - int`	Returns the parent process ID.	—
Setenv	`Setenv(name:string, value:string) - bool`	Set environment variable.	—
Sleep	`Sleep(ms:int) - bool`	Sleep current OS thread for `ms` milliseconds. Blocks all fibers - prefer `Fiber.Sleep`.	—
TempDir	`TempDir() - string`	Returns path to system temp directory.	—
Uid	`Uid() - int`	Returns the real user ID of the process.	—
Wait	`Wait(pid:int) - int`	Wait (blocking) for child process to exit. Returns its exit code, the negative signal number if killed by a signal, or `nil` on `waitpid` error.	—
Spawn	`Spawn(cmd:string, args?:array) - object`	Fork and exec `cmd` with separate stdin/stdout/stderr pipes. Returns `{Pid:int, Stdin:int, Stdout:int, Stderr:int}` where the int values are raw file descriptors for use with	—

Function	Signature	Description	JIT
		`ReadPipe` / `WritePipe` / `ClosePipe`. Returns `nil` on error. Stdout and stderr fds are non-blocking.	
ReadPipe	`ReadPipe(fd:int) - string`	Non-blocking read from a pipe fd returned by `Spawn`. Returns `nil` when no data is available or the pipe is closed.	—
WritePipe	`WritePipe(fd:int, data:string) - int`	Write `data` to a pipe fd (stdin of a spawned process). Returns bytes written, or `-1` on error.	—
ClosePipe	`ClosePipe(fd:int) - bool`	Close a pipe fd. Call on stdin to signal EOF to the child process.	—
IsAlive	`IsAlive(pid:int) - bool`	Returns `true` if the process with `pid` is still running. Non-blocking. Does not reap the child - safe to call before `TryWait` / `Wait`.	—
TryWait	`TryWait(pid:int) - object`	Non-blocking wait. Returns `{Alive:bool, Exit:int}`. When `Alive` is `false` the child has been reaped and `Exit` holds the exit code. Prefer over polling `IsAlive` + `Wait` to avoid double-reap.	—
KillChild	`KillChild(pid:int, signal:int) - bool`	Send `signal` to a process previously spawned via `Os.Spawn`. Does **not** require `--unsafe`. Returns `false` if `pid` was not spawned by this VM instance. For sending signals to arbitrary PIDs use `Os.Kill` (requires `--unsafe`).	—
RunEx	`RunEx(cmd:string, args?:array) - object`	Run `cmd` and wait for it to finish. Returns `{Exit:int, Stdout:string, Stderr:string}` with stdout and stderr captured separately. Returns `nil` on fork/spawn failure.	—
GetEnvAll	`GetEnvAll() - object`	Returns all environment variables as an object `{NAME: "value", ...}`.	—

Path

Namespace: `Path`

Cross-platform path string manipulation.

Function	Signature	Description	JIT
ChangeExtension	`ChangeExtension(path:string, ext:string) - string`	Replace file extension.	—
Combine	`Combine(part1:string, part2:string, ...more) - string`	Join path segments with separator. Up to 15 arguments.	—
GetCurrentDir	`GetCurrentDir() - string`	Returns current working directory.	—
GetDirectoryName	`GetDirectoryName(path:string) - string`	Returns directory portion of path.	—
GetExtension	`GetExtension(path:string) - string`	Returns extension including dot (e.g. `".txt"`).	—
GetFileName	`GetFileName(path:string) - string`	Returns filename with extension.	—
GetFileNameWithoutExtension	`GetFileNameWithoutExtension(path:string) - string`	Returns filename without extension.	—
GetFullPath	`GetFullPath(path:string) - string`	Resolve to absolute path (lexical `.` / `//` tidy; does **not** resolve `..` - use `Resolve`).	—
GetRelativePath	`GetRelativePath(from:string, to:string) - string`	Relative path that reaches `to` when walked from directory `from` (using `../` to climb). Both are resolved to absolute canonical form first. `"."` when equal. Pure lexical; `nil` if `getcwd` fails.	—
GetRoot	`GetRoot(path:string) - string`	Returns `"/"` for a POSIX-absolute path, `""` otherwise	—

Function	Signature	Description	JIT
		(Windows drive roots are not recognized).	
HasExtension	`HasExtension(path:string) - bool`	`true` if path has an extension.	—
IsAbsolute	`IsAbsolute(path:string) - bool`	`true` for a POSIX root (`/`), a Windows drive root (`C:\` / `C:/`), a leading backslash, or a UNC path (`\\server`). Pure lexical test - no filesystem access.	—
Normalize	`Normalize(path:string) - string`	Convert `\` to `/`, collapse `//` runs and drop `./` segments. Does **not** resolve `..` or touch the filesystem.	—
Resolve	`Resolve(path:string) - string`	Absolute path with `.` and `..` resolved (relative inputs anchored to the current directory). Unlike `Normalize` / `GetFullPath` this collapses `..`, giving a canonical form for containment checks. Purely lexical - does not resolve symlinks or require the path to exist. `nil` if `getcwd` fails.	—
TempFile	`TempFile(prefix?:string) - string`	Create a unique empty file in the system temp dir (`mkstemp`) and return its path. Optional name prefix (default `"flaris"`) must not contain path separators. The file is not auto-deleted. `nil` on failure.	—

Regex

Namespace: **`Regex`**

Regular expression matching, search, replace, and split. The engine is a lightweight custom implementation supporting the syntax documented below. Patterns are matched against UTF-8 strings, but character-level operations are byte-based.

Functions

Function	Signature	Description	JIT
IsMatch	`IsMatch(input:string, pattern:string) - bool`	`true` if pattern matches anywhere in input.	✓
Match	`Match(input:string, pattern:string) - string`	Returns first match substring, or `nil`.	✓ owned[1]
Matches	`Matches(input:string, pattern:string) - array`	Returns all non-overlapping match substrings.	—
Capture	`Capture(input:string, pattern:string) - array`	Returns `[fullMatch, group1, group2, ...]` for the first match, or `nil` if no match. Element 0 is the whole match; subsequent elements are the capture groups in order of their opening `(`. A group that did not participate in the match (e.g. an unmatched optional group) is `nil`.	—

Function	Signature	Description	JIT
Replace	`Replace(input:string, pattern:string, replacement:string) - string`	Replace all matches with replacement string.	✓ owned[1]
ReplaceFn	`ReplaceFn(input:string, pattern:string, fn:fn) - string`	Replace each match with the result of `fn(match)`. The callback receives the matched substring and must return a string (`nil` deletes the match). If the callback raises, or returns a value that is neither a string nor `nil`, the exception propagates to the caller (wrap the call in `try` / `catch` to handle it).	—
Split	`Split(input:string, pattern:string) - array`	Split input at pattern boundaries. Always includes remainder as last element.	—

Supported syntax

Anchors

Syntax	Description
`^`	Match start of string
`$`	Match end of string

Wildcards and quantifiers

Syntax	Description
`.`	Any character except `\n` and `\r`
`*`	Zero or more (greedy)
`+`	One or more (greedy)
`?`	Zero or one
`{n}`	Exactly n repetitions
`{n,}`	At least n repetitions
`{n,m}`	Between n and m repetitions (inclusive)

Character classes

Syntax	Description
`[abc]`	Match any of `a`, `b`, `c`
`[^abc]`	Match anything not in set
`[a-z]`	Character range
`[a-zA-Z0-9]`	Multiple ranges
`[[:space:]]`	POSIX named class (see below)

POSIX named classes (for use inside `[...]`)

Class	Equivalent	Description
`[:alpha:]`	`[a-zA-Z]`	Letters
`[:digit:]`	`[0-9]`	Decimal digits
`[:alnum:]`	`[a-zA-Z0-9_]`	Letters, digits, underscore
`[:space:]`	`[ \t\n\r\f\v]`	Whitespace
`[:blank:]`	`[ \t]`	Space and tab only
`[:upper:]`	`[A-Z]`	Uppercase letters
`[:lower:]`	`[a-z]`	Lowercase letters
`[:punct:]`		Punctuation characters
`[:print:]`		Printable characters (including space)

Class	Equivalent	Description
`[:graph:]`		Printable characters (excluding space)
`[:cntrl:]`		Control characters
`[:xdigit:]`	`[0-9a-fA-F]`	Hexadecimal digits

Escape sequences

Syntax	Description
`\d`	Digit `[0-9]`
`\D`	Non-digit
`\w`	Word character `[a-zA-Z0-9_]`
`\W`	Non-word character
`\s`	Whitespace
`\S`	Non-whitespace
`\t`	Tab character
`\n`	Newline character
`\r`	Carriage return
`\b`	Word boundary (zero-width)
`\B`	Non-word boundary (zero-width)
`\.` `\*` etc.	Escaped literal character

Alternation

Syntax	Description
`a|b`	Match `a` or `b`. Left branch is tried first. Multiple alternatives supported: `a|b|c`.

Grouping and capture

Syntax	Description
`(...)`	Capturing group. Scopes alternation/quantifiers (e.g. `(ab)+`, `(cat|dog)s`) and records the matched substring for `Regex.Capture`. Groups are numbered 1, 2, … by the position of their opening `(`.
`(?:...)`	Non-capturing group. Groups for alternation/quantifiers without producing a capture.

Groups may be nested and quantified. With `Regex.Capture`, a quantified group reports the substring of its **last** iteration (e.g. `(ab)+` over `"abab"` captures `"ab"`).

Flags

Syntax	Description
`(?i)`	Case-insensitive matching. Must appear at the start of the pattern.

Examples

```
// Quantifiers
Regex.IsMatch("year 2024",          "[0-9]{4}")          // true
Regex.IsMatch("123",                "^[0-9]{2,4}$")      // true

// Alternation
Regex.IsMatch("my dog",             "cat|dog")           // true
Regex.Match("I have a fish",        "cat|dog|fish")      // "fish"
```

```
// Capture groups
Regex.Capture("2024-12-31", "(\\d+)-(\\d+)-(\\d+)")     // ["2024-12-31", "2024", "12", "31"]
Regex.Capture("name: John", "(\\w+): (\\w+)")          // ["name: John", "name", "John"]
Regex.Capture("foo", "(a)?(foo)")                        // ["foo", nil, "foo"]
Regex.Capture("nope", "(\\d+)")                          // nil

// POSIX named classes
Regex.Split("a  b   c",             "[[:space:]]+")      // ["a", "b", "c"]
Regex.Replace("hello!",             "[[:punct:]]+", "")  // "hello"

// Escape sequences
Regex.IsMatch("a\tb",              "a\\tb")           // true
Regex.Split("line1\nline2\nline3","\\n")             // ["line1", "line2", "line3"]

// Word boundaries
Regex.IsMatch("concatenate",        "\\bcat\\b")     // false
Regex.IsMatch("my cat",             "\\bcat\\b")     // true
Regex.Matches("x1 22 333",          "\\b\\d+\\b")  // ["1", "22", "333"]

// Case-insensitive
Regex.IsMatch("HELLO WORLD",        "(?i)hello")         // true
Regex.Match("HELLO",                "(?i)[a-z]+")        // "HELLO"
Regex.Replace("FOO bar foo",        "(?i)foo", "baz")    // "baz bar baz"
```

Limitations

- No backreferences (`\1`, `\2`, etc.)
- No lookahead or lookbehind
- No non-greedy quantifiers (`*?`, `+?`) - all repetitions are greedy
- No Unicode-aware matching - character classes and ranges operate on bytes
- `(?i)` must be at the start of the pattern; inline flags like `foo(?i)bar` are not supported
- Up to 31 capture groups per pattern; pattern length is limited to 511 bytes

Matching semantics

The engine finds the **leftmost** match. Among matches starting at the same position, alternation is leftmost-first (`a|b` prefers `a`) and quantifiers are greedy. For multi-alternative patterns, the match starting at the earliest position in the input wins regardless of branch order - e.g. `Regex.Match("dog cat", "cat|dog")` returns `"dog"`. Matching is linear in the length of the input (no catastrophic backtracking).

Scheduler

Namespace: **`Scheduler`**

Low-level fiber scheduling primitives.

Function	Signature	Description	JIT
HasReadyFibers	`HasReadyFibers() - bool`	`true` if any fiber is ready to run.	—
Schedule	`Schedule(f:fiber) - nil`	Enqueue fiber `f` for next scheduler tick.	—

Stream

Namespace: `Stream`

Unified I/O for files, network sockets, pipes, and serial ports. All functions operate on a `stream` value backed by a file descriptor.

TLS streams. `ConnectTls` and `AcceptTls` return an ordinary `stream` whose bytes happen to be encrypted, so every function below works over TLS and any Stream-based library runs unchanged — there is no separate TLS API to port to. TLS comes from the OS stack (macOS Secure Transport, Linux OpenSSL via `dlopen`, Windows SChannel) and the system trust store; certificates and hostnames are verified by default (TLS >= 1.2).

Four operations behave differently on a TLS stream, because a TLS session is a byte conversation with no file offset:

Operation	On a TLS stream
`Seek` / `Tell` / `Size` / `Truncate`	refused (`false` / `nil`) - there is no offset to move
`Accept`	`nil` - a TLS session is a connection, not a listener
`Flush`	no-op returning `true` - nothing is buffered locally
`Peek`	reads **decrypted** bytes, never ciphertext
`SendFile`	falls back to a read/encrypt/write loop; the kernel zero-copy path cannot encrypt

`PeerAddr`, `LocalAddr`, `SetNoDelay`, `SetKeepAlive` and `Shutdown` transparently reach the underlying socket.

Server credentials depend on the platform backend, because each OS TLS stack accepts a different form:

Platform	Backend	`{ certFile, keyFile }` (PEM)	`{ pkcs12File, password }`
Linux / POSIX	OpenSSL	✅ full chain via `certFile`	✅
macOS	Secure Transport	❌	✅
Windows	SChannel	❌	✅

PKCS#12 is therefore the portable choice; handing a PEM pair to macOS or Windows fails with the exact conversion command:

```
openssl pkcs12 -export -out server.p12 -inkey privkey.pem -in fullchain.pem
```

On macOS 15 and later the bundle is imported to process memory only, so a server running as a daemon (no `HOME`, no logged-in user) needs no keychain. On earlier macOS `SecPKCS12Import` still requires an accessible login keychain.

⚠️ **The handshake is blocking.** Like every synchronous call here it parks *all* fibers until it completes, so on a public listener keep `handshakeTimeoutMs` tight (the 10 s default is generous) — one client that connects and says nothing holds the VM for that long. Reads and writes *after* the handshake have the usual `*Async` twins and suspend only the calling fiber.

```
// Client - an ordinary stream, encrypted.
let s = Stream.ConnectTls("api.example.com", 443);
Stream.WriteString(s, "GET / HTTP/1.0\r\nHost: api.example.com\r\n\r\n");
Console.WriteLine(Stream.ReadLine(s));
Stream.Close(s);

// Server - the ordinary accept loop, one extra line.
let srv = Stream.Listen("tcp", 8443);
while (true) {
    Stream.WaitReadable(srv);
    let sock = Stream.Accept(srv);
    if (sock == nil) { continue; }
    // Takes ownership of `sock` - do not Close it.
    let tls = Stream.AcceptTls(sock, { pkcs12File: "server.p12", password: "secret" });
    if (tls == nil) { Console.Error(Stream.TlsLastError()); continue; }
    Stream.WriteString(tls, "HTTP/1.1 200 OK\r\nContent-Length: 2\r\n\r\nhi");
    Stream.Close(tls);
}
```

Function	Signature	Description	JIT
Accept	`Accept(s:stream) - stream`	Accept an incoming connection on a listening socket. Returns `nil` if no connection is pending.	—
Close	`Close(s:stream) - bool`	Close the stream. Flushes first.	—
Connect	`Connect(proto:string, host:string, port:int) - stream`	Open socket. `proto`: `"tcp"` or `"udp"`. `host`: hostname or IP (v4/v6). Returns `nil` on failure.	—
ConnectAsync	`ConnectAsync(proto:string, host:string, port:int) - fiber`	Non-blocking `Connect`: the DNS lookup and TCP handshake are offloaded to the io pool, so the fiber suspends instead of stalling the VM. Use with `await`; resolves to a socket stream, or `nil` on failure. Raises on a bad `proto`/`port`.	—
AcceptTls	`AcceptTls(s:stream, options:object) - stream`	Terminate TLS on a socket `Accept` already returned; the result is an ordinary stream. **Takes ownership of `s`** - it is detached either way, so never `Close` it afterwards. `options`: `pkcs12File` + `password` (all platforms) or `certFile` + `keyFile` (PEM, OpenSSL only); `handshakeTimeoutMs` (default 10000) bounds the handshake *and* every later read/write; `requireClientCert` demands a client certificate, validated against the system trust store. `nil` on failure - reason via `TlsLastError()`.	—
ConnectTlsAsync	`ConnectTlsAsync(host:string, port:int, options?:object) - fiber`	Async `ConnectTls`: DNS, the TCP handshake **and** the TLS handshake run on the I/O pool, so only the calling fiber suspends. `await` resolves to a stream, or `nil` on failure. Same `options` as `ConnectTls`. Prefer this in any client that must stay responsive - a TLS handshake is far more expensive than a plain connect.	—
ConnectTls	`ConnectTls(host:string, port:int, options?:object) - stream`	Open a TLS connection as an ordinary stream. `options`: `insecure` skips certificate/hostname verification (trusted hosts only), `timeoutMs` (default 30000) bounds the connect	—

Function	Signature	Description	JIT
		and each later read/write, `sni` overrides the name sent and verified against the certificate. `nil` on failure - reason via `TlsLastError()`.	
Copy	`Copy(src:stream, dst:stream, limit?:int) - bool`	Copy data from `src` to `dst`. Optional byte limit.	—
Flush	`Flush(s:stream) - bool`	Flush write buffer (`fsync` for files, `tcdrain` for serial, no-op for sockets).	—
IsOpen	`IsOpen(s:stream) - bool`	`true` if stream is still open.	—
LastError	`LastError() - int`	Why the calling fiber's most recent async operation (`ReadAsync`, `WriteAsync`, `WaitReadable`) resolved to `nil`. `0` = it succeeded, so a `nil` with `LastError() == 0` means a clean end-of-file, not a failure. Per-fiber, so concurrent fibers never overwrite each other's reason. Codes: `1` timeout, `2` stream closed/invalid, `3` connection reset, `4` read exceeded the maximum block size, `5` descriptor cannot be multiplexed, `6` other OS error, `7` cancelled by `Fiber.CancelIo`.	—
Listen	`Listen(proto:string, port:int, backlog?:int) - stream`	Create a listening socket. `proto`: `"tcp"` or `"udp"`. Binds a dual-stack IPv6 socket (`IPV6_V6ONLY=0`, so IPv4 clients connect too) and falls back to IPv4-only if v6 is unavailable. Sets `SO_REUSEADDR` and `SO_REUSEPORT`. `backlog` defaults to `128`.	—
LocalAddr	`LocalAddr(s:stream) - string`	Local endpoint of a socket as `"ip:port"` (`"[ip]:port"` for IPv6). `nil` for a non-socket or on error.	—
TlsLastError	`TlsLastError() - string|nil`	Why the last `ConnectTls` / `AcceptTls` failed (bad certificate, wrong password, handshake refused). Distinct from `LastError()`, which reports an integer io-error code: a TLS setup failure is a message with no errno. `nil` if none.	—
TlsPeerCert	`TlsPeerCert(s:stream) - object|nil`	The peer's leaf certificate as `{ sha256, subject }` (`sha256` = lowercase hex fingerprint of the DER cert). Use for pinning / trust-on-first-use. `nil` for a non-TLS stream or when unavailable.	—
TlsServerAvailable	`TlsServerAvailable() - bool`	`true` if this build can terminate TLS (`AcceptTls`). Narrower than having TLS at all: an old or stripped `libssl` may support clients but not servers.	—
Open	`Open(path:string, mode:string) - stream`	Open file stream. Mode: `"r"` (read-only), `"w"` (write/create/truncate), or `"rw"` (read-write/create). Returns `nil` on failure.	—
OpenSerial	`OpenSerial(port:string, baud:int) - stream`	Open serial port in raw mode. Valid baud rates: `9600`, `19200`, `38400`, `57600`, `115200`.	—
Peek	`Peek(s:stream) - int`	Return the next byte (0–255) without consuming it. Sockets use `MSG_PEEK`; files read one byte and rewind. Returns `nil` at EOF, on error, or for a pipe/serial stream (which have no non-destructive read).	—
PeerAddr	`PeerAddr(s:stream) - string`	Remote endpoint of a connected socket as `"ip:port"` (`"[ip]:port"` for IPv6). `nil` for a non-socket or on error.	—
Pipe	`Pipe() - array`	Create a pipe. Returns `[readStream, writeStream]`.	—
ReadAll	`ReadAll(s:stream, limit?:int, timeout?:int) - block`	Read until EOF or `limit` bytes and return a `block`. On a file a short read ends the read; on a socket/pipe it reads until the peer closes (EOF) or, for a socket, the `timeout` fires (seconds, default 3) — bytes already read are returned, not discarded. Returns `nil` only on a hard I/O error.	—

Function	Signature	Description	JIT
ReadAsync	`ReadAsync(s:stream, count?:int, timeout?:int, cb?:fn) - fiber`	Async read. `count > 0` completes once exactly `count` bytes have arrived; `count` omitted or `0` reads until EOF. `await` resolves to a `block`, or `nil`. A block can be shorter than `count` when EOF arrives first **or when the deadline fires after some bytes have arrived** — a timeout hands over what it already has rather than discarding it, so always check `len()`, and call `Stream.LastError()` (`1` = timed out) to tell a short block from a complete one. `nil` means nothing was received at all, or a hard error. `timeout` in ms; `<= 0` (default) = no deadline. With `cb`, `cb(block)` is invoked on completion and the awaited result is `nil`.	—
ReadByte	`ReadByte(s:stream) - int`	Read one byte as integer.	—
ReadBytes	`ReadBytes(s:stream, count:int) - block`	Read exactly `count` bytes into block.	—
ReadLine	`ReadLine(s:stream) - string`	Read until `\n`, stripping a preceding `\r` (so `\r\n` and `\n` both work). A lone `\r` is kept as data — no speculative read-ahead, so no byte is lost on a socket/pipe. Returns `""` at EOF.	—
ReadString	`ReadString(s:stream, count:int) - string`	Read `count` bytes as string.	—
Seek	`Seek(s:stream, pos:int, whence?:int) - bool`	Seek a file stream. `whence`: `0` from start (default), `1` from the current position, `2` from the end (`pos` may be negative). Returns `true` on success.	—
SendFile	`SendFile(s:stream, path:string, offset?:int, count?:int) - int`	Zero-copy file-to-socket transfer. Uses `sendfile` on Linux and macOS/BSD; falls back to read/write loop on other targets (e.g. OpenWrt). Returns bytes sent.	—
SetKeepAlive	`SetKeepAlive(s:stream, on:bool) - bool`	Enable/disable TCP keep-alive probes (`SO_KEEPALIVE`) on a socket. Returns `true` if set.	—
SetNoDelay	`SetNoDelay(s:stream, on:bool) - bool`	Enable/disable `TCP_NODELAY` (disable Nagle) so small writes go out immediately. Returns `true` if set.	—
SetTimeout	`SetTimeout(s:stream, readMs:int, writeMs?:int) - bool`	Set read and write timeouts in milliseconds on a socket stream. If `writeMs` is omitted, both directions use `readMs`.	—
Shutdown	`Shutdown(s:stream, how:string) - bool`	Half-close a socket. `how`: `"r"` stops reads, `"w"` stops writes (sends EOF to the peer while the read side stays open), anything else shuts both. Returns `true` on success.	—
Size	`Size(s:stream) - int`	Byte length of the underlying file (`fstat`). `nil` for a socket/pipe or on error.	—
Stderr	`Stderr() - stream`	A `dup()` of the process stderr as a writable stream. Closing it (or its GC) leaves the real stderr open.	—
Stdin	`Stdin() - stream`	A `dup()` of the process stdin as a readable stream. Closing it leaves the real stdin open.	—
Stdout	`Stdout() - stream`	A `dup()` of the process stdout as a writable stream. Closing it leaves the real stdout open.	—
Tell	`Tell(s:stream) - int`	Return current byte offset.	—
Truncate	`Truncate(s:stream, size:int) - bool`	Grow or shrink the file to exactly `size` bytes (`ftruncate`). Returns `true` on success.	—
WaitReadable	`WaitReadable(s:stream, timeoutMs?:int) - bool`	Park the calling fiber until `s` (typically a listen socket from `Listen`) becomes readable. Resumes with `true` on readiness, `nil` on timeout. `timeoutMs <= 0` (default `-1`) waits without a deadline. Suspends without `await` — callable from a plain function, so a server loop is `while`	—

Function	Signature	Description	JIT
		`(Stream.WaitReadable(srv, -1)) { let c = Stream.Accept(srv); ... }`.	
WriteAll	`WriteAll(s:stream, data:string|block) - bool`	Write all bytes, retrying on partial writes.	—
WriteAsync	`WriteAsync(s:stream, data:string|block, timeout?:int) - fiber`	Async write of all of `data`. `await` resolves to the bytes written (int), or `nil` on timeout/error. `timeout` in ms; `<= 0` (default) = no deadline. `data` is retained while the write is in flight. Use `Stream.LastError()` to tell a timeout from an I/O error.	—
WriteByte	`WriteByte(s:stream, value:int) - bool`	Write single byte. Returns `true` on success, `false` on a write error.	—
WriteBytes	`WriteBytes(s:stream, buf:block, count:int) - int`	Write `count` bytes from block; returns bytes written.	—
WriteString	`WriteString(s:stream, s:string) - bool`	Write string bytes.	—
ReadU8	`ReadU8(s:stream) - int`	Read one unsigned byte (0–255). Returns `nil` on EOF.	—
ReadU16	`ReadU16(s:stream, bigEndian?:bool) - int`	Read 2 bytes as unsigned 16-bit integer. Default little-endian.	—
ReadU32	`ReadU32(s:stream, bigEndian?:bool) - int`	Read 4 bytes as unsigned 32-bit integer. Default little-endian.	—
ReadU64	`ReadU64(s:stream, bigEndian?:bool) - int`	Read 8 bytes as 64-bit integer. Default little-endian.	—
WriteU8	`WriteU8(s:stream, value:int) - bool`	Write one byte. Returns `true` on success.	—
WriteU16	`WriteU16(s:stream, value:int, bigEndian?:bool) - bool`	Write 2 bytes. Default little-endian. Returns `true` on success.	—
WriteU32	`WriteU32(s:stream, value:int, bigEndian?:bool) - bool`	Write 4 bytes. Default little-endian. Returns `true` on success.	—
WriteU64	`WriteU64(s:stream, value:int, bigEndian?:bool) - bool`	Write 8 bytes. Default little-endian. Returns `true` on success.	—
ReadI8	`ReadI8(s:stream) - int`	Read one signed byte (–128…127). `nil` on EOF.	—
ReadI16	`ReadI16(s:stream, bigEndian?:bool) - int`	Read a signed 16-bit integer. Default little-endian. `nil` on EOF.	—
ReadI32	`ReadI32(s:stream, bigEndian?:bool) - int`	Read a signed 32-bit integer. Default little-endian. `nil` on EOF.	—
ReadI64	`ReadI64(s:stream, bigEndian?:bool) - int`	Read a signed 64-bit integer. Default little-endian. `nil` on EOF.	—
ReadF32	`ReadF32(s:stream, bigEndian?:bool) - float`	Read a 32-bit IEEE-754 float. Default little-endian. `nil` on EOF.	—
ReadF64	`ReadF64(s:stream, bigEndian?:bool) - float`	Read a 64-bit IEEE-754 double. Default little-endian. `nil` on EOF.	—
WriteF32	`WriteF32(s:stream, value:float, bigEndian?:bool) - bool`	Write a 32-bit IEEE-754 float. Default little-endian.	—
WriteF64	`WriteF64(s:stream, value:float, bigEndian?:bool) - bool`	Write a 64-bit IEEE-754 double. Default little-endian.	—
ReadVarint	`ReadVarint(s:stream) - int`	Decode an unsigned LEB128 varint (1–10 bytes). `nil` on EOF/truncation or an over-long encoding.	—
WriteVarint	`WriteVarint(s:stream, value:int) - bool`	Encode `value` as an unsigned LEB128 varint. Returns `true` on success.	—

Stream instance method syntax

All `Stream` functions where the stream is the first argument can be called directly on a `stream` value:

```
let s = Stream.Open("data.bin", "r");
s.ReadLine()          // same as Stream.ReadLine(s)
s.ReadByte()          // same as Stream.ReadByte(s)
s.WriteByte(0x0A)     // same as Stream.WriteByte(s, 0x0A)
s.Seek(0)             // same as Stream.Seek(s, 0)
s.Tell()              // same as Stream.Tell(s)
s.IsOpen()            // same as Stream.IsOpen(s)
s.Flush()             // same as Stream.Flush(s)
s.Close()             // same as Stream.Close(s)
```

Factory functions (`Open`, `Connect`, `Listen`, `Pipe`, `OpenSerial`) are not available as instance methods. Both forms compile to identical bytecode when the variable is typed (`: stream` or inferred from a builtin return). Untyped variables fall back to a runtime dispatch.

String

Namespace: **`String`**

UTF-8 string operations. `Length` returns **byte count**; use `Utf8Length` for codepoint count.

All `String` functions can also be called directly on a string value:

```
let s = "Hello, World!";
s.Length()             // 13    - same as String.Length(s)
s.ToLower()            // "hello, world!"
s.Contains("World")    // true
s.Substr(7, 5)         // "World"
s.Replace("o", "0")    // "Hell0, W0rld!"
s.Split(",")           // ["Hello", " World!"]
"  hi  ".Trim()        // "hi"
```

Both forms compile to identical bytecode when the variable is typed (`: string` or inferred from a builtin return). Untyped variables fall back to a runtime dispatch.

Building strings in a loop. `s += piece` is linear: when `s` is the only reference to its value, the piece is appended into the existing buffer (which grows geometrically) instead of copying the whole accumulated string each time. Building 80,000 pieces costs about 2 ms.

The optimization is invisible to program semantics - strings remain values. As soon as the value is shared, the append falls back to producing a fresh string, so an alias never observes a later mutation:

```
let s = "hello";
let t = s;      // t shares the value
```

```
s += " world";  // s becomes a new string; t is still "hello"
```

`String.Join(parts, sep)` is still the better choice when the pieces are already in an array, since it sizes the result once.

A `nil` argument in a string position never crashes and follows one policy: transforms and formatters return `nil`, predicates return `false`, index searches return `-1`, `Count` returns `0`; `Length` and `CharAt` treat `nil` as the empty string.

Function	Signature	Description	JIT
CharAt	`CharAt(s:string, pos:int) - char`	Character at byte position `pos` as a `char` immediate. Returns `'\0'` if `pos` is out of range. Zero allocation - prefer over `Substr(s, pos, 1)` in tight loops.	✓
Contains	`Contains(s:string, sub:string) - bool`	`true` if `sub` appears anywhere in `s`.	✓
Count	`Count(s:string, sub:string) - int`	Returns the number of non-overlapping occurrences of `sub` in `s`. Returns `0` if `sub` is empty.	✓
Create	`Create(n:int, fill?:int|char) - string`	Create a mutable string of length `n` filled with `fill` (default space). Returns `""` for `n ≤ 0`, `nil` for `n` exceeding the string size limit or `fill` outside 1–255 (a NUL fill would embed `'\0'` bytes and corrupt length bookkeeping). Intended for use with JIT functions that write characters in-place via `s[i] = ch`.	✓ owned[1]
Empty	`Empty - string`	Property: the empty string constant `""`. Not a function.	—
EndsWith	`EndsWith(s:string, suffix:string) - bool`	`true` if `s` ends with `suffix`.	✓
EqualsIgnoreCase	`EqualsIgnoreCase(a:string, b:string) - bool`	Case-insensitive equality. ASCII only (`A–Z` folds to `a–z`) and locale-independent, matching `ToLower` / `ToUpper`; non-ASCII bytes compare exactly, so `"Å"` and `"å"` are **not** equal.	✓
Format	`Format(fmt:string, ...args) - string`	C#-style positional formatting: `{0}`, `{1,width}` (negative width = left-align), `{0:X4}` / `{0:x}` hex, `{0:D3}` zero-padded decimal, `{0:F2}` or `{0:.2f}` fixed-point (ints included: `{0:F2}` of `5` is `"5.00"`). `{{` and `}}` emit literal braces. The template is used **literally** - a source literal's escapes are already expanded by the compiler, so `fmt` is never unescaped again and a `\t` in e.g. a Windows path survives. `:X` / `:x` / `:D` require an int and yield `nil` otherwise; other specs fall back to the value's default rendering. Returns `nil` on a malformed template. At least 1 arg required.	—
FormatArray	`FormatArray(fmt:string, values:array) - string`	Like `Format`, but placeholder indices `{0}`, `{1}`, … refer to elements of `values`. Identical grammar and rendering - both share one engine. Returns `nil` on a malformed template or out-of-range index.	✓ owned[1]
IndexOf	`IndexOf(s:string, sub:string) - int`	Byte index of first occurrence of `sub`, or `-1`.	✓
IndexOfAnyFrom	`IndexOfAnyFrom(s:string, charset:string, start:int) - int`	Byte index of the first character in `s` (starting from `start`) that appears anywhere in `charset`, or `-1`. Uses a single `strcspn` scan - efficient for finding the first of several possible delimiter characters.	✓

Function	Signature	Description	JIT
IndexOfFrom	`IndexOfFrom(s:string, sub:string, start:int) - int`	Byte index of first occurrence of `sub` at or after byte offset `start`, or `-1`. Avoids allocating a substring; use instead of `IndexOf(Substr(s, start))` in parsing loops.	✓
LastIndexOf	`LastIndexOf(s:string, sub:string) - int`	Byte index of the **last** occurrence of `sub`, or `-1`. An empty `sub` returns `Length(s)` (as `IndexOf` returns `0`).	✓
LastIndexOfAny	`LastIndexOfAny(s:string, charset:string) - int`	Byte index of the last character of `s` that appears anywhere in `charset`, or `-1`. Use to split on the last of several delimiters, e.g. `LastIndexOfAny(path, "/\\")`.	✓
Insert	`Insert(s:string, pos:int, sub:string) - string`	Insert `sub` at byte position `pos`.	✓ owned[1]
IsAlnum	`IsAlnum(s:string) - bool`	`true` if all characters are alphanumeric. `false` for `""`.	✓
IsAlpha	`IsAlpha(s:string) - bool`	`true` if all characters are alphabetic. `false` for `""`.	✓
IsUpper	`IsUpper(s:string) - bool`	`true` if all alphabetic characters are uppercase and at least one alphabetic character is present, so `"A1"` is `true` but `"123"` and `""` are `false`.	✓
IsLower	`IsLower(s:string) - bool`	`true` if all alphabetic characters are lowercase and at least one alphabetic character is present, so `"a1"` is `true` but `"123"` and `""` are `false`.	✓
IsNumeric	`IsNumeric(s:string) - bool`	`true` if all characters are numeric digits. `false` for `""`.	✓
IsWhitespace	`IsWhitespace(s:string) - bool`	`true` if all characters are whitespace. `false` for `""`.	✓
Join	`Join(arr:array, sep:str|char) - string`	Concatenate array elements with separator.	✓ owned[1]
JsonPretty	`JsonPretty(json:string) - string`	Pretty-print a JSON string.	✓ owned[1]
Left	`Left(s:string, n:int) - string`	Return first `n` bytes.	✓ owned[1]
Length	`Length(s:string) - int`	Byte length of string.	✓
Levenshtein	`Levenshtein(a:string, b:string) - int`	Edit distance (insert/delete/substitute) between the strings. Byte-based (equals character distance for ASCII). O(n·m) in C - use for "did you mean", fuzzy matching, dedup. Raises if `n*m` exceeds `MAX_LEVENSHTEIN_CELLS` (100M) rather than stalling the VM on huge inputs.	✓
NaturalCompare	`NaturalCompare(a:string, b:string) - int`	Three-way compare with digit runs compared numerically, so `"file2" < "file10"`. ASCII case-insensitive with a byte-wise tiebreak (total, deterministic order). Sort naturally: `Array.Sort(arr, fn(x,y) { return String.NaturalCompare(x,y) < 0; })`.	✓
ProcessCallback	`ProcessCallback(fn:function, s:string, len:int, cb:function) - nil`	Processes a string with your worker function `fn(s, len)`, then calls `cb(s, result)` when finished. A **read-only** kernel (reads `s[i]`, returns an `int` / `float` — a hash, checksum, count, or scan) runs on another CPU core when simple enough; a kernel that **writes** to the string runs normally (inline) so the string stays correct. Returns immediately. See `Buffer.ProcessCallback`.	✓
ProcessEvent	`ProcessEvent(fn:function, s:string, len:int, eventId:int) - nil`	Like `ProcessCallback`, but signals event `eventId` with the result instead of calling a callback. Wait for several with `Event.WaitFor([ids])`.	✓
PadLeft	`PadLeft(s:string, width:int, fill?:str|char) - string`	Left-pad to `width` with `fill` (default space).	✓ owned[1]
PadRight	`PadRight(s:string, width:int, fill?:str|char) - string`	Right-pad to `width` with `fill` (default space).	✓ owned[1]

Function	Signature	Description	JIT
Repeat	`Repeat(s:string, n:int) - string`	Concatenate `s` with itself `n` times. Returns `""` for `n ≤ 0` and `nil` if the result would exceed the string size limit.	✓ owned[1]
Replace	`Replace(s:string, old:string, new:string, count?:int) - string`	Replace occurrences of `old` with `new`, left to right. Without `count` (or with a negative one) every occurrence is replaced; `0` replaces none. Matching is non-overlapping, so `Replace("aaa", "aa", "b")` is `"ba"`.	✓ owned[1]
Reverse	`Reverse(s:string) - string`	Reverse bytes of string.	✓ owned[1]
Right	`Right(s:string, n:int) - string`	Return last `n` bytes.	✓ owned[1]
Sanitize	`Sanitize(s:string) - string`	Keep only ASCII letters, digits and `_`; every other character (spaces, punctuation, non-ASCII) is removed. Useful for identifiers/filenames.	✓ owned[1]
Fields	`Fields(s:string) - array`	Split around **runs** of whitespace, discarding empty parts, so leading/trailing/repeated spaces are absorbed: `Fields("  a  b  ")` is `["a","b"]` where `Split(s, " ")` would yield empty parts. The usual way to tokenise a line.	—
Split	`Split(s:string, sep:str|char, limit?:int) - array`	Split by separator; returns array of strings. With a positive `limit` the result holds at most that many parts and the last keeps the unsplit remainder (`Split("a,b,c", ",", 2)` is `["a","b,c"]`); `limit ≤ 0` means no limit.	—
SplitLines	`SplitLines(s:string) - array`	Split by newlines; returns array of strings.	—
StartsWith	`StartsWith(s:string, prefix:string) - bool`	`true` if `s` starts with `prefix`.	✓
Substr	`Substr(s:string, start:int, len?:int) - string`	Byte substring starting at `start`, optional `len`.	✓ owned[1]
ToAscii	`ToAscii(s:string, replacement?:str|char|int) - string`	Strip/replace non-ASCII characters.	✓ owned[1]
ToLower	`ToLower(s:string) - string`	Lowercase ASCII characters. Non-ASCII is left untouched - use `Utf8ToLower` for Unicode.	✓ owned[1]
ToUpper	`ToUpper(s:string) - string`	Uppercase ASCII characters. Non-ASCII is left untouched - use `Utf8ToUpper` for Unicode.	✓ owned[1]
Trim	`Trim(s:string, cutset?:str|char) - string`	Remove leading and trailing whitespace, or every leading/trailing character present in `cutset` when given. An empty `cutset` trims nothing.	✓ owned[1]
TrimLeft	`TrimLeft(s:string, cutset?:str|char) - string`	Remove leading whitespace, or leading `cutset` characters.	✓ owned[1]
TrimRight	`TrimRight(s:string, cutset?:str|char) - string`	Remove trailing whitespace, or trailing `cutset` characters.	✓ owned[1]
Truncate	`Truncate(s:string, maxLen:int) - string`	Truncate to `maxLen` bytes.	✓ owned[1]

UTF-8 helpers:

Function	Signature	Description	JIT
Utf8ByteIndexOf	`Utf8ByteIndexOf(s:string, cpIndex:int) - int`	Byte offset of codepoint at index `cpIndex`.	✓
Utf8Concat	`Utf8Concat(...parts:str|char|int) - string`	Concatenate strings, chars, or codepoints. 1–5 args.	✓ owned[1]
Utf8CpAt	`Utf8CpAt(s:string, bytePos:int) - char`	Codepoint at byte position `bytePos`.	✓

Function	Signature	Description	JIT
Utf8CpNext	`Utf8CpNext(s:string, bytePos:int) – int`	Byte offset of next codepoint after `bytePos`.	✓
Utf8GetCharAt	`Utf8GetCharAt(s:string, cpIndex:int) – char`	Character at codepoint index `cpIndex`.	✓
Utf8Length	`Utf8Length(s:string) – int`	Number of Unicode codepoints.	✓
Utf8Substr	`Utf8Substr(s:string, cpStart:int, cpLen:int) – string`	Substring by codepoint indices.	✓ owned[1]
Utf8ToLower	`Utf8ToLower(s:string) – string`	Unicode-aware lowercase: `Utf8ToLower("ÅÄÖ")` is `"åäö"`, where the ASCII-only `ToLower` leaves it unchanged. Uses the same tables as `Char.ToLower`; uncased codepoints pass through. Returns `nil` on invalid UTF-8.	✓ owned[1]
Utf8ToUpper	`Utf8ToUpper(s:string) – string`	Unicode-aware uppercase: `Utf8ToUpper("åäö")` is `"ÅÄÖ"`. Returns `nil` on invalid UTF-8.	✓ owned[1]

Time

Namespace: `Time`

Unix timestamp manipulation (seconds since epoch). `timestamp` is always `int`. Wall-clock time (`Now`, `NowMillis`) is Unix time; monotonic time (`Millis`, `Ticks`) has an arbitrary epoch and is only meaningful as a delta. All broken-down fields and formatting are UTC. The richer calendar/timezone API (local zones, leap years, durations, relative "ago") lives in the `Datetime` library.

Function	Signature	Description	JIT
AddDays	`AddDays(t:int, n:number) – int`	Add `n` days to timestamp. Fractional values allowed; `nil` if the result would overflow.	✓
AddHours	`AddHours(t:int, n:number) – int`	Add `n` hours to timestamp. Fractional values allowed; `nil` if the result would overflow.	✓
AddMinutes	`AddMinutes(t:int, n:number) – int`	Add `n` minutes to timestamp. Fractional values allowed; `nil` if the result would overflow.	✓
AddSeconds	`AddSeconds(t:int, n:number) – int`	Add `n` seconds to timestamp. Fractional values allowed; `nil` if the result would overflow.	✓
AddWeeks	`AddWeeks(t:int, n:number) – int`	Add `n` weeks to timestamp. Fractional values allowed; `nil` if the result would overflow.	✓
Day	`Day(t:int) – int`	Day of month (1–31).	✓
DiffDays	`DiffDays(a:int, b:int) – int`	Whole days between two timestamps (`a – b`), truncated toward zero.	✓
Format	`Format(t:int, fmt?:string) – string`	Format timestamp as string (UTC). Named aliases: `"short"` (`YYYY-MM-DD`), `"long"` / `"log"` (`YYYY-MM-DD HH:MM:SS`), `"time"` (`HH:MM:SS`), `"iso"` (`YYYY-MM-DDTHH:MM:SSZ`). Default: `"long"`. Custom strftime patterns also accepted (capped at 128 chars); an empty pattern yields `""`. `nil` on a bad time or format failure.	✓ owned[1]
FromMillis	`FromMillis(ms:int) – int`	Convert epoch milliseconds to seconds (truncate). Inverse of `ToMillis`.	✓
Hour	`Hour(t:int) – int`	Hour (0–23).	✓
Millis	`Millis() – int`	Monotonic millisecond timestamp for timing/intervals. The epoch is arbitrary (not Unix time); use only for deltas. For wall-clock milliseconds use `NowMillis`.	—
Minute	`Minute(t:int) – int`	Minute (0–59).	✓

Function	Signature	Description	JIT
Month	`Month(t:int) - int`	Month (1–12).	✓
Now	`Now() - int`	Current Unix timestamp in seconds.	—
NowMillis	`NowMillis() - int`	Current Unix time in whole milliseconds (wall clock). `FromMillis(NowMillis()) == Now()`.	—
Parse	`Parse(s:string, fmt?:string) - int`	Parse timestamp string (UTC). Named aliases: `"ISO"` (default, `%Y-%m-%dT%H:%M:%S`), `"RFC"` (`%a, %d %b %Y %H:%M:%S`), `"DATE_ONLY"` (`%Y-%m-%d`). Custom strptime patterns also accepted. Returns `nil` on failure.	✓
Second	`Second(t:int) - int`	Second (0–59).	✓
Ticks	`Ticks() - int`	Monotonic nanosecond counter. The epoch is undefined; use only for deltas with `TicksToMs` / `TicksToUs`.	—
TicksToMs	`TicksToMs(ticks:int) - float`	Convert a nanosecond tick delta to milliseconds. Example: `Time.TicksToMs(Time.Ticks() - t0) - 1.234`	✓
TicksToUs	`TicksToUs(ticks:int) - float`	Convert a nanosecond tick delta to microseconds. Example: `Time.TicksToUs(Time.Ticks() - t0) - 1234.5`	✓
TimeZoneOffsetMins	`TimeZoneOffsetMins(t?:int) - int`	Local timezone offset in minutes from UTC, DST-aware. Uses the current time, or the given timestamp.	—
ToMillis	`ToMillis(seconds:int) - int`	Convert whole seconds to milliseconds. Inverse of `FromMillis`.	✓
Weekday	`Weekday(t:int) - int`	Day of week: 0=Sunday … 6=Saturday.	✓
Year	`Year(t:int) - int`	Four-digit year.	✓

Timers

Namespace: `Timers`

Fiber-based timer scheduling. Max 64 simultaneous timers. Timer resolution is approx 1-3ms.

Function	Signature	Description	JIT
Cancel	`Cancel(id:int|float) - bool`	Cancel a timer by its handle.	—
IsActive	`IsActive(id:int|float) - bool`	`true` if `id` refers to a live timer (a scheduled one-shot not yet fired, or a still-repeating interval).	—
SetAtTime	`SetAtTime(timestamp:int|float, func:fn ) - int|nil`	Fire `fn` once at the given **Unix timestamp** (seconds since epoch); a time already in the past fires as soon as possible. Returns a handle, or `nil` on a NaN timestamp or when the timer table (max 64) is full.	—
SetImmediate	`SetImmediate(func:fn ) - int|nil`	Fire `fn` on the next scheduler pump (a zero-delay one-shot), akin to Node's `setImmediate`. Returns a handle, or `nil` when the timer table is full.	—
SetInterval	`SetInterval(ms:int|float, func:fn ) - int|nil`	Fire `fn` every `ms` milliseconds. Returns a handle, or `nil` if `ms` is NaN or `<= 0`, or the timer table is full.	—
SetTimeout	`SetTimeout(ms:int|float, func:fn ) - int|nil`	Fire `fn` once after `ms` milliseconds (a negative delay fires as soon as possible). Returns a handle, or `nil` on a NaN delay or when the timer table is full.	—

VM

Namespace: `VM`

Virtual machine introspection and control.

Function	Signature	Description	JIT
Compile	`Compile(src:string, arity?:int) - function|nil`	Compile a STATEMENT body into a callable function. `src` becomes the body of a function taking `arity` parameters named `arg0..argN` (use `return` for the result). Max 10 KB; returns `nil` on compile failure.	—
CompactMemory	`CompactMemory()`	Run memory compaction pass.	—
CurrentAllocations	`CurrentAllocations() - int`	Current live allocation count.	—
Eval	`Eval(src:string, ...args) - any`	Compile and immediately evaluate `src`. Tried first as an EXPRESSION (`Eval("40 + 2")` → `42`); if that fails to compile it is retried as a statement body, where an explicit `return` provides the result (`nil` otherwise). Extra call arguments are bound to `arg0..argN`: `Eval("arg0 * arg1", 6, 7)` → `42`. Max 10 KB; returns `nil` on compile failure (the process is never terminated by a bad `src`). Each `Eval` compiles as its own unit: it cannot reference host globals by name, but a `global` declaration in it can rebind an existing host global (warns on stderr). Eval'd code runs with full VM authority - never pass it untrusted input.	—
Exit	`Exit(code:int) - nil`	Terminate VM with exit code.	—
GetFunctionInfo	`GetFunctionInfo(fn:callable) - object`	Return an object describing a function. Accepts `function`, `builtin`, `ffi`, and bound methods. See field table below.	—
GetModuleInfo	`GetModuleInfo(name:string) - object|nil`	Return an object describing a built-in module. `Members` array contains `{Name, Signature}` per function; `Constants` array contains `{Name, Type, Value}` per constant. Returns `nil` if the module name is not found.	—
GetStartTimeMs	`GetStartTimeMs() - int`	VM start time in milliseconds since epoch.	—
Import	`VM.Import(name:string, version:string, fingerprint?:string)`	Load or return cached module by name and version requirement. `version` uses the same syntax as `library()`: `"1.0"`, `">=1.2 <2.0"`, etc. Optional `fingerprint` is the whole-file SHA-256 (== `sha256sum`; a `sha256:` prefix is accepted) - if provided and mismatched, the VM halts regardless of `--no-verify`. `let m = VM.Import("jwt", "1.0");`. See version syntax table in Guide §6.	—
MemoryStats	`MemoryStats() - object`	Snapshot of allocator counters: `Current` (live objects), `Peak` (high-water live objects), `HeapBytes` (cumulative bytes requested from malloc/calloc/realloc), `BlockRegions` / `BlockBytes` (live block allocations and their size), `SlabCacheFree` (objects held in the free-list cache). A superset of `CurrentAllocations` / `PeakAllocations` for tooling and leak checks.	—
OnSignal	`OnSignal(signum:int, handler:fn) - bool`	Register a signal handler and return `true`. Only signals the VM installs an OS handler for are accepted: SIGHUP, SIGINT, SIGUSR1, SIGUSR2, SIGTERM (numbers are platform-specific). Any other signal - including the uncatchable SIGKILL/SIGSTOP, which could never be delivered - or a non-function handler returns `false`.	—
PatchFunction	`PatchFunction(original:fn, replacement:fn) - bool`	Replace function at runtime. Requires running unsafe-mode `--unsafe`. The replacement may be a Flaris-function, a builtin-function or a FFI-function. Pass `nil` as replacement to remove the patch. Caveats: call sites the compiler inlined (trivial single-`return` functions) and calls made from inside	—

Function	Signature	Description	JIT
		JIT-compiled functions bypass the patch; self-recursive calls inside the original body also keep calling the original.	
PeakAllocations	`PeakAllocations() - int`	Peak allocation count since VM start.	—
RaiseSignal	`RaiseSignal(signum:int) - bool`	Send signal `signum` to the current process (`raise`). A handler registered via `OnSignal` runs on the next signal pump; an uncatchable signal takes the OS default effect. Returns `false` on an out-of-range number.	—
SignalName	`SignalName(signum:int) - string|nil`	Canonical name (e.g. `"SIGINT"`) for a signal number, or `nil` if unknown on this platform.	—
SignalNumber	`SignalNumber(name:string) - int|nil`	Platform signal number for a name like `"SIGINT"` (the `"SIG"` prefix is optional), or `nil` if unknown. Use this instead of hardcoding numbers, which are platform-specific (`SIGUSR1` is 10 on Linux, 30 on macOS).	—

`GetFunctionInfo` - returned object fields

Field	Type	Description
`Kind`	string	`"function"` , `"builtin"` , `"ffi"` , or `"unknown"`
`Name`	string|nil	Function name. For builtins resolved as `"Module.Name"` . Nil if anonymous.
`Arity`	int	Number of required parameters.
`Optional`	int	Number of optional parameters (builtins only; 0 for scripted and FFI).
`Returns`	string	Return type as a human-readable string (e.g. `"string"` , `"int|nil"`).
`ReturnType`	int	Return type as a raw type-mask integer.
`ArgTypes`	array<int>	Per-argument type masks in declaration order.
`ArgTypeNames`	array<string>	Per-argument type names in declaration order.
`IsStatic`	bool	`true` if the function is a static method.
`IsAsync`	bool	`true` if declared `async` .
`IsMethod`	bool	`true` if the function is an instance method (thiscall).
`Owner`	string|nil	Class name for methods; nil for free functions.
`Signature`	string	Rendered signature, e.g. `"fn Module.Name(int, string?) - bool"` .

Bound methods are automatically unwrapped - passing a bound method returns info about the underlying function.

R9 - Debug Reference

Debug Opcodes

The compiler emits these opcodes when debug symbols are enabled (default; disabled by `--strip` flag):

Opcode	Arguments	Purpose	When emitted
`OP_DBG_LINE`	`u16 line`	Mark source line number	Before each statement
`OP_DBG_FUNC_NAME`	`u16 const_idx`	Mark function name	At function entry
`OP_DBG_FILE_NAME`	`u16 const_idx`	Mark source file	Once per compilation unit
`OP_DBG_BREAK`	`u16 code`	Breakpoint	At `breakpoint` statement

Bytecode size overhead with debug symbols: ~15%. **Execution overhead:** ~2–3%.

Bytecode Example

```
fn compute(x) { return x * 2; }
```

With debug symbols:

```
[0000] OP_DBG_FILE_NAME    "main.fls"
[0003] OP_DBG_FUNC_NAME    "compute"
[0006] OP_DBG_LINE         line=2
[0009] OP_GET_LOCAL        x
[0011] OP_IMM8             2
[0013] OP_MULTIPLY
[0014] OP_RETURN
```

With `--strip` (stripped):

```
[0000] OP_GET_LOCAL
[0002] OP_IMM8             2
[0004] OP_MULTIPLY
[0005] OP_RETURN
```

Quick Reference

CLI Flags for Debugging

Flag	Effect	When to use
(default)	Debug symbols ON	Development, testing
`--strip`	Debug symbols OFF	Production, embedded
`--verbose`	Verbose output	High-level debugging
`--trace`	Very verbose	Opcode-level tracing
`--time`	Timing report	Performance analysis
`--stats`	Statistics	Memory debugging
`--debug`	Stop at `Main`, open the prompt	Interactive debugging

Debug Module Quick Summary

Function	Purpose
`Debug.Stack()`	Dump operand stack to stdout
`Debug.Pool()`	Memory slab statistics (requires `--unsafe`)
`Debug.Value(v)`	Inspect value (type, ref-count, address) (requires `--unsafe`)
`Debug.Assert(a, b)`	Equality check - aborts the VM on failure
`Debug.AssertTrue(c)`	Truthiness check - aborts the VM on failure
`Debug.GuardAddress(p)`	Watch memory address - traps on access (requires `--unsafe`)
`Debug.StackPtr()`	Current stack depth as integer
`Debug.Refs(o)`	External reference count of a value (requires `--unsafe`)
`Debug.Here(msg?)`	Print location marker (file/line/function)

Breakpoint Statement

```
breakpoint 100;              // Unconditional – always triggers
if (error) breakpoint 200;   // Conditional
```

Codes are user-defined integers. Use a namespace convention (e.g. 1000 = startup, 2000 = after validation) to make them meaningful.

A `breakpoint` statement is not the only way in. `--debug` stops at the start of `Main` and opens the prompt there, so a session can begin without editing the source at all - set breakpoints by function or by line and continue:

```
$ flarisvm --debug app.fls
[Entry] Main (app.fls:40) [fiber 1]
(fdb) b inspectOrder
breakpoint 1 at app.fls:21 (inspectOrder)
(fdb) b Order.Describe
breakpoint 2 at app.fls:18 (Order.Describe)
(fdb) c

[Breakpoint 1] inspectOrder (app.fls:21) [fiber 1]
(fdb) n
[Step] inspectOrder (app.fls:22) [fiber 1]
(fdb)                        <- an empty line repeats the step
```

The stop waits for `Main` rather than the first line executed: the module prologue runs first and is what defines the program's classes and functions, so stopping before it finished would leave nothing to set a breakpoint on.

Reaching a breakpoint opens an interactive prompt that pauses the whole VM and lets you walk the call stack and inspect values:

```
[Breakpoint:42] inspectOrder (app.fls:26) [fiber 1]
(fdb) bt
-> #0  inspectOrder (app.fls:26)
   #1  handleRequest (app.fls:33)
   #2  Main (app.fls:39)
(fdb) locals
  [0] order            = (instance) Instance: Order, {1017, 249.5, espresso}
  [1] retries          = (int) 2
  [2] items            = (array) [10, 20, 30]
(fdb) p order.label
order.label = (string) espresso
(fdb) c
```

Command	Purpose
`s` / `n`	Step into / step over one source line
`finish`	Run until the selected frame returns
(empty line)	Repeat the last step
`b [<file>:]<line>`	Set a line breakpoint (bare line = the selected frame's file)

Command	Purpose
`b <function>`	Break at a function's first line; `Class.Method` works too
`delete [<id>]`	Delete one breakpoint, or all of them
`info`	List breakpoints with hit counts
`bt`	Backtrace, innermost frame as `#0`
`frame <n>`	Select frame `<n>`
`up` / `down [n]`	Move `n` frames outward / inward (default 1)
`locals`	Locals of the selected frame, named from debug info
`p <path>`	Print a value: `name`, `name.field`, `name[0]`, or a chain of those
`list`	Source lines around the selected frame
`dis [n]`	Bytecode around the instruction the selected frame is stopped at, `n` either side (default 6)
`dis ir [fn]`	JIT IR of the selected frame, or of a named function
`stack`	Operand stack contents
`fibers`	Every live fiber, its state, position and what it waits on
`c`	Continue execution
`q`	Quit (exit code 5)
`help`	Command list

Inspection is read-only: the prompt never allocates a value, changes a reference count or raises, so looking at a program cannot change how it behaves.

Editor Debugging (DAP)

`flarisvm --dap=<port>` serves the Debug Adapter Protocol, so an editor can drive the same debugger: breakpoints in the gutter, stepping, a variables pane and a call stack. The VSCode extension in `vscode-flaris/` contributes a `flaris` debug type that starts the VM and connects for you; a minimal `launch.json` is:

```
{
  "type": "flaris",
  "request": "launch",
  "name": "Debug current Flaris file",
  "program": "${file}",
  "cwd": "${workspaceFolder}"
}
```

The transport is a TCP socket on loopback, never stdio: the debugged program owns stdout, and a protocol sharing that stream would be corrupted by the first `Console.WriteLine`. The adapter binds `127.0.0.1` only, so a session is never reachable off the machine.

A DAP thread is a fiber. The mapping is exact - fibers have ids, their own call stacks and their own scheduling state - so the editor's thread picker becomes a fiber picker.

Supported requests: `initialize`, `launch`, `setBreakpoints`, `configurationDone`, `threads`, `stackTrace`, `scopes`, `variables`, `evaluate`, `disassemble`, `continue`, `next`, `stepIn`, `stepOut`, `pause`, `disconnect`.

`evaluate` resolves the same paths the prompt's `p` does (`order.items[0]`), not arbitrary expressions, and reports a failure rather than inventing a value. Conditional breakpoints and `setVariable` are not implemented, and the adapter says so during `initialize` so the editor does not offer them.

Pausing and editing breakpoints while running. Requests are normally served while the program is stopped, but `pause` and `setBreakpoints` also have to be noticed mid-run. The adapter checks its socket every 128 line events - often enough to feel immediate, rare enough that it is not paying a syscall per source line. A `pause` takes effect at the next line boundary, which is where every other stop lands too; there is no way to suspend mid-instruction. Requests that would read a paused state (`stackTrace`, `variables`, stepping) are refused while running rather than answered with stale frames.

Disassembly. The editor's Disassembly View is served from the same disassembler `dis` uses at the prompt. A bytecode VM has no machine address to report, but the client parses memory references and instruction addresses *numerically*, so a position is dressed as one: the frame level occupies the high byte and the bytecode offset the low 24 bits (`0x0000003A` is offset `0x3A` in frame 0). A non-numeric reference makes the view report "Disassembly not available". Only real instructions are returned - padding rows would need invented addresses, and the client indexes rows by address.

Open it by right-clicking a frame in the Call Stack and choosing **Open Disassembly View**. Its step buttons step by source line: instruction-granularity stepping is not offered, because the debugger's hook is on line events.

Because `--dap` implies `--debug`, the JIT is disabled for the session and the program stops at the start of `Main` before the first line runs.

Uncaught Exceptions

An exception that reaches the top of a fiber with no handler is reported with its own code and message, and the process exits with that code:

```
Unhandled exception (code 42): boom
Stack trace (depth: 2):
  at Main (app.fls) [ip=34, line=9]
  at level3 (app.fls) [ip=31, line=3]
```

Under `--debug` the prompt opens at the throw site instead, with every frame still standing, so the values that produced the failure can be inspected:

```
[Unhandled exception: code 42] level3 (app.fls:3) [fiber 1]
(fdb) locals
  [0] x                = (int) 11
  [1] doomed           = (int) 22
(fdb) up
-> #1  Main (app.fls:9)
```

The program cannot resume from there - `c` and `q` both exit, with the exception's code.

Inspecting Fibers

`fibers` lists every live fiber with its state, current position and what it is parked on. Scheduling is cooperative and single-threaded, so the whole program is frozen while the prompt is open and the listing is a consistent snapshot - which is what makes it useful for a program that has stopped making progress:

```
(fdb) fibers
-> #1   running     frames=2    Main (app.fls:18)
   #2   yielded     frames=1    producer (app.fls:3)
   #3   wait-fiber  frames=1    consumer (app.fls:9)  waiting on fiber 2
```

Stepping and line breakpoints are decided at `OP_DBG_LINE`, the opcode that marks each source line. Nothing is examined unless a step is pending or a breakpoint exists, so an ordinary run costs one predicted branch, and a `--strip` build never executes the opcode at all.

Notes:

- A step belongs to the fiber that issued it. Other fibers run between two line events of the stepped one, and their lines never satisfy the step.
- `--debug` disables the JIT. A JIT-compiled function runs natively and never executes a line event, so breakpoints and steps could not reach inside one; turning the JIT off is what makes stopping reliable rather than silent.
- The prompt opens when stdin is a terminal. Otherwise the breakpoint prints its location and execution continues, so a `breakpoint` left in committed code cannot hang a piped or CI run. Pass `--debug` to open the prompt anyway, which is also how a session is driven from a script.
- `--strip` removes variable names, not breakpoints: slots and their values are still listed, as `<unnamed>`.
- `bt` shows the frames that actually exist. A tail call reuses its caller's frame, so a function that ends in `return f(...)` does not appear above `f`.
- Names resolve against the selected frame's locals first, then module and global scope.

Real-World Debugging Examples

Breakpoint with Bytecode Context

`dis` at the prompt shows the instruction stream around the instruction the selected frame is stopped at, marked `>>`. `--verbose` prints the same window automatically when the breakpoint fires.

```
[Breakpoint:100] main (app.fls:1649) [fiber 1]
(fdb) dis
bytecode at ip=4957 in main:
   [4929] LESS_EQUAL
   [4930] CALL1_BUILTIN          mod-id:12 func-id:1 -> Console.WriteLine
   [4933] POP
>> [4957] BREAKPOINT             code=100
   [4960] LINE                   line=1651
```

Selecting an outer frame with `up` marks the call that frame is waiting inside, so `dis` always points at where that frame actually is.

The window always starts on a real instruction boundary. Because instructions are variable length, it is found by decoding forward from the start of the chunk rather than by stepping back a fixed number of bytes.

For a JIT-compiled function, `dis ir <fn>` shows the typed register IR it was compiled from. A function containing a `breakpoint` is never JIT-eligible, so name the kernel you want rather than expecting IR for the frame you stopped in:

```
(fdb) dis ir kernel
JIT IR for kernel:
  ── JIT IR  (regs=5  ir_bytes=54) ──
    0022  ARITH_IMM8     r3, MUL, r2, 3
    0027  ARITH_II       r1, ADD, r1, r3
    0031  LOOP           -40  -000c
    0034  RET            r1
```

Memory Guard Trigger

```
Debug.GuardAddress(critical_data);
run_entire_program(critical_data);
// Any code that touches this object will stop here:
// [Mem-guard] R/W-operation on memory address 0xC98CDC1D0 in function main, line 1491
```

Tracking Stack Leaks

```
let before = Debug.StackPtr();
risky_operation(data);
let after = Debug.StackPtr();
if (after != before) {
    Console.WriteLine("Stack leak: delta =", after - before);
    Debug.Stack();
}
```

Full Debug Session Pattern

```
fn investigate(data) {
    Debug.Here("start");
    let alloc_before = VM.CurrentAllocations();
    let stack_before = Debug.StackPtr();
    Debug.Pool();

    Debug.GuardAddress(data);
    breakpoint 100;

    let result = risky_operation(data);

    breakpoint 200;

    if (Debug.StackPtr() != stack_before) Console.WriteLine("STACK LEAK!");
```

```
    Debug.Value(result);
    Debug.Pool();
    Console.WriteLine("Alloc delta:", VM.CurrentAllocations() - alloc_before);
    Debug.Here("end");
}
```

For FFI plugin development, see `ffi.md`.

R10 - Writing Fast Flaris Code

This chapter translates the execution model from R7 into concrete coding patterns. Each recommendation is grounded in how many VM dispatch cycles a pattern consumes per iteration.

Dispatch count is the most direct proxy for loop speed.

Counting loops: use `iter` for known integer ranges

`iter` is the fastest loop construct in Flaris. A single fused instruction both increments the counter and checks the bound - one dispatch for what `while` needs three to do.

```
// Best — 2 dispatches/iteration overhead
iter (i from 0 to n - 1) {
    sum += arr[i];
}
```

Equivalent `while` costs at least 3 dispatches overhead (compare-jump + increment + loop-back). Use `iter` whenever the range is a fixed integer expression known at the loop head.

`iter` allocates no iterator object. `i` is a local slot that starts at `from` and runs while `i <= to`.

Both bounds are inclusive. `iter (i from 0 to 3)` runs four times, with `i` taking 0, 1, 2 and 3 - so walking a collection is `from 0 to len(x) - 1`, not `from 0 to len(x)`. A range whose start exceeds its end (`from 0 to -1`, which is what an empty collection produces) runs zero times.

Comparing locals to constants: `while` fuses to a single instruction

When the loop condition is `local < constant` (or `<=`, `>`, `>=`), the compiler fuses the compare and branch into a single instruction instead of the four-instruction sequence a generic condition would require. A literal bound up to ±127 encodes in 1 byte; larger bounds encode in 4 bytes - both still one dispatch.

```
// Fused — 1 dispatch for the entire condition
while (i < 1_000_000) { ... }   // large literal bound (4-byte encoding)
while (i < 100)       { ... }   // small literal bound (1-byte encoding)
```

```
// Also fused - local vs local
while (i < len) { ... }
```

All comparison operators (`<`, `<=`, `>`, `>=`, `==`, `!=`) are eligible. The bound must be a literal integer or a local variable resolved at compile time. If it is an expression (e.g. `arr.count + 1`), cache it into a local before the loop:

```
let limit = arr.count + 1;      // evaluated once
while (i < limit) { ... }       // fused, 1 dispatch
```

Incrementing counters: prefer `++` over `+= 1` over `= x + 1`

Three ways to increment a counter produce meaningfully different instruction sequences:

Form	Encoding	Dispatches
`i++`	2 bytes	1
`i += 1`	4 bytes	1
`i = i + 1`	6 bytes (padded)	3

`i = i + 1` is peephole-optimized in-place, but the padding bytes that fill the freed space still execute as individual dispatches.

`i++` is the most compact form - a 2-byte instruction with no arithmetic subcode. It is both the smallest encoding and the clearest signal to the compiler.

```
// Best - 2 bytes, 1 dispatch
while (i < n) {
    process(arr[i]);
    i++;
}

// Good - 4 bytes, 1 dispatch
while (i < n) {
    process(arr[i]);
    i += 1;
}

// Avoid - 6 bytes padded, 3 dispatches
while (i < n) {
    process(arr[i]);
    i = i + 1;
}
```

The same rule applies to `--` and any compound-assign: `n += x` is always cheaper than `n = n + x`.

Compound assignment beats explicit assignment for all arithmetic

`n += x`, `n -= x`, `n *= x` etc. compile to a compact fused instruction (zero stack, 1 dispatch). Writing `n = n + x` instead forces the optimizer to patch the result in-place, leaving padding bytes that still dispatch individually.

```
// Good — zero-stack, 1 dispatch
sum += arr[i];
n   *= factor;

// Slower — fused + 1–3 padding dispatches depending on operand sizes
sum = sum + arr[i];
n   = n * factor;
```

This matters most inside loops. The operand encoding determines how much padding remains after optimization.

Cache repeated property and array accesses into locals

Every `obj.prop` or `arr[expr]` inside a loop re-executes a hashmap lookup or bounds-checked index. Pull stable reads out of the loop body.

```
// Without caching — hashmap + bounds check every iteration
while (i < data.count) {
    process(data.items[i]);
    i++;
}

// With caching — local reads only inside the loop
let items = data.items;
let count = data.count;
while (i < count) {
    process(items[i]);
    i++;
}
```

This combines with the fused-compare rule: `while (i < count)` fuses to a single instruction because `count` is now a local.

For array element reads inside loops, the compiler uses a zero-stack fast path when both the array and the index are locals, avoiding the generic dispatch path entirely.

The same principle applies to method calls. Storing a method reference in a local before the loop avoids repeating the hash lookup on every iteration:

```
// Without caching — hash lookup on every call
while (i < count) {
```

```
    obj.process(data[i]);
    i++;
}

// With caching - direct call, no lookup
let fn = obj.process;
while (i < count) {
    fn(data[i]);
    i++;
}
```

Keep variables type-stable inside loops

The VM fast-paths for arithmetic and comparison check the type tag on every operation. A variable that starts as an integer and becomes a float mid-loop forces the slow path on every subsequent instruction.

```
// Good - tag-int fast path throughout
let sum: int = 0;
iter (i from 0 to n - 1) {
    sum += values[i];   // stays int the whole way
}

// Risky - if any values[i] is a float, sum flips type mid-loop
let sum = 0;
iter (i from 0 to n - 1) {
    sum += values[i];
}
```

Type annotations on locals (`let sum: int = 0`) are enforced by the analyzer and allow the compiler to take the fastest arithmetic path.

Type your function parameters for faster calls

Flaris supports two styles: quick untyped scripts and fully typed code. The style you choose affects call performance.

When a function is called, the VM validates each argument's type against the declared parameter type. For untyped parameters the check still runs - it just accepts anything. For performance-critical functions that are called many times, this overhead adds up.

```
// Untyped - runtime validates each argument on every call
fn fib(n) {
    if (n < 2) return n;
    return fib(n - 1) + fib(n - 2);
}
```

```
// Typed — compiler verifies types statically; runtime skips the check entirely
fn fib(n: int): int {
    if (n < 2) return n;
    return fib(n - 1) + fib(n - 2);
}
```

When the compiler can prove at the call site that every argument's type matches the declared parameter type, it emits `CALL_TYPED` instead of `CALL`. The VM fast path then skips argument validation completely - saving roughly 10 cycles per call.

Style	Validation	Bytecode emitted
Untyped params	At runtime, every call	`CALL`
Typed params + typed arguments	Skipped - done at compile time	`CALL_TYPED`

The gain is proportional to call frequency. A function called 100 million times saves ~300 ms on M-series hardware when typed. A function called once saves nothing measurable.

Guideline: write untyped for glue code, one-off scripts, and prototypes. Add type annotations to any function that sits on a hot call path.

Inline small functions to remove call overhead

A function marked `inline` (or a trivial expression-bodied function the compiler auto-inlines) has its body expanded into each call site, removing the call frame entirely. Arguments are evaluated once, in order, and bound to fresh temporaries; name collisions with caller locals, over-deep recursive inlining (past 8 levels), and arity mismatches fall back to a normal call, so inlining never changes behavior.

```
fn inline sq(x) { return x * x; }   // expands at each call site
fn dist2(a, b) { return sq(a) + sq(b); }
```

Inline aggressively only for small, hot helpers - a large inlined body is duplicated at every call site and can bloat the bytecode.

Tail calls run in constant stack space

When a `return` directly returns a call - `return f(x);` - the compiler emits a **tail call** that reuses the current frame instead of pushing a new one. A self-recursive tail call (`return self(...)`) is lowered even more tightly (`OP_TAIL_SELF` , reading the current frame's function directly). This lets recursion-as-iteration run in O(1) stack:

```
fn sum(n, acc) {
    if (n == 0) { return acc; }
    return sum(n - 1, acc + n);   // tail call — constant stack depth
}
```

Note: a self tail call always re-enters the original function body; it is not affected by a later `VM.PatchFunction` override of that name.

Strip debug symbols for production

By default, the compiler injects debug tracking instructions for line numbers and symbol names. These execute as individual dispatches inside every function, including loop bodies. With `--strip` the compiler omits them.

Mode	Extra dispatches per loop iteration
Default (debug on)	1–3 per statement
`--strip` (debug off)	0

For a 10 million-iteration loop with two body statements, debug symbols add roughly 20–30 ms on M-series hardware. Use `--strip` for any compiled production binary.

```
# Development — full debug symbols, optimizations on
flarisvm myapp.fls

# Production — strip debug symbols (optimizations are on by default)
flarisvm --strip myapp.fls

# Compile for distribution
flarisvm --compile myapp.fls myapp.flx --strip
```

Dispatch budget summary for common loop patterns

All figures measured with `--strip` on M-series hardware. "Overhead" is the number of VM dispatches per iteration that are not the loop body.

Pattern	Overhead dispatches	Notes
`iter (i from 0 to N) { ... }`	2	fused increment+check + loop-back
`while (i < N) { i++; }`	3	fused compare + compact increment + loop-back
`while (i < N) { i += 1; }`	3	fused compare + compound assign + loop-back
`while (i < N) { i = i + 1; }`	5	same + 2 padding dispatches
`while (i < N) { ... }` without `--strip`	+1–3	per body statement, debug tracking
`obj.method()` per iteration	-	hash lookup each call
`let fn = obj.method; fn()` per iteration	-	direct call; saves the lookup

The gap between the best and worst row above is 2.5×. On a 10 million-iteration loop that is the difference between 70 ms and 175 ms for a single counter pattern alone.

Worked example: arithmetic benchmark

```
// Baseline — explicit assignments, no fusion (slowest)
fn slow_sum(n) {
    let sum = 0;
    let i = 0;
    while (i < n) {
        sum = sum + i;   // padded, 3 dispatches
        i = i + 1;       // padded, 3 dispatches
```

```
    }
    return sum;
}

// Optimized - compound assign + iter (fastest)
fn fast_sum(n) {
    let sum = 0;
    iter (i from 0 to n) {
        sum += i;        // fused, 1 dispatch
    }
    return sum;
}
```

`fast_sum` inner loop (with `--strip`): 3 dispatches per iteration for 2 meaningful operations - fused increment+check, compound assign, loop-back.

R11 - JIT Compilation

On ARM64 and x86-64, Flaris includes a second execution tier: a function-level JIT compiler that translates eligible functions to native machine code. JIT-compiled functions run much faster than the bytecode interpreter for numeric workloads.

The JIT is **on by default** and gates the pipeline at both ends. `--jit-disable` turns it off at whichever end you pass it:

Step	Command	Effect
Compile	`flarisvm --compile app.fls app.flx`	Generates JIT IR and embeds it in the `.flx` alongside standard bytecode
Run	`flarisvm --exec app.flx`	Compiles the embedded JIT IR to native code on first function load

With `--jit-disable` at compile time, no JIT IR is written - the `.flx` is standard bytecode only. With `--jit-disable` at run time, any JIT IR in the file is loaded but not compiled; functions run on the bytecode interpreter.

The JIT backend is automatically selected at run time based on the CPU: ARM64 on Apple Silicon / Raspberry Pi / etc., and x86-64 on Linux/macOS/Windows PCs. On other platforms the bytecode interpreter is the only execution tier.

The JIT boundary - why it exists

The JIT compiler works on **unboxed, statically-typed values**. Inside a JIT-compiled function, integers are bare 64-bit integers, floats are bare 64-bit doubles, and strings, blocks, and typed arrays are raw pointers to their objects. There is no dynamic dispatch, and heap allocation is limited to a few tracked patterns.

What runs natively inside a JIT function:

Allowed in a JIT function	Constraint
Integer/float/bool/char arithmetic and comparisons	-

Allowed in a JIT function	Constraint
Typed-array and block element access (`a[i]`, `Buffer.*At`)	Array/block must arrive as a typed parameter or a tracked local
String comparison (`==`, `!=`, `<`, `<=`, `>`, `>=`)	Both operands strings (variables, literals, or string fields of `this`)
String concatenation `s + t`	Only as a local initializer, a return value, or a builtin-call argument
`Buffer.Create` / `Array.Create` and other allocating builtins	Only as a local initializer or a return value (the frame owns and releases the result)
Homogeneous `[int]` / `[float]` array literals with computed elements (`[x, y, x*2]`)	Only as a local initializer (allocated + filled as an owned typed array). Fully-constant literals are already const-folded
`this.<field>` reads (scalar unboxed; string/array/block/object/float borrowed) and stores (int/bool/char inline, `float` copy-on-write, object fields borrowed)	Declared fields of the method's own class
`<recv>.<field>` reads and stores, where `recv` is a **class-typed parameter** (`a: Vec2`) or a **`new`-pinned local**	The receiver's class is known at compile time; int/bool/char store inline, `float` fields store copy-on-write (no per-store allocation), object fields take a borrowed value
`<recv>.method(...)` on a class-typed parameter or `new`-pinned local	The method must be a non-overridden instance method that is itself JIT-eligible
`let p = new C()` for a **constructor-less** class	The frame owns and releases the instance; `p` must not be reassigned
Calls to other JIT-compiled script functions	Callee must be JIT-eligible and return a scalar (object returns stay at the interpreter boundary)
Calls to stdlib builtins marked in the `JIT` column of R8	See the column legend below
`param == nil` tests	Folds to a constant - a native call always has all arguments present and typed

What still requires the interpreter:

Not allowed in a JIT function	Why
Object literals `{a: 1}`; `[char]` / `[bool]` or heterogeneous array literals	Untracked heap allocation (homogeneous `[int]` / `[float]` literals in a declaration are lowered)
`new Foo(...)` with a constructor, or with arguments	Constructor dispatch — only argument-less `let p = new C()` on a constructor-less class is lowered
`obj.field` on an object of unknown class	Dynamic property lookup — resolved only for `this`, a class-typed parameter, or a `new`-pinned local
`try` / `catch`, `yield`, `await`, closures	Interpreter machinery
Global variable reads/writes	Globals live in the interpreter environment

A function that uses an unsupported construct is simply not JIT-compiled - it runs on the bytecode interpreter with identical semantics. `flarisvm --check file.fls` prints the per-function verdicts and the exact reason a function stays interpreted.

The `JIT` column in the R8 stdlib tables

Cell	Meaning
✓	Callable directly from JIT-compiled code (scalar result, arguments passed borrowed)
✓ owned[1]	Callable; the fresh object result must be consumed where the frame can track it: a local declaration's initializer, a return value, or a string-concatenation operand
✓ `--unsafe`	Callable, but the builtin itself requires unsafe mode (`--unsafe`) at runtime
—	Not callable natively - a call makes the surrounding function fall back to the interpreter

Callable builtins require every argument to be a JIT value (typed scalar, string, block, or typed array); optional trailing arguments may be omitted exactly as in interpreted code. The core builtins `len(x)`, `int(x)`, and `float(x)` are also JIT-lowered.

The practical rule remains: **assemble complex data structures in normal interpreter code; pass typed arrays, blocks, and scalars into JIT functions that do the heavy computation**.

Structuring code for JIT

The recommended split is:

```
// Interpreter layer: set up data, call the compute function, use results
fn process_signal(raw: array) : array {
    let n: int = len(raw);
    let buf: [float] = Array.Create(n, Type.Float);
    // copy raw values into typed float array
    for (let i: int = 0; i < n; i++) { buf[i] = float(raw[i]); }

    // hand off to JIT - all heavy work happens here
    normalize(buf, n);

    return buf;
}

// JIT layer: typed arrays in, typed scalars/arrays out - no allocation
fn normalize(a: [float], n: int) : int {
    let max: float = 0.0;
    let i: int = 0;
    while (i < n) {
        if (a[i] > max) { max = a[i]; }
        i++;
    }
    if (max == 0.0) { return 0; }
    i = 0;
    while (i < n) { a[i] = a[i] / max; i++; }
    return 1;
}
```

Key points:

- The interpreter function creates the `[float]` array and fills it - that allocation lives outside the compute-heavy path
- The compute function receives the array as a parameter and works on it in-place - no allocation, no boxing
- Multiple JIT functions can call each other directly without re-entering the interpreter (see JIT fast-call for callbacks)

Small objects can also flow through JIT code when their class is statically known - a class-typed parameter, or a local constructed in place. Field reads and writes and method calls on such a receiver are native; `float` fields are stored copy-on-write, so a tight update loop allocates nothing:

```
class Vec2 {
    let x: float = 0.0;
    let y: float = 0.0;
    fn len2() : float { return this.x * this.x + this.y * this.y; }
}

// Class-typed parameter: `v`'s class is verified once at entry, then
// `v.x`, `v.y` and `v.len2()` are resolved to direct slot access / calls.
fn dist2(v: Vec2) : float { return v.len2(); }

// `new`-pinned local: construct in place, fill fields, loop - all native.
fn sweep(n: int) : float {
    let p = new Vec2();          // constructor-less class, frame-owned
    p.x = 3.0;
    p.y = 4.0;
    let acc: float = 0.0;
    for (let i: int = 0; i < n; i++) { acc = acc + p.len2(); }
    return acc;
}
```

Eligibility (automatic)

There is no keyword to mark a function for JIT. Unless `--jit-disable` is given, **every eligible function is compiled automatically** - just write ordinary functions with typed parameters. A function is **JIT-eligible** when all of the following hold:

- All parameters have explicit type annotations
- The return type is explicitly annotated, **or** the analyzer can infer it: when every `return` expression provably has the same single concrete scalar type, the function is typed automatically (chains of unannotated functions calling each other resolve too). Mixed returns (`return 1;` / `return 2.5;`) keep the function dynamic.
- Functions with no value returns (procedures) are also eligible - they produce `nil` exactly like the interpreter
- Parameter and return types are: `int` , `float` , `bool` , `char` , `string` , or `block` ; typed arrays `[int]` , `[float]` , `[char]` , `[bool]` ; or a **user-defined class** (`a: Vec2`) - the receiver's class is verified at the native entry, so field and method access on it is resolved statically. Optional parameters (`k?: int`) are fine - a native call always has every argument present and typed, and other call shapes take the interpreter path
- The function body contains **none** of: `try` / `catch` , `yield` , `await` , member-access on an object of unknown class (`obj.field` - resolved for `this` , a class-typed parameter, or a `new` -pinned local), object literals (`{k: v}`), global variable access, or calls to non-eligible script functions. `new` is allowed only as `let p = new C()` for a constructor-less class, and array literals only as a `let a = [..]` initializer of a homogeneous `[int]` / `[float]` array

- String operations are native: comparisons (`==`, `!=`, ordering) anywhere, concatenation (`+`) as a local initializer, return value, or builtin-call argument, and string-literal returns
- `foreach` over a typed-array or `string` parameter/local is allowed; `foreach (v, i in arr)` with an index variable is also supported

Allowed exceptions:

- Calls to every stdlib builtin marked ✓ in the R8 `JIT` column run at native speed; ✓ owned[1] builtins (`Buffer.Create`, `Array.Create`, `String.Replace`, …) additionally require their result to be consumed as a local initializer, a return value, or a concat operand
- Class methods are compiled too: `this.<field>` access (scalar/borrowed reads and int/bool/char/`float`/object stores), `this.method()`, static and `super.method()` calls all stay native when the callee is eligible and returns a scalar (`int`, `float`, `bool`, or `char`)
- **Objects flow through JIT code** when their class is statically known - a class-typed parameter (`a: Vec2`), or a local pinned by `let p = new C()` (constructor-less class, not reassigned). On such a receiver, `p.field` reads and stores (int/bool/char inline, `float` copy-on-write, object fields borrowed) and `p.method(...)` calls are all native. This lets a self-contained numeric kernel construct a small object, fill its fields, and loop over them without leaving native code
- Calls to other eligible functions become direct JIT-to-JIT calls when the callee returns a scalar; object-returning script callees keep the caller on the interpreter

Fibers and long JIT loops: a JIT-compiled function runs as one native call and cannot be preempted mid-loop. Every loop back-edge decrements a scheduling counter; on expiry (every 10,000 iterations) the JIT pumps pending IO (timers, sockets, completions) and flags the fiber so it yields to other fibers as soon as the JIT call returns. Async work therefore keeps flowing during long numeric kernels, but a single very long JIT call still occupies the CPU until it finishes.

```
fn sum_to(n: int) : int {
    let s: int = 0;
    for (let i: int = 1; i <= n; i++) { s = s + i; }
    return s;
}
```

A function that fails the eligibility check simply runs on the bytecode interpreter - no error, no warning. Use `--verbose` to see which functions qualified:

```
flarisvm --verbose myfile.fls
# [JIT] signature-eligible: sum_to (line 20)
# [JIT] signature-eligible: normalize (line 35)
```

If a function you expected to be JIT-compiled does not appear, look for one of the disqualifiers above - untyped parameters and member access on arbitrary objects (`obj.field`) are the most common surprises; `--check` prints the exact reason per function.

Checking eligibility with `--check`

`flarisvm --check <file.fls>` compiles the file without running it and prints a per-function eligibility report - the quickest way to see what runs native and, for anything that doesn't, exactly why:

```
flarisvm --check mymath.fls
```

```
JIT eligibility  (✓ native   · interpreted):

  ✓ sum_to                      (line 12)
  ✓ dot                         (line 20)
  · scale_all                   (line 28) - expression has no JIT lowering
  · label                       (line 34) - a parameter is untyped, or not a JIT type
(int/float/bool/char/string/block, or a typed array)
  · Point.dist                  (line 51) - member access (only this.<field> is JIT-lowered)

  2 of 5 functions are JIT-eligible.
```

The reason on each `·` line names the first disqualifier and its line, so you can fix or restructure without guessing. It also works on library files, e.g. `flarisvm --check libs/Signals.fls --libs='./libs'`, to see which functions of a module are numeric kernels.

JIT IR in compiled .flx files

The JIT is on by default at both ends. When you compile a `.flx`, the JIT code for every eligible function is embedded alongside the regular bytecode; without `--jit` at compile time, none is written:

```
flarisvm --compile myfile.fls myfile.flx   # compile - embeds JIT code
flarisvm --exec myfile.flx                 # execute - runs functions natively
flarisvm    --exec myfile.flx              # execute - runs bytecode only (JIT code present but
inactive)
```

The embedded JIT code is architecture-independent - the same `.flx` runs natively on any platform with a JIT backend, and stays fully portable: it is only activated when `--jit` is passed at run time; otherwise functions run on the bytecode interpreter. (Self-contained binaries built with `--embed` always run with the JIT active.)

Supported operations

Category	What is JIT-compiled
Int arithmetic	`+` `-` `*` `/` `%` on `int` locals
Float arithmetic	`+` `-` `*` `/` on `float` locals; unary negation
Bitwise / shifts	`&` `|` `^` `<<` `>>` on `int`
Comparisons	`<` `<=` `>` `>=` `==` `!=` (int and float)
Logical	&& and || with short-circuit evaluation
Bool operations	`bool` params/return; `true` / `false` literals; bool in `if`
Control flow	`if` / `else`, `while`, `for`, `iter`, `foreach`, `return`
Switch	`switch` on `int`, `bool`, or `char` expression with literal case values; `break` and fallthrough supported
Counter ops	`i++` `i--` `i += n` `i -= n` (also `*=`, `/=`, `%=`, `&=`, `|=`, `^=`, `<<=`, `>>=`)
Type conversions	`float(n)` widens int; `int(x)` truncates float
Int array reads	`a[i]` and `a[constant]` where `a: [int]`
Int array writes	`a[i] = expr` and `a[constant] = expr` where `a: [int]`
Float array reads	`a[i]` and `a[constant]` where `a: [float]` (returns unboxed double)

Category	What is JIT-compiled
Float array writes	`a[i] = expr` and `a[constant] = expr` where `a: [float]`
Char array reads	`a[i]` and `a[constant]` where `a: [char]` (returns unboxed codepoint)
Char array writes	`a[i] = expr` and `a[constant] = expr` where `a: [char]`
Bool array reads	`a[i]` and `a[constant]` where `a: [bool]` (returns unboxed 0/1)
Bool array writes	`a[i] = expr` and `a[constant] = expr` where `a: [bool]`
Array returns	Functions may return a typed array (`[int]` , `[float]` , etc.); the array object is passed through unboxed
Logical NOT	`!expr` where expr is a bool or bool-typed local
String char reads	`s[i]` and `s[constant]` where `s: string` (returns codepoint)
String char writes	`s[i] = expr` and `s[constant] = expr` where `s: string`
Array length	`len(a)` where `a` is any array or string
Foreach - int/char/bool array	`foreach (v in a)` and `foreach (v, i in a)` where `a: [int]` / `[char]` / `[bool]` ; element is unboxed
Foreach - float array	`foreach (v in a)` where `a: [float]` ; element is unboxed double
Foreach - string	`foreach (c in s)` where `s: string` ; yields unboxed codepoints
Math builtins	`Math.Sin` , `Math.Sqrt` , etc. - see table below
Char builtins	`Char.IsAlpha` , `Char.ToUpper` , etc. - see table below
Math int builtins	`Math.BitCount` , `Math.LeadingZeros` , `Math.TrailingZeros` , `Math.BitLength`
String comparison	`==` `!=` `<` `<=` `>` `>=` on strings (variables, literals, string fields of `this`) - exact interpreter semantics
String concatenation	`a + b + …` on strings, as a local initializer, return value, or builtin-call argument
String literal returns	`return "text"` in string-returning functions
Block (buffer) access	`b[i]` reads/writes on `block` params/locals; `Buffer.ReadU8At` -family via descriptor rows
Buffer/array creation	`let b = Buffer.Create(n, w)` / `Array.Create(n, Type.X)` as local initializer; the result may be returned
Stdlib builtin calls	Every builtin marked ✓ in the R8 `JIT` column (pure or JIT-safe, scalar or owned-object result)
Class fields	`this.<field>` reads/writes for scalar fields of the method's own class (declared `let x: int = …`)
Method calls	`this.method(…)` , `ClassName.Static(…)` , `super.method(…)` to eligible scalar-returning callees
Optional parameters	`k?: int` params; `k == nil` folds to a constant on the native path
Void procedures	Functions with no value returns compile and produce `nil` at the boundary

Out-of-bounds array or string accesses in JIT-compiled functions return `nil` safely.

`switch` in JIT functions - requirements and limitations:

- The switch expression must have a static type of `int` , `bool` , or `char`
- All `case` values must be integer, bool, or char literals - computed case expressions disqualify the function
- Multiple values per case use stacked `case` labels: `case 1: case 2: body` (not `case 1, 2:`)
- `break` inside a case body works correctly
- Fallthrough (no `break` between cases) is supported
- Nested `break` (e.g., `break` inside an `if` inside a `case`) is **not** supported in the JIT - it will be emitted but will not break out of the enclosing switch

> **Fiber scheduling:** a JIT-compiled function runs as one native call and cannot be preempted mid-call. Every loop back-edge decrements a scheduling counter; on expiry (every 10,000 iterations) the JIT pumps pending IO and flags the fiber so it yields as soon as the native call returns. Async work keeps flowing during long numeric kernels, but a single very long JIT call still occupies the CPU until it finishes.

> **Operator precedence note:** In Flaris, bitwise `&` has higher precedence than comparison operators (`>=`, `<=`, etc.). Use `&&` (logical AND) to combine comparison results: `x >= lo && x <= hi`.

Math builtins in JIT functions

JIT functions may call the following `Math.*` functions directly, at native speed (no interpreter overhead). All take and return `float`:

Call	Meaning
`Math.Sqrt(x)`	Square root
`Math.Cbrt(x)`	Cube root
`Math.Sin(x)` `Math.Cos(x)` `Math.Tan(x)`	Trigonometric (radians)
`Math.Asin(x)` `Math.Acos(x)` `Math.Atan(x)`	Inverse trigonometric
`Math.Atan2(y, x)`	Angle of the point `(x, y)`
`Math.Sinh(x)` `Math.Cosh(x)` `Math.Tanh(x)`	Hyperbolic
`Math.Asinh(x)` `Math.Acosh(x)` `Math.Atanh(x)`	Inverse hyperbolic
`Math.Exp(x)`	e to the power x
`Math.Expm1(x)`	`Exp(x) - 1`, accurate for small x
`Math.Pow(x, y)`	x to the power y
`Math.Log(x)`	Natural logarithm
`Math.Log10(x)` `Math.Log2(x)`	Base-10 / base-2 logarithm
`Math.Log1p(x)`	`Log(1 + x)`, accurate for small x
`Math.LogBase(x, base)`	Logarithm of x in the given base
`Math.AbsF(x)`	Absolute value
`Math.Floor(x)` `Math.Ceil(x)` `Math.Round(x)` `Math.Trunc(x)`	Rounding
`Math.Hypot(x, y)`	$\sqrt{x^2 + y^2}$ without overflow
`Math.MinF(x, y)` `Math.MaxF(x, y)`	Smaller / larger of two floats
`Math.IEEERemainder(x, y)`	IEEE floating-point remainder
`Math.CopySign(x, y)`	Magnitude of x with the sign of y
`Math.Deg(x)` `Math.Rad(x)`	Radians→degrees / degrees→radians
`Math.Fract(x)`	Fractional part, `x - Floor(x)`
`Math.Clamp01(x)`	Clamp to the range `[0, 1]`
`Math.Sech(x)` `Math.Csch(x)`	`1/Cosh(x)` / `1/Sinh(x)`
`Math.Cot(x)` `Math.Csc(x)` `Math.Sec(x)`	`1/Tan(x)` / `1/Sin(x)` / `1/Cos(x)`
`Math.Acot(x)` `Math.Acoth(x)`	Inverse cotangent / inverse hyperbolic cotangent
`Math.Gamma(x)` `Math.LnGamma(x)`	Gamma Γ(x) / log-gamma
`Math.Erf(x)` `Math.Erfc(x)`	Error function / complementary error function

These always return a `float` and follow IEEE semantics (see the *Math* module below): domain errors return `NaN`, poles return `±Inf`. The fused form `a * b + c` is also recognised and optimised.

```
fn gaussian(x: float, mu: float, sigma: float): float {
    let d: float = (x - mu) / sigma;
    return Math.Exp(-0.5 * d * d) / (sigma * Math.Sqrt(6.283185307));
}
```

Math integer intrinsics in JIT functions

The following `Math.*` functions take and return `int` and run at native speed:

Call	Description
`Math.BitCount(n)`	Number of set bits (popcount)
`Math.LeadingZeros(n)`	Count of leading zero bits (64-bit)
`Math.TrailingZeros(n)`	Count of trailing zero bits (64-bit)
`Math.BitLength(n)`	Bit length: floor(log2(abs(n))) + 1, or 0 for 0
`Math.Abs(n)`	Integer absolute value (int arg only)
`Math.Sign(n)`	-1 / 0 / 1 (int arg only)
`Math.Min(a, b)`	Integer minimum
`Math.Max(a, b)`	Integer maximum

`Math.Abs` / `Sign` / `Min` / `Max` are `int|float -> int` in general; the native path applies only when **all** arguments are `int`. A call with a float argument runs on the interpreter. `Math.Lerp(a, b, t)` (float) also runs natively, computed as `a + (b - a) * t` with the same rounding as the interpreter.

Char builtins in JIT functions

JIT functions may call the following `Char.*` functions directly, at native speed:

Call	Description	Returns
`Char.IsAlpha(c)`	True if `c` is an alphabetic character	`bool`
`Char.IsDigit(c)`	True if `c` is a decimal digit	`bool`
`Char.IsAlNum(c)`	True if `c` is alphanumeric	`bool`
`Char.IsUpper(c)`	True if `c` is uppercase	`bool`
`Char.IsLower(c)`	True if `c` is lowercase	`bool`
`Char.IsSpace(c)`	True if `c` is whitespace	`bool`
`Char.IsPunct(c)`	True if `c` is punctuation (ASCII)	`bool`
`Char.IsXDigit(c)`	True if `c` is a hex digit (ASCII)	`bool`
`Char.IsPrint(c)`	True if `c` is printable	`bool`
`Char.IsControl(c)`	True if `c` is a control character	`bool`
`Char.IsAscii(c)`	True if `c` < 128	`bool`
`Char.IsCased(c)`	True if `c` has case (upper/lower)	`bool`
`Char.IsNewline(c)`	True if `c` is a line-ending character	`bool`
`Char.ToUpper(c)`	Uppercase mapping of `c`	`char`
`Char.ToLower(c)`	Lowercase mapping of `c`	`char`
`Char.SwapCase(c)`	Toggle case of `c`	`char`
`Char.Utf8Len(c)`	Number of UTF-8 bytes needed to encode `c`	`int`
`Char.DigitValue(c)`	Numeric digit value of `c` (0–9, -1 if not a digit)	`int`
`Char.Code(c)`	Unicode codepoint of `c`	`int`
`Char.FromInt(n)`	Char for codepoint `n` (`n` masked to 32 bits)	`char`

Parameters and return values are unboxed `char` or `int` inside the JIT. The function signature must use `char` or `int` as appropriate.

Block indexing and memory access in JIT functions

JIT functions may index a `block` parameter and call a small set of memory/buffer functions directly, with the same bounds checks and results as the interpreter:

Construct	Meaning
`b[i]` (read)	Signed element read of block `b` (honours the block's element size)
`b[i] = v` (write)	Truncated element write

Construct	Meaning
`Memory.Read8/16/32/64(addr[, off])`	Unsigned integer at a **raw integer** address, native byte order (requires `--unsafe`)
`Memory.Write8/16/32/64(addr, val[, off])`	Store an integer at a raw integer address (requires `--unsafe`)
`Memory.Swap16/32/64(v)`	Byte-order (endianness) swap of a plain integer
`Memory.Compare(a, b, len)`	Compare `len` bytes at two **raw integer** addresses, like `memcmp` (requires `--unsafe`)
`Memory.ReadFloat32/64(addr[, off])`	Read an IEEE float/double at a raw integer address (requires `--unsafe`)
`Memory.WriteFloat32/64(addr, val[, off])`	Store an IEEE float/double at a raw integer address (requires `--unsafe`)
`Buffer.ReadU8/16/32/64At(buf, off)`	Little-endian unsigned read from a block; `0` if out of range
`Buffer.WriteU8/16/32/64At(buf, off, v)`	Little-endian bounded store into a block; returns `bool`

Two constraints apply: `Memory.*` runs natively only when the address argument is a plain `int` (passing a `block` / `string` / `pointer` uses the interpreter), and the `Buffer.Read/Write` calls run natively in their little-endian form (the default) — passing the optional big-endian flag falls back to the interpreter.

```
fn count_upper(s: string, n: int): int {
    let count: int = 0;
    let i: int = 0;
    while (i < n) {
        if (Char.IsUpper(s[i])) { count = count + 1; }
        i++;
    }
    return count;
}
```

JIT fast-call for callbacks

When a JIT-compiled function is passed as a callback to one of the builtins below, the builtin runs it natively per element instead of through the bytecode interpreter. This eliminates the per-element interpreter overhead for high-throughput loops.

Array methods - callback receives the element (and index where noted):

Method	Callback signature
`Array.ForEach(arr, fn)`	`fn(elem: T): any`
`Array.Select(arr, fn)`	`fn(elem: T): U`
`Array.Where(arr, fn)`	`fn(elem: T): bool`
`Array.Reduce(arr, fn, init)`	`fn(acc: T, elem: T): T`
`Array.Find(arr, fn)`	`fn(elem: T): bool`
`Array.Count(arr, fn)`	`fn(elem: T): bool`
`Array.Process(arr, fn)`	`fn(elem: T): T` - mutates in place
`Array.ProcessIdx(arr, fn)`	`fn(elem: T, idx: int): T` - mutates in place
`Array.SortBy(arr, fn)`	`fn(elem: T): key`
`Array.Sort(arr, fn)`	`fn(a: T, b: T): bool` - comparator; return true if a < b

Other builtins:

Method	Callback signature
`Memory.Process(addr, count, size, fn)`	`fn(val: int, idx: int): int` - rewrites each element
`Buffer.ProcessCallback(fn, buf, len, blocksize, cb)`	`fn(val: int, idx: int): int` - rewrites each element

The fast path is selected automatically at runtime unless `--jit-disable` is given, when the passed function is JIT-eligible. Otherwise (`--jit-disable`, or the function isn't eligible) the builtin falls back to the normal interpreter path transparently.

```
fn brighten(v: int, idx: int): int {
    // Scale a sample and clamp - a typed, allocation-free body, so it lowers.
    let x: int = v * 2;
    if (x > 255) { x = 255; }
    return x;
}

let buf = Buffer.Create(1024, 1);
Memory.Process(Buffer.GetAddress(buf), 1024, 1, brighten);  // native per-element loop
```

Calling convention and memory

JIT-compiled functions share the same call sites as interpreter functions - callers use the normal path. The compiled function pointer is stored and invoked directly once available.

Machine code is allocated with `mmap`/`mprotect` (no entitlement required on macOS).

Full workflow example

```
# 1. Compile with JIT IR embedded
flarisvm --compile myapp.fls myapp.flx

# 2. Verify which functions are eligible (structural check)
flarisvm --verbose myapp.fls

# 3. Run with JIT active
flarisvm --exec myapp.flx

# 4. Run without JIT (fallback to bytecode interpreter, even if JIT IR present)
flarisvm --exec myapp.flx

# 5. Source file directly: compile+run in one step with JIT
flarisvm myapp.fls
```

Benchmark reference

Workload	JIT	Interpreter	Speedup
Gaussian sum to 10 M	~9.5 ms	~63 ms	6.6×

Workload	JIT	Interpreter	Speedup
sum_range(10 000) × 10 000 reps	~54 ms	~606 ms	11×
count_down(10 000) × 10 000 reps	~53 ms	~954 ms	18×

Results are wall-clock time on a single fiber. Workloads with tight integer loops show the largest gains; workloads dominated by string or object operations are unaffected.

R12 - Static Analyzer

Between the parser and the optimizer, every compile runs a ten-pass static analyzer over the AST. It resolves names and types, reports problems, and decorates the tree with the facts the optimizer, code generator and JIT depend on (`resolved_fn_decl` drives inlining, purity stamps drive LICM, JIT eligibility drives native compilation).

The analyzer runs on every compile - `flarisvm file.fls`, `--compile`, `VM.Eval`, and module loading. It is not a separate lint step and cannot be skipped.

Diagnostic format

```
❌ Error: file.fls:12:5: 1000: Operator '-' does not accept string as its left operand.
⚠️ Warning: file.fls:19:9: 2000: Local 'tmp' is never read (prefix with '_' to suppress).
```

`file:line:column: code: message` - all diagnostics go to **stderr**. The summary lines (`Errors detected: N`) go to stdout.

Diagnostics are collected during analysis and emitted once, sorted by source position. Output is therefore **reproducible**: the same input produces byte-identical output on every run, which makes it safe to diff build logs or assert on compiler output in tests.

A compile stops after **100 errors** and stops recording after **200 warnings**; the remainder are counted and summarised. Compilation fails if any error was reported. Warnings never fail a compile.

Text interpolated into a diagnostic (identifiers, string-literal contents) has its control bytes escaped as `\xNN`, so source containing terminal escape sequences cannot repaint the terminal or forge compiler output.

Error codes (1000+)

Errors cannot be suppressed; the **Name** column is what appears in JSON diagnostics.

Code	Name	Symbol	Meaning
1000	`type-error`	`TYPE_ERROR`	Operand, assignment or argument type mismatch
1001	`arg-count`	`ARG_COUNT`	Wrong number of arguments to a function, method or constructor
1002	`invalid-return`	`INVALID_RETURN`	A path can finish without returning a declared value, or `return;` where a value is required
1003	`undefined-symbol`	`UNDEFINED_SYMBOL`	Name does not resolve - undefined variable, function, export or base class
1004	`redeclaration`	`REDECLARATION`	Name already declared in this scope
1005	`const-assign`	`CONST_ASSIGN`	Write to a `const` binding, or to one of its properties or elements
1006	`invalid-control-flow`	`INVALID_CONTROL_FLOW`	`break` / `continue` outside a loop or switch, `return` outside a function, or control leaving a `finally`

Code	Name	Symbol	Meaning
1007	`invalid-throw`	`INVALID_THROW`	Thrown value is not an `Exception` (or subclass) instance
1008	—	`INVALID_CAST`	*Reserved* - a cast is an assertion and is not checked
1009	—	`IMPORT_RESOLUTION`	*Reserved* - see Reserved codes
1010	`class-this-usage`	`CLASS_THIS_USAGE`	`this` used outside an instance method
1011	`async-await-misuse`	`ASYNC_AWAIT_MISUSE`	`await` outside an `async` function, or applied to a non-fiber
1012	`missing-entry`	`MISSING_ENTRY`	The program has neither `fn Main` nor any `export`
1013	`invalid-assign`	`INVALID_ASSIGN`	Assignment used where a value is required (e.g. `if (x = 1)`)
1014	`undeclared-field`	`UNDECLARED_FIELD`	Member is not declared on a sealed instance - unknown field or method
1015	`circular-inheritance`	`CIRCULAR_INHERITANCE`	A class extends itself through its base chain
1016	`missing-receiver`	`MISSING_RECEIVER`	An instance method is called through the class name - call it on an instance, or declare it `fn static`

Warning codes (2000+)

The **Name** column is the exact spelling accepted by `-Wno-<name>` and `// flaris-ignore[<name>]`, and the one emitted in JSON diagnostics. It is not always a mechanical kebab-casing of the symbol - `2002`, `2003` and `2010` differ - so copy it from this column rather than deriving it.

Code	Name	Symbol	Meaning
2000	`unused-symbol`	`UNUSED_SYMBOL`	Declared but never read - local, parameter, global or import
2001	`possible-nil-flow`	`POSSIBLE_NIL_FLOW`	A `guard` expression may be nil
2002	`float-eq`	`FLOAT_EQ_SUGGEST_APPROX`	Exact `==` / `!=` on two floats; use the approximate operator `≈`
2003	`oob-index`	`OOB_INDEX_POSSIBLE`	Constant index outside the bounds of an array literal
2004	`infinite-loop`	`INFINITE_LOOP`	Constant-true loop with no `break`, `return` or `throw`
2005	—	`RECURSION_NO_BASE`	*Reserved* - no recursion analysis is implemented
2006	`unreachable-code`	`UNREACHABLE_CODE`	Statement after a `return`, `throw`, `break` or `continue`
2007	—	`STYLE`	*Reserved* - no style rules are implemented
2008	`shadowed-variable`	`SHADOWED_VARIABLE`	Declaration hides one in an enclosing scope
2009	`empty-block`	`EMPTY_BLOCK`	Empty function, `if`, `else`, loop, `try`, `catch` or `finally` body
2010	`type-change`	`TYPE_ERROR`	Assignment changes a variable's declared type
2011	`no-effect`	`NO_EFFECT`	Expression statement with no effect (e.g. a bare literal)
2012	`class-reserved-name`	`CLASS_RESERVED_NAME`	Method shadows a built-in instance property and can never be called
2013	`duplicate-key`	`DUPLICATE_KEY`	Repeated key in an object literal - the first value is kept
2014	`duplicate-case`	`DUPLICATE_CASE`	Repeated `case` value - only the first match runs
2015	`override-mismatch`	`OVERRIDE_MISMATCH`	An override takes a different number of parameters than the method it replaces
2016	`class-shadows-module`	`CLASS_SHADOWS_MODULE`	A non-static method is named after a built-in namespace *and* the class calls that namespace - the call resolves to the method and yields `nil`

Every warning above (the *Reserved* codes excepted) is suppressible; **no error is**. A raw number works anywhere a name does, e.g. `-Wno-2000`.

Reserved codes

Codes marked *Reserved* are never emitted. Their numbers stay retired rather than being reused, so tooling that matches on a code can rely on it never changing meaning.

`1009 IMPORT_RESOLUTION` is reserved rather than implemented because telling "this module does not exist" apart from "this module exports nothing" needs a resolver with no side effects, and the runtime's module resolver performs network fetches and version-pin enforcement - neither is acceptable during a compile. An unresolvable import is reported by the runtime loader instead.

Suppressing an unused-symbol warning

Prefix the name with `_`. This works for **locals, parameters and globals**:

```
fn handler(_event, payload) {   // _event is intentionally unused
    let _debugOnly = payload;   // kept for inspection, never read
    return payload;
}
```

Loop variables and `catch` variables are exempt automatically - iterating or catching without using the value is normal.

Suppressing a specific warning

`// flaris-ignore[<name>]` silences the listed warnings. A pragma trailing a line of code applies to that line; one on a line of its own applies to the next line, so it can sit above what it refers to:

```
global buildStamp = 1;   // flaris-ignore[unused-symbol]

// flaris-ignore[shadowed-variable, unused-symbol]
let value = compute();
```

With no bracket list (`// flaris-ignore`) it silences every warning on the target line. Names are the **Name** column of the warning table above; a raw number (`2000`) also works.

Warnings only. There is deliberately no way to suppress an error: an error means the program is invalid and would not survive code generation.

Warning flags

Flag	Effect
`-Wno-<name>`	Silence one warning across the whole compile, e.g. `-Wno-unused-symbol`. `<name>` is the **Name** column of the warning table above (or its raw number). An unknown or non-suppressible name is a hard error, so a typo cannot silently do nothing.
`-Werror`	Any warning that survives suppression fails the compile
`-w`	Silence all warnings

`-Wno-` rejects an unknown name, and refuses error codes, rather than silently doing nothing. All three flags also apply to `VM.Eval` and `VM.Compile`.

Machine-readable diagnostics

`--diagnostics json` writes a single JSON array to **stdout** instead of the human-readable form, for editors and CI:

```
flarisvm --compile app.fls app.flx --diagnostics json
```

```
[
  {"severity":"warning","code":2000,"name":"unused-
symbol","file":"app.fls","line":1,"column":1,"message":"Global 'unusedOne' is never read."},
  {"severity":"error","code":1000,"name":"type-
error","file":"app.fls","line":4,"column":29,"message":"Operator '-' does not accept string as its
left operand."}
]
```

In this mode stdout carries nothing but the array - the usual summary and fingerprint lines are suppressed so the output can be piped straight into a parser. Diagnostics keep their sorted order.

Suggestions

When a name does not resolve, the analyzer offers the closest declared name within a short edit distance:

```
1003: 'totl' is not defined. Did you mean 'total'?
1014: Class 'Counter' has no member 'cont'. Did you mean 'count'?
```

Nothing is suggested when no candidate is close enough. Ties are broken by name, so the suggestion is reproducible.

Control flow rules

- `break` and `continue` require an enclosing loop; `break` also accepts an enclosing `switch`.
- `return` requires an enclosing function.
- Control may **not** leave a `finally` body via `return`, `break` or `continue` (matching C# CS0157). A `throw` from a `finally` is allowed, and a `break` / `continue` targeting a loop opened *inside* the `finally` is fine.
- A function with a declared return type must return on every path. Falling off the end would yield nil, which a typed caller cannot detect.

These rules apply inside **function literals** exactly as they do in named functions - a lambda body is its own control-flow scope.

Operator operand checking

Arithmetic and bitwise operators are checked against the operand types the VM actually accepts:

Operators	Accepted operands
`-` `*` `/` `%` `^^`	`int`, `float`, `bool`, `char`
`&` `|` `^` `<<` `>>`	`int`, `char`, `bool` - a `float` operand is an error, not a widening
`~`	`int`

Operators	Accepted operands
`+`	**anything** - see below

`+` is never a type error: it is overloaded by operand type (numeric add, char and container concatenation/merge), and when no numeric, array or object rule applies it falls back to string concatenation, so any operand pair produces a value (`nil + 5` is `"null5"`). The full per-type rules are listed under R5 - Operator Precedence.

A diagnostic is only reported when an operand's type is fully known and shares nothing with the operator's domain. Dynamic values - unannotated parameters, `any`, results of dynamic calls - are never diagnosed.

Bitwise and shift operators always produce an `int`, whatever the operands infer to, because the VM refuses to run them on the float lane at all.

Class checks

Instances are sealed: the member set is fixed by the class body (see Sealed instance fields). Where the analyzer can prove which class a value belongs to, it checks member access against that class - own members and inherited ones:

```
class Counter {
    let count = 0;
    fn Constructor(start) { this.count = start; }
    fn Bump(step) { this.count = this.count + step; }
}

let c = new Counter(1, 2, 3);   // 1001 – Constructor expects 0–1 arguments
c.Nope();                       // 1014 – Counter has no method 'Nope'
Console.WriteLine(c.missing);   // 1014 – Counter has no member 'missing'
```

A receiver's class is known when it is `this`, a local initialised by `new C()`, or a parameter annotated with a class type (`fn f(v: Vec2)`). Any other receiver is dynamic and is not checked.

Base classes may be declared in any order - `class Dog : Animal` compiles with `Animal` declared later in the file, just as calling a function declared later does.

A class whose base is imported from a compiled module is **lenient**: its full member set is not visible, so no membership check fires for it and the VM's runtime guard applies instead.

Limitations

- **Argument minimums are a lower bound.** An unannotated parameter is indistinguishable from an optional one once parsed, so a missing argument on an unannotated parameter is not reported. Passing too many is always caught.
- **Typed arrays and unions share bits.** `[string]` and the union `array|string` have the same internal representation; an ambiguous type is read as a typed array.
- **Member checks need a provable receiver.** Chained access (`a.b.Method()`) and values from dynamic sources are not checked.

- **No nil-narrowing.** `if (x != nil)` does not narrow `x` in the branch. Expression types are memoized per AST node, so a variable has one type throughout a function; flow-sensitive narrowing would need that layer replaced.
- **No exhaustiveness checking for `switch`** over an enum.

Definite-assignment analysis is not needed: `let x;` without an initializer is a syntax error, so every variable is assigned where it is declared.